

目录

数学部分索引	4
从题目结构找到建模方法	4
数学技巧与易错点	5
整除分块	5
0/1 分数规划与 WQS 二分	6
Lagrange-Bürmann 反演	6
树上背包复杂度	6
64 位 Miller-Rabin 素性测试	6
模质数意义下的行列式	7
CDQ 分治卷积	8
原根	9
Pollard-Rho 质因数分解	11
BSGS 与扩展 BSGS	12
拉格朗日插值	14
扩展 Lucas: 组合数模合数	15
快速沃尔什变换	18
子集卷积	19
单调斜率优化	21
少项式技巧	22
模意义下的函数迭代	22
少项式的形式幂级数幂	23
生成函数与 Bostan-Mori	25
线性递推的普通生成函数	25
常见陷阱	26
Lucas 定理	26
NTT 与形式幂级数基础版	27
前置条件与复杂度	31
组合数学基础	32
阶乘、组合数与排列数	32
Stirling 数	34
递推、换基与生成函数	34
模 998244353 快速计算	34
线性筛与因数卷积	36
线性筛	36
约数/倍数 Zeta 与 Möbius 变换	37

线性递推	39
已知递推求第 n 项	39
Berlekamp-Massey	40
生成函数基础与建模技巧	42
OGF 与 EGF	42
重复选择时, EGF 自动统计时间顺序	42
用单项式记录信息, 最后施加线性映射	43
为什么这不是普通的“给 x 代入一个数”	43
先消去常数项, 再消去无限时间	44
指数换元: 把重复抽样变成有限多项式	44
从最终状态到首次达到目标	45
稀疏因子与一次标记	45
二元多项式压成一元卷积	46
标记变量与阶乘矩	46
从递推到有理生成函数	46
系数提取技巧	47
两个建模例子	47
结构化万能欧几里得	47
通用模板	48
为什么递归成立	49
示例: 怎样从目标式设计结构体	49
第一步: 规定每个字段的局部含义	49
第二步: 推导左右段的合并式	50
第三步: 写空串与两个原子	50
第四步: 把推导翻译成结构体	50
如何迁移到别的题	52
完整多项式与形式幂级数库	52
方法、推导与复杂度	53
NTT 与卷积	53
FPS Newton 迭代的一般形式	53
FPS 逆: 牛顿迭代从哪里来	53
求导、积分与形式对数	54
FPS 指数: 对 $\ln f - h = 0$ 做牛顿迭代	54
FPS 幂	54
FPS 平方根	55
多项式除法	55
多点求值	56
快速插值	56
Bostan-Mori	56
Taylor shift	56
多项式复合	56
转置原理: 乘法、除法和复合的转置	57
幂投影: 复合转置的专用实现	59
复合逆	59
完整实现	59
计数公式、反演与生成函数速查	81
组合数与二项式	82
基本变形	82
二项式定理与广义二项式	82
Hockey-stick 与 Vandermonde	82
高频求和	83

插板、上下界与不相邻选择	83
容斥原理	84
Catalan、反射原理与 Ballot	85
Fibonacci 恒等式	85
Stirling 数与幂的换基	86
常见反演	86
二项式反演	86
Möbius 反演	87
子集 Zeta/Möbius	87
单位根过滤	87
卷积与变换该选哪一种	87
常见对象的直接计数	88
排列、多重集与圆排列	88
标号树与 Prüfer 序列	88
盒子模型速查	88
生成函数操作字典	89
对数导数、Euler 变换与 Newton 恒等式	89
没有 NTT 友好模数时	90
整数分拆与五边形数	90
Burnside 与 Pólya	92
推式子的常用检查方法	92
参考	93
高斯消元：秩、线性方程组与逆矩阵	93
浮点版本	93
模质数版本	95
异或线性基	98
哪些输入元素是必须基	99
线性同余方程组与 exCRT	100
为什么这样化简	103
合并与边界	103
杜教筛	104
Min_25 筛：Euler ϕ 前缀和实例	106
怎样改成其他积性函数	108
洲阁筛 (Lucy DP)	108
GF(2) 高斯消元	110
Sprague-Grundy 定理与 SG 打表	111
打表思路	111
数论常用定理速查	112
Bézout、Euler、Fermat 与 Wilson	112
阶乘和组合数中的素因子	112
LTE (指数提升)	112
二次剩余与二次互反	113
概率与期望建模	113
看到什么结构，先想什么方法	113
条件概率、全概率与独立性	114
期望线性性：把答案拆成贡献	114

全期望与全方差	114
尾和公式：等待时间转成“还没结束”	114
高频分布与等待阶段	115
收集问题与 Min-Max 容斥	115
联合成功概率、二项式矩与阈值等待时间	116
容斥的另一层：从“指定若干个成功”恢复“总共成功几个”	116
两层容斥如何在重复抽样中配合	116
先减去极限，才能对时间求和	117
直接还原第 p 个任务完成的期望	118
首次转移分析：自环、环与二阶矩	118
协方差与二阶矩	119
概率生成函数与随机和	119
泊松化与 EGF 的联系	120
概率界与概率法	120
模意义下的概率与期望	120

数学部分索引

赛场上按需求查找：

需求	文件
阶乘、组合数、常见恒等式	17-combinatorics.md、24-counting-formulas.md
Stirling 数、幂与下降幂换基	18-stirling.md
OGF/EGF 建模与系数提取	21-generating-functions.md、 24-counting-formulas.md
重复抽样的顺序计数、指数换元、一次标记	21-generating-functions.md
单项式记录总权重、线性映射、容斥多项式	21-generating-functions.md
联合成功概率、二项式矩、阈值等待时间反演	34-probability-expectation.md、 24-counting-formulas.md
二元卷积与 Kronecker 代换	21-generating-functions.md；代码见 23-polynomial-advanced.md 的 BiPoly
概率、期望、方差、尾和、收集与吸收过程	34-probability-expectation.md
PGF、随机和、泊松化、Chernoff 界	34-probability-expectation.md
分拆、五边形数、Euler 变换、Newton 恒等式	24-counting-formulas.md
非 NTT 模数下的选择路线	24-counting-formulas.md 的“没有 NTT 友好模数 时”
完整 NTT/FPS、除法、复合与转置	23-polynomial-advanced.md
线性递推、BM、Bostan-Mori	20-linear-recurrence.md、14-分式求系数.md
反演、GCD/LCM 卷积、筛法卷积	24-counting-formulas.md、 19-sieve-convolutions.md
线性同余、CRT/exCRT	27-crt.md
数论定理与整除估值	33-number-theory-theorems.md
素性、分解、离散对数、原根	02-miller rabin.md、06-Rho.md、07-Bsgs.md、 05-原根.md
积性函数前缀和	28-dujiao-sieve.md、29-min25-sieve.md、 30-lucy-sieve.md
实数/模素数/GF(2) 线性方程	25-gaussian-elimination.md、 31-gf2-gaussian.md
异或空间	26-xor-linear-basis.md
公平组合游戏	32-sprague-grundy.md

从题目结构找到建模方法

观察到的结构	为什么能转化, 以及去哪里查
方案难枚举, 但某个元素的贡献容易算	交换“枚举方案”和“枚举贡献”的顺序; 随机问题用指示变量与期望线性性, 见 34-probability-expectation.md
重复选择, 各类只有次数限制	EGF 的阶乘分母自动统计时间位置的交错, 见 21-generating-functions.md
子集贡献只取决于权重之和	先用指数记录总权重, 再对单项式施加线性映射, 见 21-generating-functions.md
指定若干事件同时成功容易算, 但成功个数分布难算	联合概率之和是二项式矩, 用反演恢复分布或阈值概率, 见 34-probability-expectation.md
只问某个统计量的和或平方和	用标记变量的一、二阶导数, 可能不必保存完整分布, 见 21-generating-functions.md
“全部满足”很难, 但指定一些条件失败很容易	对失败集合容斥; 恰好满足若干项用二项式反演, 见 24-counting-formulas.md
出现 k^r , 直接求和困难	用 Stirling 数转成下降幂; 下降幂对应有顺序选取, 见 18-stirling.md 、 24-counting-formulas.md
求和带 gcd 或 lcm	用整除关系把条件拆开, 转换为约数或倍数求和, 见 19-sieve-convolutions.md 、 24-counting-formulas.md
旋转、翻转后的方案算相同	稳定子大小未必相同, 不能直接除群大小; 按不动点使用 Burnside, 见 24-counting-formulas.md
问递推序列极远的一项	用特征多项式降幂或有理生成函数求系数, 见 20-linear-recurrence.md 、 14-分式求系数.md
隐式方程 $F = x\phi(F)$ 描述递归结构	用 Lagrange 反演直接求系数, 见 01-tricky.md
二元下标范围都可控	用足够大的基数打包成一元卷积, 见 21-generating-functions.md

先检查模数是否为素数、所除元素是否可逆、答案是否需要完整整数, 以及卷积规模是否真的值得使用 NTT。很多计数题在这些前提不同后, 公式仍成立, 但实现方法并不相同。

数学技巧与易错点

本页只保留没有独立模板承载的技巧。GCD/LCM 卷积统一见 [19-sieve-convolutions.md](#) 和 [24-counting-formulas.md](#), Burnside-Pólya 统一见 [24-counting-formulas.md](#)。

整除分块

对固定正整数 n , $\lfloor n/i \rfloor$ 在一段连续下标上相同。已知左端点 l 时,

$$q = \left\lfloor \frac{n}{l} \right\rfloor, \quad r = \left\lfloor \frac{n}{q} \right\rfloor.$$

枚举下一段令 $l = r + 1$, 总段数为 $O(\sqrt{n})$ 。涉及向上取整时, 优先用

$$\left\lceil \frac{a}{b} \right\rceil = \left\lfloor \frac{a+b-1}{b} \right\rfloor$$

(仅适用于这里的非负整数), 避免混用左右端点公式。

0/1 分数规划与 WQS 二分

- 判定是否存在 $A/B \geq x$ ($B > 0$)，可改为判定 $A - xB \geq 0$ 。
- WQS 二分给“选择数量”附加线性代价，要求最优选择数量随参数单调。相等时统一按数量较多或较少破同值，否则二分边界可能不稳定。

Lagrange-Bürmann 反演

设 $f(0) = g(0) = 0$ 、 $f'(0)g'(0) \neq 0$ ，且 f 与 g 互为复合逆。对 $n > 0$ ，

$$[x^n]\Phi(f(x)) = \frac{1}{n}[t^{n-1}]\Phi'(t) \left(\frac{t}{g(t)} \right)^n.$$

等价的留数形式为

$$[x^n]\Phi(f(x)) = [t^{-1}] \frac{\Phi(t)g'(t)}{g(t)^{n+1}}.$$

模质数实现要同时确认 n 以及所需常数项在模意义下可逆。

树上背包复杂度

合并儿子时不要机械地把每次卷积都估成 $O(n^2)$ 。常见计数方法是把一对状态归到它们第一次相遇的 LCA；每对点只贡献一次，因此许多无额外上限的树背包总复杂度就是 $O(n^2)$ 。若状态上限为 m ，将循环边界截到子树大小与 m 的较小值，常可得到 $O(nm)$ ，但必须针对具体转移证明，不能只凭“子树大小和”为 n 。

64 位 Miller-Rabin 素性测试

这组底数对所有 `uint64_t` 都是确定性的，不是概率测试。模乘使用 `__uint128_t`，避免 64 位乘法溢出。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

namespace number_theory {

using u64 = uint64_t;
using u128 = __uint128_t;

u64 multiply_mod(u64 lhs, u64 rhs, u64 modulus) {
    return (u64)((u128)lhs * rhs % modulus);
}

u64 power_mod(u64 base, u64 exponent, u64 modulus) {
    u64 result = 1 % modulus;
    while (exponent > 0) {
        if (exponent & 1) result = multiply_mod(result, base, modulus);
        base = multiply_mod(base, base, modulus);
        exponent >>= 1;
    }
    return result;
}
```

```

bool is_prime(u64 value) {
    if (value < 2) return false;
    for (u64 prime : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                     19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
        if (value % prime == 0) return value == prime;
    }

    u64 odd_part = value - 1;
    int power_of_two = 0;
    while ((odd_part & 1) == 0) {
        odd_part >>= 1;
        ++power_of_two;
    }

    for (u64 base : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL,
                    9780504ULL, 1795265022ULL}) {
        if (base % value == 0) continue;
        u64 current = power_mod(base % value, odd_part, value);
        if (current == 1 || current == value - 1) continue;
        bool composite = true;
        for (int round = 1; round < power_of_two; ++round) {
            current = multiply_mod(current, current, value);
            if (current == value - 1) {
                composite = false;
                break;
            }
        }
        if (composite) return false;
    }
    return true;
}

} // namespace number_theory

```

复杂度为 $O(\log n)$ 次 128 位模乘。不要把待测的有符号负数直接转换为 `uint64_t`；应先在调用处排除负数。

模质数意义下的行列式

模数必须是质数。矩阵元素会先正规化到 $[0, p)$ ；使用高斯消元，复杂度为 $O(n^3)$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

long long power_mod(long long base, long long exponent, int modulus) {
    long long result = 1;
    while (exponent > 0) {
        if (exponent & 1) result = result * base % modulus;
        base = base * base % modulus;
        exponent >>= 1;
    }
}

```

```

    return result;
}

int determinant_mod_prime(vector<vector<int>> matrix, int prime) {
    int size = (int)matrix.size();
    for (const auto& row : matrix) assert((int)row.size() == size);
    for (auto& row : matrix) {
        for (int& value : row) {
            value %= prime;
            if (value < 0) value += prime;
        }
    }

    long long determinant = 1;
    for (int column = 0; column < size; ++column) {
        int pivot = column;
        while (pivot < size && matrix[pivot][column] == 0) ++pivot;
        if (pivot == size) return 0;
        if (pivot != column) {
            swap(matrix[pivot], matrix[column]);
            determinant = prime - determinant;
        }
        int pivot_value = matrix[column][column];
        determinant = determinant * pivot_value % prime;
        long long pivot_inverse = power_mod(pivot_value, prime - 2, prime);
        for (int row = column + 1; row < size; ++row) {
            long long factor = matrix[row][column] * pivot_inverse % prime;
            for (int next_column = column; next_column < size; ++next_column) {
                matrix[row][next_column] = (int)(
                    (matrix[row][next_column] -
                     factor * matrix[column][next_column]) % prime);
                if (matrix[row][next_column] < 0) {
                    matrix[row][next_column] += prime;
                }
            }
        }
    }
    return (int)determinant;
}

```

不要在合数模数下使用 Fermat 逆元。合数模数可用不除法的欧几里得消元，或分别模质数幂计算后 CRT 合并。

CDQ 分治卷积

求形式递推

$$f_i = base_i + \sum_{0 \leq j < i} f_j g_{i-j}.$$

下面代码依赖 16-polynomial.md 的 polynomial::multiply 与 mod。CDQ 保证计算右半部分前，左半部分

已经完整。

```
namespace polynomial {

void cdq_convolution(vector<int>& values, const vector<int>& kernel,
                    int left, int right) {
    if (left == right) return;
    int middle = left + (right - left) / 2;
    cdq_convolution(values, kernel, left, middle);

    vector<int> lhs(values.begin() + left, values.begin() + middle + 1);
    int kernel_size = min((int)kernel.size(), right - left + 1);
    vector<int> rhs(kernel.begin(), kernel.begin() + kernel_size);
    vector<int> product = multiply(lhs, rhs);
    for (int index = middle + 1; index <= right; ++index) {
        int product_index = index - left;
        if (product_index < (int)product.size()) {
            values[index] += product[product_index];
            if (values[index] >= mod) values[index] -= mod;
        }
    }
    cdq_convolution(values, kernel, middle + 1, right);
}

vector<int> solve_convolution_recurrence(vector<int> base,
                                        const vector<int>& kernel) {
    if (!base.empty()) {
        assert(!kernel.empty());
        assert(kernel[0] == 0);
        cdq_convolution(base, kernel, 0, (int)base.size() - 1);
    }
    return base;
}

} // namespace polynomial
```

kernel[0] 必须为 0，因为递推只允许 $j < i$ 。当前写法每层分别做 NTT，总复杂度为 $O(n \log^2 n)$ ；使用带撤销或更精细的在线卷积可以进一步优化常数或复杂度。

原根

若模 m 的乘法群是循环群，则原根 g 的阶等于 $\varphi(m)$ 。通过分解 $\varphi(m)$ ，只需检查每个不同质因子 q ：

$$g^{\varphi(m)/q} \not\equiv 1 \pmod{m}.$$

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using i128 = __int128_t;
```

```

i64 power_mod(i64 base, i64 exponent, i64 modulus) {
    i64 result = 1 % modulus;
    while (exponent > 0) {
        if (exponent & 1) result = (i64)((i128)result * base % modulus);
        base = (i64)((i128)base * base % modulus);
        exponent >>= 1;
    }
    return result;
}

vector<i64> distinct_prime_factors(i64 value) {
    vector<i64> factors;
    for (i64 prime = 2; prime <= value / prime; ++prime) {
        if (value % prime != 0) continue;
        factors.push_back(prime);
        while (value % prime == 0) value /= prime;
    }
    if (value > 1) factors.push_back(value);
    return factors;
}

i64 euler_phi(i64 value) {
    i64 result = value;
    for (i64 prime : distinct_prime_factors(value)) {
        result = result / prime * (prime - 1);
    }
    return result;
}

i64 minimum_primitive_root(i64 modulus) {
    assert(modulus >= 2);
    if (modulus == 2) return 1;
    i64 phi = euler_phi(modulus);
    vector<i64> factors = distinct_prime_factors(phi);
    for (i64 candidate = 2; candidate < modulus; ++candidate) {
        if (gcd(candidate, modulus) != 1) continue;
        bool valid = true;
        for (i64 factor : factors) {
            if (power_mod(candidate, phi / factor, modulus) == 1) {
                valid = false;
                break;
            }
        }
        if (valid) return candidate;
    }
    return -1;
}

```

存在原根的正整数恰为 $2, 4, p^k, 2p^k$ ，其中 p 是奇质数。函数在不存在时返回 -1；试除分解与枚举候选适合中等模数，64 位大模数应配合 Pollard-Rho。

Pollard-Rho 质因数分解

依赖 02-miller_rabin.md 中的 u64、multiply_mod 和 is_prime。返回结果包含重数并按升序排列。

```
namespace number_theory {

u64 pollard_rho(u64 value) {
    if (value % 2 == 0) return 2;
    if (value % 3 == 0) return 3;

    static mt19937_64 random_engine(
        chrono::steady_clock::now().time_since_epoch().count());
    auto next_value = [&](u64 current, u64 constant) {
        return (multiply_mod(current, current, value) + constant) % value;
    };

    while (true) {
        u64 current = random_engine() % (value - 2) + 2;
        u64 constant = random_engine() % (value - 1) + 1;
        u64 block_size = 128;
        u64 cycle_length = 1;
        u64 divisor = 1;
        u64 checkpoint = 0;
        u64 fallback = 0;

        while (divisor == 1) {
            checkpoint = current;
            for (u64 step = 0; step < cycle_length; ++step) {
                current = next_value(current, constant);
            }
            for (u64 begin = 0;
                begin < cycle_length && divisor == 1;
                begin += block_size) {
                fallback = current;
                u64 product = 1;
                u64 steps = min(block_size, cycle_length - begin);
                for (u64 step = 0; step < steps; ++step) {
                    current = next_value(current, constant);
                    u64 difference = checkpoint > current
                        ? checkpoint - current : current - checkpoint;
                    product = multiply_mod(product, difference, value);
                }
                divisor = gcd(product, value);
            }
            cycle_length <= 1;
        }

        if (divisor == value) {
            do {
                fallback = next_value(fallback, constant);
                u64 difference = checkpoint > fallback
                    ? checkpoint - fallback : fallback - checkpoint;
                divisor = gcd(difference, value);
            } while (divisor == 1);
        }
    }
}
```

```

    }
    if (divisor != value) return divisor;
}
}

void factor_recursively(u64 value, vector<u64>& factors) {
    if (value == 1) return;
    if (is_prime(value)) {
        factors.push_back(value);
        return;
    }
    u64 divisor = pollard_rho(value);
    factor_recursively(divisor, factors);
    factor_recursively(value / divisor, factors);
}

vector<u64> factorize(u64 value) {
    vector<u64> factors;
    factor_recursively(value, factors);
    sort(factors.begin(), factors.end());
    return factors;
}

} // namespace number_theory

```

`factorize(1)` 返回空数组。Pollard-Rho 的运行时间具有随机性；若某次多项式迭代退化，外层循环会自动换随机参数重试。

BSGS 与扩展 BSGS

求最小非负整数 x 使 $a^x \equiv b \pmod{m}$ 。扩展版本允许 $\gcd(a, m) \neq 1$ ，无解返回 -1 。下面实现约定 $1 \leq m \leq 2^{63} - 1$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

namespace discrete_logarithm {

using i64 = long long;
using i128 = __int128_t;

i64 multiply_mod(i64 lhs, i64 rhs, i64 modulus) {
    return (i64)((i128)lhs * rhs % modulus);
}

i64 power_mod(i64 base, i64 exponent, i64 modulus) {
    i64 result = 1 % modulus;
    while (exponent > 0) {
        if (exponent & 1) result = multiply_mod(result, base, modulus);
        base = multiply_mod(base, base, modulus);
        exponent >>= 1;
    }
}

```

```

    }
    return result;
}

i64 extended_gcd(i64 lhs, i64 rhs, i64& lhs_coefficient,
                i64& rhs_coefficient) {
    if (rhs == 0) {
        lhs_coefficient = 1;
        rhs_coefficient = 0;
        return lhs;
    }
    i64 next_lhs = 0;
    i64 next_rhs = 0;
    i64 divisor = extended_gcd(rhs, lhs % rhs, next_lhs, next_rhs);
    lhs_coefficient = next_rhs;
    rhs_coefficient = next_lhs - lhs / rhs * next_rhs;
    return divisor;
}

i64 modular_inverse(i64 value, i64 modulus) {
    i64 coefficient = 0;
    i64 ignored = 0;
    i64 divisor = extended_gcd(value, modulus, coefficient, ignored);
    assert(divisor == 1);
    coefficient %= modulus;
    if (coefficient < 0) coefficient += modulus;
    return coefficient;
}

i64 bsgs_coprime(i64 base, i64 target, i64 modulus) {
    i64 block = (i64)ceil(sqrtl((long double)modulus));
    unordered_map<i64, i64> baby_step;
    baby_step.reserve((size_t)block * 2 + 1);
    i64 current = 1 % modulus;
    for (i64 exponent = 0; exponent < block; ++exponent) {
        baby_step.emplace(current, exponent);
        current = multiply_mod(current, base, modulus);
    }

    i64 inverse_block =
        modular_inverse(power_mod(base, block, modulus), modulus);
    current = target;
    for (i64 block_id = 0; block_id <= block; ++block_id) {
        auto found = baby_step.find(current);
        if (found != baby_step.end()) {
            return block_id * block + found->second;
        }
        current = multiply_mod(current, inverse_block, modulus);
    }
    return -1;
}

i64 extended_bsgs(i64 base, i64 target, i64 modulus) {

```

```

assert(modulus >= 1);
base %= modulus;
target %= modulus;
if (modulus == 1 || target == 1 % modulus) return 0;

i64 removed_steps = 0;
i64 accumulated = 1 % modulus;
while (true) {
    i64 divisor = gcd(base, modulus);
    if (divisor == 1) break;
    if (target == accumulated) return removed_steps;
    if (target % divisor != 0) return -1;
    target /= divisor;
    modulus /= divisor;
    accumulated = multiply_mod(accumulated, base / divisor, modulus);
    ++removed_steps;
}

target = multiply_mod(target, modular_inverse(accumulated, modulus),
                      modulus);
i64 remaining = bsgs_coprime(base % modulus, target, modulus);
return remaining == -1 ? -1 : removed_steps + remaining;
}

} // namespace discrete_logarithm

```

时间和空间复杂度均为 $O(\sqrt{m})$ 。哈希表可能被构造数据攻击；需要更稳定常数时可把 baby step 存入数组后排序，用二分查找代替 `unordered_map`。

拉格朗日插值

给定模质数意义下互不相同的横坐标，计算次数小于点数的唯一多项式在 `target` 处的值。复杂度为 $O(n^2 + n \log p)$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

int lagrange_interpolation(const vector<int>& x, const vector<int>& y,
                          int target, int prime) {
    assert(x.size() == y.size());
    int size = (int)x.size();
    target %= prime;
    if (target < 0) target += prime;

    auto power_mod = [&](long long base, int exponent) {
        long long result = 1;
        while (exponent > 0) {
            if (exponent & 1) result = result * base % prime;
            base = base * base % prime;
            exponent >>= 1;
        }
    };
}

```

```

        return (int)result;
    };

    long long answer = 0;
    for (int point = 0; point < size; ++point) {
        int normalized_x = (x[point] % prime + prime) % prime;
        if (normalized_x == target) return (y[point] % prime + prime) % prime;
        long long numerator = 1;
        long long denominator = 1;
        for (int other = 0; other < size; ++other) {
            if (other == point) continue;
            int other_x = (x[other] % prime + prime) % prime;
            numerator = numerator * (target - other_x + prime) % prime;
            denominator =
                denominator * (normalized_x - other_x + prime) % prime;
        }
        assert(denominator != 0);
        long long normalized_y = (y[point] % prime + prime) % prime;
        answer += normalized_y * numerator % prime *
            power_mod(denominator, prime - 2) % prime;
        answer %= prime;
    }
    return (int)answer;
}

```

若横坐标恰为连续的 $0, 1, \dots, n-1$ ，可预处理阶乘、逆阶乘和目标点的前后缀积，将单次求值降为 $O(1)$ 。

扩展 Lucas：组合数模合数

计算 $\binom{n}{k} \bmod m$ ，其中 m 不要求为质数。先把 m 分解成若干互质的质数幂，对每个 p^q 计算答案，再用 CRT 合并。

该实现需要为每个质数幂分配 $O(p^q)$ 空间，适合模数不大的竞赛题；若某个质数幂很大，应换分块乘积或题目特定算法。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

namespace composite_binomial {

using i64 = long long;
using i128 = __int128_t;

i64 multiply_mod(i64 lhs, i64 rhs, i64 modulus) {
    return (i64)((i128)lhs * rhs % modulus);
}

i64 power_mod(i64 base, i64 exponent, i64 modulus) {
    i64 result = 1 % modulus;
    while (exponent > 0) {
        if (exponent & 1) result = multiply_mod(result, base, modulus);
        base = multiply_mod(base, base, modulus);
    }
}

```

```

        exponent >>= 1;
    }
    return result;
}

i64 extended_gcd(i64 lhs, i64 rhs, i64& lhs_coefficient,
                i64& rhs_coefficient) {
    if (rhs == 0) {
        lhs_coefficient = 1;
        rhs_coefficient = 0;
        return lhs;
    }
    i64 next_lhs = 0;
    i64 next_rhs = 0;
    i64 divisor = extended_gcd(rhs, lhs % rhs, next_lhs, next_rhs);
    lhs_coefficient = next_rhs;
    rhs_coefficient = next_lhs - lhs / rhs * next_rhs;
    return divisor;
}

i64 modular_inverse(i64 value, i64 modulus) {
    i64 coefficient = 0;
    i64 ignored = 0;
    i64 divisor = extended_gcd(value, modulus, coefficient, ignored);
    assert(divisor == 1);
    coefficient %= modulus;
    if (coefficient < 0) coefficient += modulus;
    return coefficient;
}

class PrimePowerBinomial {
    i64 prime;
    i64 modulus;
    vector<i64> prefix_product;

    i64 prime_exponent_in_factorial(i64 value) const {
        i64 exponent = 0;
        while (value > 0) {
            value /= prime;
            exponent += value;
        }
        return exponent;
    }

    i64 factorial_without_prime(i64 value) const {
        if (value == 0) return 1;
        i64 complete_blocks =
            power_mod(prefix_product[modulus], value / modulus, modulus);
        i64 remainder = prefix_product[value % modulus];
        return multiply_mod(
            multiply_mod(complete_blocks, remainder, modulus),
            factorial_without_prime(value / prime), modulus);
    }
}

```



```

public:
    PrimePowerBinomial(i64 prime_factor, i64 prime_power)
        : prime(prime_factor), modulus(prime_power),
          prefix_product(prime_power + 1, 1) {
        for (i64 value = 1; value <= prime_power; ++value) {
            prefix_product[value] = prefix_product[value - 1];
            if (value % prime_factor != 0) {
                prefix_product[value] =
                    multiply_mod(prefix_product[value], value, prime_power);
            }
        }
    }

    i64 choose(i64 n, i64 k) const {
        if (k < 0 || k > n) return 0;
        i64 exponent = prime_exponent_in_factorial(n) -
            prime_exponent_in_factorial(k) -
            prime_exponent_in_factorial(n - k);
        i64 numerator = factorial_without_prime(n);
        i64 denominator = multiply_mod(
            factorial_without_prime(k),
            factorial_without_prime(n - k), modulus);
        i64 result = multiply_mod(
            numerator, modular_inverse(denominator, modulus), modulus);
        return multiply_mod(result, power_mod(prime, exponent, modulus),
            modulus);
    }

};

i64 choose_mod_composite(i64 n, i64 k, i64 modulus) {
    assert(n >= 0 && modulus >= 1);
    if (modulus == 1) return 0;

    vector<pair<i64, i64>> congruences;
    i64 remaining = modulus;
    for (i64 prime = 2; prime <= remaining / prime; ++prime) {
        if (remaining % prime != 0) continue;
        i64 prime_power = 1;
        while (remaining % prime == 0) {
            remaining /= prime;
            prime_power *= prime;
        }
        PrimePowerBinomial calculator(prime, prime_power);
        congruences.push_back({calculator.choose(n, k), prime_power});
    }
    if (remaining > 1) {
        PrimePowerBinomial calculator(remaining, remaining);
        congruences.push_back({calculator.choose(n, k), remaining});
    }

    i64 result = 0;
    i64 current_modulus = 1;
    for (auto [remainder, next_modulus] : congruences) {

```

```

        i64 difference = (remainder - result) % next_modulus;
        if (difference < 0) difference += next_modulus;
        i64 multiplier = multiply_mod(
            difference,
            modular_inverse(current_modulus % next_modulus, next_modulus),
            next_modulus);
        result = (i64)((i128)current_modulus * multiplier + result);
        current_modulus *= next_modulus;
        result %= current_modulus;
    }
    return result;
}

} // namespace composite_binomial

```

单次预处理和空间复杂度为 $O(\sum p_i^{q_i})$ ，之后一次查询还需要 $O(\sum \log_{p_i} n)$ 次递归与快速幂。CRT 中间乘法使用 `__int128_t`，但最终模数仍要求能装入正的 `long long`。

快速沃尔什变换

数组长度必须是 2 的幂，所有值位于模数范围内。OR/AND 逆变换只需加减；XOR 逆变换需要 2 在模意义下可逆。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

enum class BitwiseConvolution {
    bit_or,    // 下标按位或卷积
    bit_and,   // 下标按位与卷积
    bit_xor    // 下标按位异或卷积
};

void fwt(vector<int>& values, BitwiseConvolution type,
        bool inverse, int modulus) {
    int size = (int)values.size();
    assert(size > 0 && (size & (size - 1)) == 0);
    int inverse_two = (modulus + 1) / 2;
    for (int half = 1; half < size; half <= 1) {
        for (int begin = 0; begin < size; begin += half * 2) {
            for (int offset = 0; offset < half; ++offset) {
                int lhs = values[begin + offset];
                int rhs = values[begin + half + offset];
                if (type == BitwiseConvolution::bit_or) {
                    values[begin + half + offset] = inverse
                        ? (rhs - lhs + modulus) % modulus
                        : (rhs + lhs) % modulus;
                } else if (type == BitwiseConvolution::bit_and) {
                    values[begin + offset] = inverse
                        ? (lhs - rhs + modulus) % modulus
                        : (lhs + rhs) % modulus;
                } else {

```

```

        values[begin + offset] = (lhs + rhs) % modulus;
        values[begin + half + offset] =
            (lhs - rhs + modulus) % modulus;
        if (inverse) {
            values[begin + offset] = (int)(
                (long long)values[begin + offset] *
                inverse_two % modulus);
            values[begin + half + offset] = (int)(
                (long long)values[begin + half + offset] *
                inverse_two % modulus);
        }
    }
}

vector<int> bitwise_convolution(vector<int> lhs, vector<int> rhs,
                               BitwiseConvolution type, int modulus) {
    assert(lhs.size() == rhs.size());
    fwt(lhs, type, false, modulus);
    fwt(rhs, type, false, modulus);
    for (int i = 0; i < (int)lhs.size(); ++i) {
        lhs[i] = (int)((long long)lhs[i] * rhs[i] % modulus);
    }
    fwt(lhs, type, true, modulus);
    return lhs;
}

```

复杂度为 $O(n2^n)$ （数组长度为 2^n ）。XOR 版本要求模数为奇数：常用质数模数满足这一点。

子集卷积

计算

$$h(S) = \sum_{T \subseteq S} f(T)g(S \setminus T).$$

使用按集合大小分层的子集 Zeta 变换，复杂度为 $O(n^2 2^n)$ ，空间复杂度为 $O(n 2^n)$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

void subset_zeta(vector<int>& values, int bit_count,
                bool inverse, int modulus) {
    int size = 1 << bit_count;
    for (int bit = 0; bit < bit_count; ++bit) {
        for (int mask = 0; mask < size; ++mask) {
            if (!(mask >> bit & 1)) continue;
            if (inverse) {
                values[mask] -= values[mask ^ (1 << bit)];
            }
        }
    }
}

```

```

        if (values[mask] < 0) values[mask] += modulus;
    } else {
        values[mask] += values[mask ^ (1 << bit)];
        if (values[mask] >= modulus) values[mask] -= modulus;
    }
}
}
}

vector<int> subset_convolution(const vector<int>& lhs,
                             const vector<int>& rhs,
                             int modulus) {
    assert(lhs.size() == rhs.size());
    int size = (int)lhs.size();
    assert(size > 0 && (size & (size - 1)) == 0);
    int bit_count = countr_zero((unsigned)size);

    vector<vector<int>> lhs_layer(bit_count + 1, vector<int>(size));
    vector<vector<int>> rhs_layer(bit_count + 1, vector<int>(size));
    vector<vector<int>> result_layer(bit_count + 1, vector<int>(size));
    for (int mask = 0; mask < size; ++mask) {
        int bits = popcount((unsigned)mask);
        lhs_layer[bits][mask] = lhs[mask];
        rhs_layer[bits][mask] = rhs[mask];
    }
    for (int bits = 0; bits <= bit_count; ++bits) {
        subset_zeta(lhs_layer[bits], bit_count, false, modulus);
        subset_zeta(rhs_layer[bits], bit_count, false, modulus);
    }
    for (int total_bits = 0; total_bits <= bit_count; ++total_bits) {
        for (int lhs_bits = 0; lhs_bits <= total_bits; ++lhs_bits) {
            for (int mask = 0; mask < size; ++mask) {
                result_layer[total_bits][mask] = (int)(
                    (result_layer[total_bits][mask] +
                     (long long)lhs_layer[lhs_bits][mask] *
                     rhs_layer[total_bits - lhs_bits][mask]) % modulus);
            }
        }
        subset_zeta(result_layer[total_bits], bit_count, true, modulus);
    }

    vector<int> result(size);
    for (int mask = 0; mask < size; ++mask) {
        result[mask] = result_layer[popcount((unsigned)mask)][mask];
    }
    return result;
}

```

输入系数必须先正规化到 $[0, modulus)$ 。当 $n = 21$ 时三组二维数组已经很大，应按内存限制选择静态数组、滚动层或其他题目特定优化。

单调斜率优化

维护直线 $y = kx + b$ 的最小值。要求斜率按严格递减顺序加入、查询横坐标单调不减；相同斜率只保留截距更小者。比较使用 `__int128_t`，前提是斜率、截距、横坐标的乘积不超过 128 位范围。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class MonotoneConvexHull {
    using i64 = long long;
    using i128 = __int128_t;

    struct Line {
        i64 slope;
        i64 intercept;

        i128 value(i64 x) const {
            return (i128)slope * x + intercept;
        }
    };

    deque<Line> lines;

    static bool redundant(const Line& lhs, const Line& middle,
                          const Line& rhs) {
        return ((i128)middle.intercept - lhs.intercept) *
            ((i128)middle.slope - rhs.slope) >=
            ((i128)rhs.intercept - middle.intercept) *
            ((i128)lhs.slope - middle.slope);
    }

public:
    void add_line(i64 slope, i64 intercept) {
        if (!lines.empty() && lines.back().slope == slope) {
            if (lines.back().intercept <= intercept) return;
            lines.pop_back();
        }
        assert(lines.empty() || lines.back().slope > slope);
        Line added{slope, intercept};
        while (lines.size() >= 2 &&
            redundant(lines[lines.size() - 2], lines.back(), added)) {
            lines.pop_back();
        }
        lines.push_back(added);
    }

    i128 query(i64 x) {
        assert(!lines.empty());
        while (lines.size() >= 2 &&
            lines[1].value(x) <= lines[0].value(x)) {
            lines.pop_front();
        }
        return lines.front().value(x);
    }
};
```

```
    }  
};
```

每条直线最多入队、出队一次，因此一批操作总复杂度为线性。若斜率或查询横坐标不单调，使用李超线段树；若只离线查询，可按横坐标排序后恢复原顺序。

少项式技巧

少项式指只有少量非零项的多项式。应把复杂度写成非零项数 m 的函数，而不是最高次数的函数。

模意义下的函数迭代

设

$$f(x) = \sum_{(d, a_d)} a_d x^d, \quad x_{t+1} = f(x_t) \bmod p.$$

因为 $f(x) \bmod p$ 只依赖 $x \bmod p$ ，余数 $0, 1, \dots, p-1$ 构成一张函数图。先计算每个余数的一步转移，再对转移函数倍增，即可在 $O(\log y)$ 内求 $f^{(y)}(x)$ 。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using i128 = __int128_t;
using u64 = unsigned long long;

class SparsePolynomialIteration {
    int modulus;
    vector<vector<int>>> jump;

    int power_mod(i64 base, u64 exponent) const {
        i64 result = 1 % modulus;
        base %= modulus;
        if (base < 0) base += modulus;
        while (exponent > 0) {
            if (exponent & 1) result = (i64)((i128)result * base % modulus);
            base = (i64)((i128)base * base % modulus);
            exponent >>= 1;
        }
        return (int)result;
    }
}

public:
    SparsePolynomialIteration(
        int iteration_modulus, const vector<pair<u64, i64>>>& terms,
        u64 maximum_steps) : modulus(iteration_modulus) {
        assert(iteration_modulus >= 1);
        int level_count = max(1, (int)bit_width(maximum_steps));
        jump.assign(level_count, vector<int>(modulus));
        for (int value = 0; value < iteration_modulus; ++value) {
```

```

        i64 next = 0;
        for (auto [degree, coefficient] : terms) {
            coefficient %= modulus;
            if (coefficient < 0) coefficient += modulus;
            next = (i64)((next + (i128)coefficient *
                           power_mod(value, degree)) % modulus);
        }
        jump[0][value] = (int)next;
    }
    for (int level = 1; level < level_count; ++level) {
        for (int value = 0; value < modulus; ++value) {
            jump[level][value] =
                jump[level - 1][jump[level - 1][value]];
        }
    }
}

int iterate(i64 start, u64 steps) const {
    int value = (int)(start % modulus);
    if (value < 0) value += modulus;
    for (int level = 0; steps > 0; ++level, steps >>= 1) {
        assert(level < (int)jump.size());
        if (steps & 1) value = jump[level][value];
    }
    return value;
}
};

```

预处理一步转移需要 $O(pm \log D)$, 其中 D 是最大指数; 倍增需要 $O(p \log y)$ 时间和空间。构造时的 `maximum_steps` 必须不小于之后任何询问的步数。若 p 很大而查询很少, 直接在函数图上做环检测通常更省空间。

少项式的形式幂级数幂

令 $G(x) = A(x)^k$ 。由

$$G'(x)A(x) = kG(x)A'(x)$$

比较 x^n 的系数可得

$$(n+1)a_0g_{n+1} = \sum_{d=1}^{n+1} ((k+1)d - (n+1))a_dg_{n+1-d}.$$

因此若 A 只有 m 个非零项, 可以在 $O(Nm)$ 内求出 $A(x)^k \bmod x^{N+1}$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

vector<int> sparse_power_series(vector<pair<int, int>> terms,

```

```

        i64 exponent, int degree,
        int prime) {
assert(exponent >= 0 && 0 <= degree && degree < prime);
auto normalize = [&](i64 value) {
    value %= prime;
    if (value < 0) value += prime;
    return (int)value;
};
auto power_mod = [&](i64 base, i64 power) {
    i64 result = 1;
    while (power > 0) {
        if (power & 1) result = result * base % prime;
        base = base * base % prime;
        power >>= 1;
    }
    return (int)result;
};

map<int, int> merged;
for (auto [term_degree, coefficient] : terms) {
    assert(term_degree >= 0);
    int& current = merged[term_degree];
    current = normalize((i64)current + coefficient);
}
vector<pair<int, int>> nonconstant_terms;
int constant = merged.contains(0) ? merged[0] : 0;
assert(constant != 0);
for (auto [term_degree, coefficient] : merged) {
    if (term_degree > 0 && term_degree <= degree && coefficient != 0) {
        nonconstant_terms.push_back({term_degree, coefficient});
    }
}

vector<int> result(degree + 1);
result[0] = power_mod(constant, exponent);
i64 exponent_mod = exponent % prime;
for (int current_degree = 1;
    current_degree <= degree; ++current_degree) {
    i64 numerator = 0;
    for (auto [term_degree, coefficient] : nonconstant_terms) {
        if (term_degree > current_degree) break;
        i64 factor = normalize(
            (i64)((i128)(exponent_mod + 1) * term_degree -
                current_degree));
        numerator = (i64)((numerator + (i128)factor * coefficient % prime *
            result[current_degree - term_degree]) % prime);
    }
    int denominator = (int)((i64)current_degree * constant % prime);
    result[current_degree] = (int)(
        numerator * power_mod(denominator, prime - 2) % prime);
}
return result;
}

```


这里要求 `prime` 为质数、 $a_0 \neq 0$ 、 $0 \leq N < \text{prime}$ 且 $k \geq 0$ ，以保证递推中的分母可逆。输入中的同次项会先合并。若 $a_0 = 0$ ，应先提取最低次幂；通用的大规模多项式幂见 `16-polynomial.md`。

生成函数与 Bostan-Mori

线性递推的普通生成函数

若序列满足

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_d a_{n-d},$$

则普通生成函数 $A(x) = \sum_{n \geq 0} a_n x^n$ 可以写成

$$A(x) = \frac{P(x)}{Q(x)}, \quad Q(x) = 1 - c_1 x - c_2 x^2 - \cdots - c_d x^d.$$

当 $\deg P < d$ 时，

$$P_i = a_i - \sum_{j=1}^i c_j a_{i-j} \quad (0 \leq i < d).$$

因此求递推第 n 项等价于求 $[x^n]P(x)/Q(x)$ 。Bostan-Mori 每轮把下标减半：

$$\frac{P(x)}{Q(x)} = \frac{P(x)Q(-x)}{Q(x)Q(-x)}.$$

分母只含偶数次项；根据 n 的奇偶，只保留分子的偶数项或奇数项。

下面代码依赖 `16-polynomial.md` 中的 `polynomial::multiply`、`mod_inverse` 和 `mod`。

```
namespace polynomial {

vector<int> parity_coefficients(const vector<int>& poly, int parity) {
    vector<int> result;
    for (int i = parity; i < (int)poly.size(); i += 2) {
        result.push_back(poly[i]);
    }
    return result;
}

int bostan_mori(vector<int> numerator, vector<int> denominator,
                long long index) {
    assert(index >= 0);
    assert(!denominator.empty() && denominator[0] != 0);
    while (index > 0) {
        vector<int> denominator_negative = denominator;
        for (int i = 1; i < (int)denominator_negative.size(); i += 2) {
            if (denominator_negative[i] != 0) {
                denominator_negative[i] =
                    mod - denominator_negative[i];
            }
        }
    }
}
```

```

    }
    numerator = parity_coefficients(
        multiply(numerator, denominator_negative), index & 1);
    denominator = parity_coefficients(
        multiply(denominator, denominator_negative), 0);
    index >>= 1;
}
int constant = numerator.empty() ? 0 : numerator[0];
return (int)((long long)constant * mod_inverse(denominator[0]) % mod);
}

} // namespace polynomial

```

复杂度为 $O(M(d) \log n)$ ，其中 $M(d)$ 是 d 次多项式乘法复杂度。输入应满足 $\deg P < \deg Q$ ；从递推构造时自然满足这一条件。

常见 OGF/EGF 及建模规则统一见 21-generating-functions.md，公式速查见 24-counting-formulas.md。

常见陷阱

1. $Q(0)$ 必须可逆；通常先把分子分母同时除以 $Q(0)$ ，令分母常数项为 1。
2. 模数下的负系数要正规化到 $[0, p)$ 。
3. 从递推构造 P 时只保留前 d 项，不是直接把初值数组当作分子。
4. Bostan-Mori 的奇偶取自当前 `index`，取完系数后才右移。

Lucas 定理

模数必须是质数。预处理 $0, 1, \dots, p-1$ 的阶乘和逆阶乘后，可以在 $O(\log_p n)$ 内计算 $\binom{n}{k} \bmod p$ 。当 p 太大、无法分配 $O(p)$ 空间时不应使用此版本。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

class LucasBinomial {
    using i64 = long long;

    int prime;
    vector<int> factorial;
    vector<int> inverse_factorial;

    int power_mod(i64 base, i64 exponent) const {
        i64 result = 1;
        while (exponent > 0) {
            if (exponent & 1) result = result * base % prime;
            base = base * base % prime;
            exponent >>= 1;
        }
        return (int)result;
    }

    int small_choose(int n, int k) const {

```

```

        if (k < 0 || k > n) return 0;
        return (int)((i64)factorial[n] * inverse_factorial[k] % prime *
                    inverse_factorial[n - k] % prime);
    }

public:
    explicit LucasBinomial(int prime_modulus)
        : prime(prime_modulus), factorial(prime_modulus),
          inverse_factorial(prime_modulus) {
        assert(prime_modulus >= 2);
        factorial[0] = 1;
        for (int value = 1; value < prime_modulus; ++value) {
            factorial[value] =
                (int)((i64)factorial[value - 1] * value % prime_modulus);
        }
        inverse_factorial[prime_modulus - 1] =
            power_mod(factorial[prime_modulus - 1], prime_modulus - 2);
        for (int value = prime_modulus - 1; value > 0; --value) {
            inverse_factorial[value - 1] =
                (int)((i64)inverse_factorial[value] * value % prime_modulus);
        }
    }

    int choose(i64 n, i64 k) const {
        if (k < 0 || k > n) return 0;
        i64 result = 1;
        while (n > 0 || k > 0) {
            result = result *
                small_choose((int)(n % prime), (int)(k % prime)) %
                prime;
            if (result == 0) return 0;
            n /= prime;
            k /= prime;
        }
        return (int)result;
    }
};

```

构造类之前必须确认 `prime` 确为质数：否则 Fermat 逆元和 Lucas 分解都不成立。

NTT 与形式幂级数基础版

本模板固定模数为 $998244353 = 119 \times 2^{23} + 1$ ，原根为 3，因此单次 NTT 长度不能超过 2^{23} 。所有多项式按低次到高次存储，即 `a[i]` 是 x^i 的系数。

本文件是赛场轻量版，只包含卷积、求导、积分、逆、对数、指数和非负整数幂。需要 FPS 开方、除法、多点求值/插值、Bostan-Mori、Taylor shift、多项式复合、幂投影或复合逆时，使用 `23-polynomial-advanced.md` 的自包含完整模板。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

```

```

namespace polynomial {

using i64 = long long;
constexpr int mod = 998244353;
constexpr int primitive_root = 3;

int mod_power(i64 base, i64 exponent) {
    i64 result = 1;
    while (exponent > 0) {
        if (exponent & 1) result = result * base % mod;
        base = base * base % mod;
        exponent >>= 1;
    }
    return (int)result;
}

int mod_inverse(int value) {
    assert(value != 0);
    return mod_power(value, mod - 2);
}

void ntt(vector<int>& values, bool inverse) {
    int size = (int)values.size();
    assert(size > 0 && (size & (size - 1)) == 0);
    assert(size <= (1 << 23));

    for (int i = 1, reversed = 0; i < size; ++i) {
        int bit = size >> 1;
        while (reversed & bit) {
            reversed ^= bit;
            bit >>= 1;
        }
        reversed ^= bit;
        if (i < reversed) swap(values[i], values[reversed]);
    }

    for (int length = 2; length <= size; length <= 1) {
        int root = mod_power(primitive_root, (mod - 1) / length);
        if (inverse) root = mod_inverse(root);
        for (int begin = 0; begin < size; begin += length) {
            i64 current_root = 1;
            for (int offset = 0; offset < length / 2; ++offset) {
                int even = values[begin + offset];
                int odd = (int)(values[begin + offset + length / 2] *
                                current_root % mod);
                values[begin + offset] = even + odd;
                if (values[begin + offset] >= mod) values[begin + offset] -= mod;
                values[begin + offset + length / 2] = even - odd;
                if (values[begin + offset + length / 2] < 0) {
                    values[begin + offset + length / 2] += mod;
                }
            }
            current_root = current_root * root % mod;
        }
    }
}

```

```

    }
}

if (inverse) {
    int size_inverse = mod_inverse(size);
    for (int& value : values) {
        value = (int)((i64)value * size_inverse % mod);
    }
}

vector<int> multiply(vector<int> lhs, vector<int> rhs) {
    if (lhs.empty() || rhs.empty()) return {};
    int result_size = (int)lhs.size() + (int)rhs.size() - 1;
    if (min(lhs.size(), rhs.size()) <= 32) {
        vector<int> result(result_size);
        for (int i = 0; i < (int)lhs.size(); ++i) {
            for (int j = 0; j < (int)rhs.size(); ++j) {
                result[i + j] = (int)(
                    (result[i + j] + (i64)lhs[i] * rhs[j]) % mod);
            }
        }
        return result;
    }

    int ntt_size = 1;
    while (ntt_size < result_size) ntt_size <= 1;
    lhs.resize(ntt_size);
    rhs.resize(ntt_size);
    ntt(lhs, false);
    ntt(rhs, false);
    for (int i = 0; i < ntt_size; ++i) {
        lhs[i] = (int)((i64)lhs[i] * rhs[i] % mod);
    }
    ntt(lhs, true);
    lhs.resize(result_size);
    return lhs;
}

vector<int> derivative(const vector<int>& poly) {
    if (poly.size() <= 1) return {};
    vector<int> result(poly.size() - 1);
    for (int i = 1; i < (int)poly.size(); ++i) {
        result[i - 1] = (int)((i64)poly[i] * i % mod);
    }
    return result;
}

vector<int> integral(const vector<int>& poly) {
    vector<int> result(poly.size() + 1);
    for (int i = 0; i < (int)poly.size(); ++i) {
        result[i + 1] = (int)((i64)poly[i] * mod_inverse(i + 1) % mod);
    }
}

```

```

    return result;
}

vector<int> inverse(const vector<int>& poly, int result_size) {
    assert(result_size >= 1);
    assert(!poly.empty() && poly[0] != 0);
    vector<int> result{mod_inverse(poly[0])};
    for (int length = 2; length < 2 * result_size; length <= 1) {
        int current_size = min(length, result_size);
        vector<int> prefix(min((int)poly.size(), current_size));
        copy(poly.begin(), poly.begin() + prefix.size(), prefix.begin());
        vector<int> product = multiply(prefix, result);
        product.resize(current_size);
        for (int& value : product) {
            if (value != 0) value = mod - value;
        }
        product[0] = (product[0] + 2) % mod;
        result = multiply(result, product);
        result.resize(current_size);
    }
    result.resize(result_size);
    return result;
}

vector<int> logarithm(const vector<int>& poly, int result_size) {
    assert(result_size >= 1);
    assert(!poly.empty() && poly[0] == 1);
    vector<int> result = multiply(derivative(poly), inverse(poly, result_size));
    if ((int)result.size() >= result_size) result.resize(result_size - 1);
    result = integral(result);
    result.resize(result_size);
    return result;
}

vector<int> exponential(const vector<int>& poly, int result_size) {
    assert(result_size >= 1);
    assert(poly.empty() || poly[0] == 0);
    vector<int> result{1};
    for (int length = 2; length < 2 * result_size; length <= 1) {
        int current_size = min(length, result_size);
        vector<int> difference(current_size);
        copy(poly.begin(), poly.begin() + min((int)poly.size(), current_size),
            difference.begin());
        vector<int> result_log = logarithm(result, current_size);
        for (int i = 0; i < current_size; ++i) {
            difference[i] -= result_log[i];
            if (difference[i] < 0) difference[i] += mod;
        }
        difference[0] = (difference[0] + 1) % mod;
        result = multiply(result, difference);
        result.resize(current_size);
    }
    result.resize(result_size);
}

```

```

    return result;
}

vector<int> power(const vector<int>& poly, i64 exponent, int result_size) {
    assert(exponent >= 0);
    vector<int> result(result_size);
    if (result_size == 0) return result;
    if (exponent == 0) {
        result[0] = 1;
        return result;
    }

    int first_nonzero = 0;
    while (first_nonzero < (int)poly.size() && poly[first_nonzero] == 0) {
        ++first_nonzero;
    }
    if (first_nonzero == (int)poly.size() ||
        first_nonzero > (result_size - 1) / exponent) {
        return result;
    }

    int shift = (int)(first_nonzero * exponent);
    int remaining_size = result_size - shift;
    int leading = poly[first_nonzero];
    int leading_inverse = mod_inverse(leading);
    vector<int> normalized(min((int)poly.size() - first_nonzero, remaining_size));
    for (int i = 0; i < (int)normalized.size(); ++i) {
        normalized[i] =
            (int)((i64)poly[first_nonzero + i] * leading_inverse % mod);
    }

    vector<int> normalized_log = logarithm(normalized, remaining_size);
    int reduced_exponent = (int)(exponent % mod);
    for (int& value : normalized_log) {
        value = (int)((i64)value * reduced_exponent % mod);
    }
    vector<int> powered = exponential(normalized_log, remaining_size);
    int leading_power = mod_power(leading, exponent);
    for (int i = 0; i < remaining_size; ++i) {
        result[i + shift] = (int)((i64)powered[i] * leading_power % mod);
    }
    return result;
}

} // namespace polynomial

```

前置条件与复杂度

- `inverse(a, n)` 要求 $a_0 \neq 0$, 返回 $a^{-1} \bmod x^n$ 。
- `logarithm(a, n)` 要求 $a_0 = 1$ 。
- `exponential(a, n)` 要求 $a_0 = 0$ 。
- `power(a, k, n)` 支持 $k \geq 0$, 包括前导零多项式。
- 乘法复杂度为 $O(n \log n)$; 逆、对数、指数和幂均为 $O(n \log n)$, 常数不同。

比赛中不要把普通整数负数直接存入系数数组；先执行 $(\text{value} \% \text{mod} + \text{mod}) \% \text{mod}$ 。积分要求次数小于模数，否则分母可能在模意义下不可逆。

组合数学基础

本节默认模数是质数，且预处理上界严格小于模数。若上界达到或超过模数，阶乘会变成 0，不能再用阶乘逆元；应改用 Lucas、扩展 Lucas 或题目特定方法。

本文件保存可直接复制的基础代码；更完整的组合恒等式、插板与上下界、容斥、Catalan/反射、各类反演、整数分拆以及 Burnside-Pólya 见 `24-counting-formulas.md`。

阶乘、组合数与排列数

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;

class PrimeCombinations {
    int prime;
    vector<int> factorial;
    vector<int> inverse_factorial;

    int power_mod(i64 base, i64 exponent) const {
        i64 result = 1;
        while (exponent > 0) {
            if (exponent & 1) result = result * base % prime;
            base = base * base % prime;
            exponent >>= 1;
        }
        return (int)result;
    }

public:
    PrimeCombinations(int maximum, int prime_modulus)
        : prime(prime_modulus), factorial(maximum + 1),
          inverse_factorial(maximum + 1) {
        assert(0 <= maximum && maximum < prime_modulus);
        factorial[0] = 1;
        for (int value = 1; value <= maximum; ++value) {
            factorial[value] =
                (int)((i64)factorial[value - 1] * value % prime_modulus);
        }
        inverse_factorial[maximum] =
            power_mod(factorial[maximum], prime_modulus - 2);
        for (int value = maximum; value >= 1; --value) {
            inverse_factorial[value - 1] =
                (int)((i64)inverse_factorial[value] * value % prime_modulus);
        }
    }
}
```



```

int choose(int n, int k) const {
    if (k < 0 || k > n) return 0;
    assert(n < (int)factorial.size());
    return (int)((i64)factorial[n] * inverse_factorial[k] % prime *
                inverse_factorial[n - k] % prime);
}

int permutations(int n, int k) const {
    if (k < 0 || k > n) return 0;
    assert(n < (int)factorial.size());
    return (int)((i64)factorial[n] * inverse_factorial[n - k] % prime);
}

int multiset_choose(int types, int count) const {
    if (count < 0 || types < 0) return 0;
    if (types == 0) return count == 0;
    return choose(types + count - 1, count);
}

int catalan(int n) const {
    assert(2 * n < (int)factorial.size());
    int answer = choose(2 * n, n) - choose(2 * n, n - 1);
    if (answer < 0) answer += prime;
    return answer;
}

int fact(int n) const {
    return factorial[n];
}

int inv_fact(int n) const {
    return inverse_factorial[n];
}
};

vector<int> derangements(int maximum, int modulus) {
    vector<int> result(maximum + 1);
    result[0] = 1 % modulus;
    if (maximum >= 1) result[1] = 0;
    for (int n = 2; n <= maximum; ++n) {
        result[n] = (int)((long long)(n - 1) *
                        (result[n - 1] + result[n - 2]) % modulus);
    }
    return result;
}

```

- $\text{choose}(n, k)$ 计算 $\binom{n}{k}$; $\text{permutations}(n, k)$ 计算下降阶乘 $n^{\underline{k}}$ 。
- $\text{multiset_choose}(m, n) = \binom{m+n-1}{n}$ 是从 m 类物品中可重复选 n 个，也等价于把 n 个相同物品放进 m 个有标号盒子。
- $\text{catalan}(n) = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}$ 。差值写法避免额外求逆，但仍要求预处理到 $2n$ 。
- 错排数满足 $D_0 = 1, D_1 = 0, D_n = (n-1)(D_{n-1} + D_{n-2})$ 。

Hockey-stick、Vandermonde、二项式反演等恒等式统一收录在 24-counting-formulas.md；本文件只保留

可直接复制的组合数代码及其接口说明。

Stirling 数

本节固定采用以下符号：

- $\begin{bmatrix} n \\ k \end{bmatrix}$ 是无符号第一类 Stirling 数，表示 n 个元素的排列恰有 k 个环的方案数；
- $\left\{ \begin{matrix} n \\ k \end{matrix} \right\}$ 是第二类 Stirling 数，表示把 n 个不同元素划分成 k 个非空无标号集合的方案数。

边界均为 $S(0,0) = 1$ ，且 $n > 0$ 时 $S(n,0) = 0$ 。

Stirling 反演、幂和换基、Bell 数关系及配套计数公式见 `24-counting-formulas.md`。

递推、换基与生成函数

$$\begin{bmatrix} n \\ k \end{bmatrix} = \begin{bmatrix} n-1 \\ k-1 \end{bmatrix} + (n-1) \begin{bmatrix} n-1 \\ k \end{bmatrix},$$

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \left\{ \begin{matrix} n-1 \\ k-1 \end{matrix} \right\} + k \left\{ \begin{matrix} n-1 \\ k \end{matrix} \right\}.$$

无符号第一类对应上升幂，带符号第一类 $s(n,k) = (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix}$ 才对应下降幂：

$$x^{\overline{n}} = \sum_{k=0}^n \begin{bmatrix} n \\ k \end{bmatrix} x^k, \quad x^n = \sum_{k=0}^n s(n,k) x^k,$$

$$x^n = \sum_{k=0}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^{\underline{k}}.$$

固定 k 的指数生成函数为

$$\sum_{n \geq k} \begin{bmatrix} n \\ k \end{bmatrix} \frac{x^n}{n!} = \frac{(-\ln(1-x))^k}{k!},$$

$$\sum_{n \geq k} \left\{ \begin{matrix} n \\ k \end{matrix} \right\} \frac{x^n}{n!} = \frac{(e^x - 1)^k}{k!}.$$

模 998244353 快速计算

下面代码依赖 `16-polynomial.md` 的 `polynomial::multiply`、`power`、`mod_power`、`mod_inverse` 和 `mod`。

```
namespace combinatorics {  
  
using polynomial::mod;  
using polynomial::mod_inverse;  
using polynomial::mod_power;  
using polynomial::multiply;  
using polynomial::power;  
  
struct Factorials {  
    vector<int> factorial;
```

```

vector<int> inverse_factorial;

explicit Factorials(int maximum)
: factorial(maximum + 1), inverse_factorial(maximum + 1) {
    assert(0 <= maximum && maximum < mod);
    factorial[0] = 1;
    for (int value = 1; value <= maximum; ++value) {
        factorial[value] =
            (int)((long long)factorial[value - 1] * value % mod);
    }
    inverse_factorial[maximum] = mod_inverse(factorial[maximum]);
    for (int value = maximum; value >= 1; --value) {
        inverse_factorial[value - 1] =
            (int)((long long)inverse_factorial[value] * value % mod);
    }
}

};

vector<int> stirling_second_row(int n) {
    assert(n >= 0);
    Factorials values(n);
    vector<int> powers(n + 1);
    vector<int> alternating_inverse_factorial(n + 1);
    for (int index = 0; index <= n; ++index) {
        powers[index] = (int)((long long)mod_power(index, n) *
                                values.inverse_factorial[index] % mod);
        alternating_inverse_factorial[index] =
            index & 1 ? mod - values.inverse_factorial[index]
                : values.inverse_factorial[index];
    }
    vector<int> result = multiply(powers, alternating_inverse_factorial);
    result.resize(n + 1);
    return result;
}

vector<int> rising_factorial(int left, int right) {
    if (left == right) return {1};
    if (left + 1 == right) return {left % mod, 1};
    int middle = left + (right - left) / 2;
    return multiply(rising_factorial(left, middle),
                    rising_factorial(middle, right));
}

vector<int> stirling_first_row(int n) {
    assert(n >= 0);
    return rising_factorial(0, n);
}

vector<int> stirling_second_column(int maximum_n, int k) {
    assert(0 <= k && k <= maximum_n && maximum_n < mod);
    Factorials values(maximum_n);
    vector<int> exponential_minus_one(maximum_n + 1);
    for (int degree = 1; degree <= maximum_n; ++degree) {

```

```

        exponential_minus_one[degree] = values.inverse_factorial[degree];
    }
    vector<int> generating_function =
        power(exponential_minus_one, k, maximum_n + 1);
    vector<int> result(maximum_n + 1);
    for (int n = 0; n <= maximum_n; ++n) {
        result[n] = (int)((long long)generating_function[n] *
            values.factorial[n] % mod *
            values.inverse_factorial[k] % mod);
    }
    return result;
}

vector<int> stirling_first_column(int maximum_n, int k) {
    assert(0 <= k && k <= maximum_n && maximum_n < mod);
    Factorials values(maximum_n);
    vector<int> negative_logarithm(maximum_n + 1);
    for (int degree = 1; degree <= maximum_n; ++degree) {
        negative_logarithm[degree] = mod_inverse(degree);
    }
    vector<int> generating_function =
        power(negative_logarithm, k, maximum_n + 1);
    vector<int> result(maximum_n + 1);
    for (int n = 0; n <= maximum_n; ++n) {
        result[n] = (int)((long long)generating_function[n] *
            values.factorial[n] % mod *
            values.inverse_factorial[k] % mod);
    }
    return result;
}

} // namespace combinatorics

```

复杂度：

- 第二类同一行使用显式公式 $\{n_k\} = \sum_{i=0}^k (-1)^{k-i} i^n / (i!(k-i)!)$ ，只需一次卷积，为 $O(M(n))$ ；
- 第一类同一行计算 $x(x+1)\cdots(x+n-1)$ ，分治乘法为 $O(M(n) \log n)$ ；
- 固定一列使用 EGF 和 FPS 幂，为 $O(M(n))$ 量级。

当前 NTT 最长为 2^{23} ，实际可计算规模还受卷积补零长度与内存限制。只求单个 $S(n, k)$ 且 k 很小时， $O(nk)$ 递推通常更简单。

Bell 数满足 $B_n = \sum_k \{n_k\}$ ，EGF 为 $\exp(e^x - 1)$ 。因此求一整段 Bell 数时，也可以先构造 `exponential_minus_one`，调用 `polynomial::exponential`，最后把第 n 项乘以 $n!$ 。

线性筛与因数卷积

线性筛

一次预处理得到每个数的最小质因子、Möbius 函数和 Euler 函数。每个合数只由“当前数乘它的最小质因子”筛到一次，复杂度 $O(n)$ 。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct LinearSieve {
    vector<int> primes;
    vector<int> least_prime;
    vector<int> mobius;
    vector<int> phi;

    explicit LinearSieve(int maximum)
        : least_prime(maximum + 1), mobius(maximum + 1),
          phi(maximum + 1) {
        assert(maximum >= 1);
        mobius[1] = 1;
        phi[1] = 1;
        for (int value = 2; value <= maximum; ++value) {
            if (least_prime[value] == 0) {
                least_prime[value] = value;
                primes.push_back(value);
                mobius[value] = -1;
                phi[value] = value - 1;
            }
            for (int prime : primes) {
                if (prime > maximum / value) break;
                int product = value * prime;
                least_prime[product] = prime;
                if (value % prime == 0) {
                    mobius[product] = 0;
                    phi[product] = phi[value] * prime;
                    break;
                }
                mobius[product] = -mobius[value];
                phi[product] = phi[value] * (prime - 1);
            }
        }
    }
};

```

恒等式

$$\sum_{d|n} \mu(d) = [n = 1], \quad \sum_{d|n} \varphi(d) = n$$

是检查实现和推导 Möbius 反演时最常用的两条式子。

约数/倍数 Zeta 与 Möbius 变换

数组下标从 1 开始, 0 号位置不使用。以下写法是 $O(n \log n)$ 的调和级数实现, 输入系数应位于 $[0, modulus)$ 。

```

void divisor_transform(vector<int>& values, bool inverse,
                      int modulus) {
    int maximum = (int)values.size() - 1;

```

```

    if (inverse) {
        for (int divisor = 1; divisor <= maximum; ++divisor) {
            for (int multiple = divisor * 2;
                multiple <= maximum; multiple += divisor) {
                values[multiple] -= values[divisor];
                if (values[multiple] < 0) values[multiple] += modulus;
            }
        }
    } else {
        vector<int> source = values;
        fill(values.begin() + 1, values.end(), 0);
        for (int divisor = 1; divisor <= maximum; ++divisor) {
            for (int multiple = divisor;
                multiple <= maximum; multiple += divisor) {
                values[multiple] += source[divisor];
                if (values[multiple] >= modulus) values[multiple] -= modulus;
            }
        }
    }
}

void multiple_transform(vector<int>& values, bool inverse,
                       int modulus) {
    int maximum = (int)values.size() - 1;
    if (inverse) {
        for (int divisor = maximum; divisor >= 1; --divisor) {
            for (int multiple = divisor * 2;
                multiple <= maximum; multiple += divisor) {
                values[divisor] -= values[multiple];
                if (values[divisor] < 0) values[divisor] += modulus;
            }
        }
    } else {
        vector<int> source = values;
        fill(values.begin() + 1, values.end(), 0);
        for (int divisor = 1; divisor <= maximum; ++divisor) {
            for (int multiple = divisor;
                multiple <= maximum; multiple += divisor) {
                values[divisor] += source[multiple];
                if (values[divisor] >= modulus) values[divisor] -= modulus;
            }
        }
    }
}

vector<int> lcm_convolution(vector<int> lhs, vector<int> rhs,
                           int modulus) {
    assert(lhs.size() == rhs.size());
    divisor_transform(lhs, false, modulus);
    divisor_transform(rhs, false, modulus);
    for (int value = 1; value < (int)lhs.size(); ++value) {
        lhs[value] = (int)((long long)lhs[value] * rhs[value] % modulus);
    }
}

```

```

    divisor_transform(lhs, true, modulus);
    return lhs;
}

vector<int> gcd_convolution(vector<int> lhs, vector<int> rhs,
                           int modulus) {
    assert(lhs.size() == rhs.size());
    multiple_transform(lhs, false, modulus);
    multiple_transform(rhs, false, modulus);
    for (int value = 1; value < (int)lhs.size(); ++value) {
        lhs[value] = (int)((long long)lhs[value] * rhs[value] % modulus);
    }
    multiple_transform(lhs, true, modulus);
    return lhs;
}

```

方向不要记反：

- $\text{lcm}(i, j) \mid n$ 当且仅当 $i \mid n$ 且 $j \mid n$ ，所以 LCM 卷积使用约数 Zeta；
- $n \mid \text{gcd}(i, j)$ 当且仅当 $n \mid i$ 且 $n \mid j$ ，所以 GCD 卷积使用倍数 Zeta。

卷积结果只统计下标不超过数组上界的项。若真实 LCM 可能超过上界，那些项会被截掉；GCD 不存在这个越界问题。

线性递推

若

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k},$$

则特征多项式给出 $x^k \equiv c_1 x^{k-1} + \cdots + c_k$ 。对 x^n 快速幂并始终模该多项式，可在 $O(k^2 \log n)$ 内求第 n 项。

已知递推求第 n 项

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using u64 = unsigned long long;

vector<int> combine_recurrence(const vector<int>& lhs,
                              const vector<int>& rhs,
                              const vector<int>& recurrence,
                              int modulus) {
    int order = (int)recurrence.size();
    vector<int> product(2 * order - 1);
    for (int i = 0; i < order; ++i) {
        for (int j = 0; j < order; ++j) {
            product[i + j] = (int)(
                (product[i + j] + (i64)lhs[i] * rhs[j]) % modulus);
        }
    }
}

```

```

    }
}
for (int degree = 2 * order - 2; degree >= order; --degree) {
    for (int step = 1; step <= order; ++step) {
        product[degree - step] = (int)(
            product[degree - step] +
            (i64)product[degree] * recurrence[step - 1]) % modulus);
    }
}
product.resize(order);
return product;
}

int linear_recurrence_nth(const vector<int>& initial,
                        const vector<int>& recurrence,
                        u64 index, int modulus) {
    int order = (int)recurrence.size();
    assert(order > 0 && (int)initial.size() >= order);
    if (index < initial.size()) return initial[(size_t)index];

    vector<int> result(order);
    vector<int> base(order);
    result[0] = 1;
    if (order == 1) base[0] = recurrence[0];
    else base[1] = 1;
    while (index > 0) {
        if (index & 1) {
            result = combine_recurrence(result, base, recurrence, modulus);
        }
        base = combine_recurrence(base, base, recurrence, modulus);
        index >>= 1;
    }

    i64 answer = 0;
    for (int position = 0; position < order; ++position) {
        answer = (answer + (i64)result[position] * initial[position]) % modulus;
    }
    return (int)answer;
}

```

`initial[i]=a_i`, `recurrence[j]=c_{j+1}`。输入系数必须正规化到 $[0, modulus)$ 。若 k 很大且模数支持 NTT，可进一步用多项式取模或 Bostan-Mori 降低复杂度。

Berlekamp-Massey

给出模质数域上的若干前缀项，BM 返回能够生成该前缀的最短线性递推。一般需要至少约两倍真实阶数的可靠项；短前缀上得到的递推可能不能预测后续。

```

int recurrence_power_mod(i64 base, i64 exponent, int prime) {
    i64 result = 1;
    while (exponent > 0) {
        if (exponent & 1) result = result * base % prime;
        base = base * base % prime;
        exponent >>= 1;
    }
}

```



```

    }
    return (int)result;
}

vector<int> berlekamp_massey(const vector<int>& sequence, int prime) {
    vector<int> connection{1};
    vector<int> previous{1};
    int order = 0;
    int shift = 1;
    int previous_discrepancy = 1;

    for (int index = 0; index < (int)sequence.size(); ++index) {
        i64 discrepancy = sequence[index];
        for (int i = 1; i <= order; ++i) {
            discrepancy += (i64)connection[i] * sequence[index - i];
            discrepancy %= prime;
        }
        if (discrepancy == 0) {
            ++shift;
            continue;
        }

        vector<int> old_connection = connection;
        int scale = (int)(discrepancy *
            recurrence_power_mod(previous_discrepancy, prime - 2, prime) %
            prime);
        if (connection.size() < previous.size() + (size_t)shift) {
            connection.resize(previous.size() + shift);
        }
        for (int i = 0; i < (int)previous.size(); ++i) {
            connection[i + shift] = (int)(
                (connection[i + shift] - (i64)scale * previous[i]) % prime);
            if (connection[i + shift] < 0) connection[i + shift] += prime;
        }

        if (2 * order <= index) {
            order = index + 1 - order;
            previous = move(old_connection);
            previous_discrepancy = (int)discrepancy;
            shift = 1;
        } else {
            ++shift;
        }
    }

    connection.resize(order + 1);
    connection.erase(connection.begin());
    for (int& coefficient : connection) {
        if (coefficient != 0) coefficient = prime - coefficient;
    }
    return connection;
}

```

全零序列会得到空递推；调用 `linear_recurrence_nth` 前应单独返回 0。BM 使用逆元，因此要求模数为质

数（更一般地，所有非零差异都必须可逆）。

生成函数基础与建模技巧

生成函数是对序列的形式编码。在算法题中通常只关心系数恒等式；但把无限概率和变成期望时，仍须证明收敛，不能直接把有限域中的无限级数当作实数级数求和。

更完整的常见 OGF/EGF 表、单位根过滤、整数分拆与五边形数定理见 [24-counting-formulas.md](#)；高级 FPS 实现与每种算法的推导见 [23-polynomial-advanced.md](#)。

OGF 与 EGF

序列 (a_n) 的普通生成函数（OGF）和指数生成函数（EGF）分别是

$$A(x) = \sum_{n \geq 0} a_n x^n, \quad \widehat{A}(x) = \sum_{n \geq 0} a_n \frac{x^n}{n!}.$$

OGF 乘法对应普通卷积：

$$[x^n]A(x)B(x) = \sum_{k=0}^n a_k b_{n-k}.$$

EGF 乘法对应“从 n 个标号元素中选择哪些交给左结构”：

$$n![x^n]\widehat{A}(x)\widehat{B}(x) = \sum_{k=0}^n \binom{n}{k} a_k b_{n-k}.$$

经验上，无标号的长度、重量、金额常用 OGF；由不同标号元素组成的集合、排列和划分常用 EGF。

高频级数、SEQ/SET/CYC 构造和操作字典统一收录在 [24-counting-formulas.md](#)。使用前仍须检查对象是否有序、是否带标号以及是否允许空结构。

重复选择时，EGF 自动统计时间顺序

当条件只依赖各类对象出现多少次，而每一种选择顺序都要计数时，可以先按类别独立建模，再用 EGF 乘法自动合并所有交错顺序。这里没有把不同顺序视为同一种方案；只是不用显式枚举顺序。

总共选择 n 次，第 i 类出现 k_i 次，满足 $\sum_i k_i = n$ 。固定这组次数后的顺序数是

$$\frac{n!}{\prod_i k_i!}.$$

各类 EGF 相乘提供分母 $\prod_i k_i!$ ，最后提取 $n![z^n]$ 恰好补上这些顺序。例如 A 选两次、B 选一次， $3![z^3](z^2/2!)z = 3$ ，对应 AAB、ABA、BAA。

设每一步独立抽样，第 i 类概率恒为 w_i ，所有类别互斥且 $\sum_i w_i = 1$ 。允许第 i 类出现的次数集合为 K_i ，则

$$A(z) = \prod_i \left(\sum_{k \in K_i} \frac{(w_i z)^k}{k!} \right), \quad \Pr(n \text{ 步后满足全部次数限制}) = n![z^n]A(z).$$

某一类的次数限制	对应因子
任意次	e^{wz}
至少一次	$e^{wz} - 1$
恰好 r 次	$(wz)^r / r!$
至多 r 次	$\sum_{j=0}^r (wz)^j / j!$
至少 r 次	$e^{wz} - \sum_{j=0}^{r-1} (wz)^j / j!$

若每类有 c_i 个等价选项，也可用整数权重 c_i 建模： $n![z^n]A(z)$ 得到选择序列数，再除以总选项数的 n 次幂得到概率。不受限制的类别也要保留其指数因子，或先合并成一个“其他”类别。

这套乘法来自时间位置的分配，不要求各类别在固定步数下的出现次数独立；事实上它们的总和固定，通常不独立。若条件依赖相邻关系、先后关系或前缀状态，仅记录次数就不够，需要额外状态或自动机。

用单项式记录信息，最后施加线性映射

生成函数的变量不一定表示时间或概率；它可以只负责记录一个可加的统计量。如果最终表达式只依赖某个集合的总权重，就先把总权重放到指数上，让多项式乘法完成合并，最后再解释每一个单项式。

设权重 a_i 为非负整数，目标是计算

$$\sum_{A \subseteq [m]} (-1)^{|A|} g \left(\sum_{i \in A} a_i \right).$$

先构造

$$P(x) = \prod_i (1 - x^{a_i}) = \sum_k c_k x^k, \quad c_k = \sum_{\substack{A \subseteq [m] \\ \sum_{i \in A} a_i = k}} (-1)^{|A|}.$$

然后定义线性映射（线性泛函）

$$\mathcal{L}_g(x^k) = g(k), \quad \mathcal{L}_g \left(\sum_k c_k x^k \right) = \sum_k c_k g(k).$$

于是目标就是 $\mathcal{L}_g(P)$ 。 g 可以是概率、倒数或其他函数，不必具有多项式形式；因为我们只是在对已经按权重合并的系数做加权求和。可用 $\prod_i (1 + yx^{a_i})$ 同时记录集合大小，再取 $[y^j]$ 得到大小恰好为 j 的版本。

为什么这不是普通的“给 x 代入一个数”

重复抽样中，令课程列表总大小为 S ，每步独立均匀抽取一个位置。固定 t 步，定义

$$\mathcal{L}_t(x^k) = \left(1 - \frac{k}{S}\right)^t.$$

这里 t 是外部时间参数， x^k 中的 k 记录被禁用的列表位置总数。 $0 \leq k \leq S$ 时，右侧就是这 k 个位置在 t 步里全部未被选中的概率； $t=0$ 时所有单项式均映射为 1，包括此处按空乘积解释的 0^0 。

例如两类课程的权重为 $a, b > 0$ 且 $a + b \leq S$ ，则

$$\mathcal{L}_t((1 - x^a)(1 - x^b)) = 1 - (1 - a/S)^t - (1 - b/S)^t + (1 - (a + b)/S)^t.$$

这是两类都至少出现一次的容斥式。多项式已把“选哪些缺失类别”的信息压缩成“总共禁用了多少位置”。但一般有

$$\mathcal{L}_t(x^{a+b}) = (1 - (a+b)/S)^t \neq (1 - a/S)^t (1 - b/S)^t = \mathcal{L}_t(x^a) \mathcal{L}_t(x^b).$$

所以 $\mathcal{L}_t$ 只保留线性组合，不能穿过乘法：先把因子相乘，合并总权重，再映射；不能先把每个因子变成概率再相乘。这也解释了为什么固定步数下事件即使相关，仍可能在多项式层面相乘编码。

乘法能否编码联合事件仍须单独证明。上述课程模型要求类别对应的列表位置互不重叠；把多个任务的因子直接相乘时，还要求任务所需类别不重叠。若共享课程，指数相加会重复计算同一批位置，必须改成统计并集，不能直接套用。

先消去常数项，再消去无限时间

对于 $1 \leq k \leq S$ ，几何级数给出

$$\sum_{t \geq 0} \mathcal{L}_t(x^k) = \frac{S}{k}.$$

因此当 $R(x) = \sum_{k=1}^S r_k x^k$ 常数项为零时，

$$\sum_{t \geq 0} \mathcal{L}_t(R) = \sum_{k=1}^S r_k \frac{S}{k} = S \int_0^1 \frac{R(x)}{x} dx.$$

积分只是“把第 k 项除以 k ”的另一种写法。 x^0 对应永远为 1 的序列，其时间和发散；不能把 $k=0$ 随手跳过后仍声称计算的是原式。应先在概率意义上构造趋于零的差值，再对它应用这条公式。完整的完成个数、二项式反演和等待时间例子见 `34-probability-expectation.md` 的“联合成功概率、二项式矩与阈值等待时间”。

这与下一节 EGF 的联系是

$$\mathcal{L}_t(P) = t! [z^t] (e^z P(e^{-z}/S)).$$

对单项式检查即可： $e^z e^{-kz/S} = e^{(1-k/S)z}$ 。因此同一个容斥多项式既能用“按指数施加线性映射”解释，也能用 EGF 系数提取解释；无需强行把 x 本身解释成一个概率。

指数换元：把重复抽样变成有限多项式

若概率都有公共分母 $w_i = c_i/S$ ，其中 c_i 是非负整数，则可先在有理数上令

$$s = e^{z/S}, \quad e^{c_i z/S} = s^{c_i}.$$

由这些指数的有限加法、乘法构成的表达式就成为普通多项式 $Q(s) = \sum_a q_a s^a$ ，并且

$$n! [z^n] Q(e^{z/S}) = \sum_a q_a (a/S)^n.$$

指数次数转成多项式次数后，次数上界由整数权重总和与权重控制，通常不再需要展开无限多项 EGF。这个替换需要公共整数权重结构；若还出现单独的 z 因子，不能自动宣称结果仍是 s 的多项式。

从最终状态到首次达到目标

“第 n 步已经完成”与“恰好在第 n 步首次完成”是不同事件。对于一旦完成就永久完成的目标，前者描述 $\Pr(T \leq n)$ ，后者描述 $\Pr(T = n)$ 。

若每类课程只属于一个任务，任务 i 所需各类权重为 $c_{ij} > 0$ ，令 $d_i = \sum_j c_{ij}$ 。以下多项式分别表示完成、未完成、下一步首次完成该任务：

$$F_i(s) = \prod_j (s^{c_{ij}} - 1), \quad G_i(s) = s^{d_i} - F_i(s),$$

$$H_i(s) = \sum_j \frac{c_{ij}}{S} \prod_{h \neq j} (s^{c_{ih}} - 1).$$

H_i 中缺少的那类在此前出现零次，最后一步以概率 c_{ij}/S 选中它。不同任务的课程集合不交，因此一步至多完成一个任务，最后完成的任务可以唯一标记。

令 $C = \sum_i d_i \leq S$ ，用 u 记录已完成任务数、 t 记录最后完成的任务，得到

$$Q_p(s) = s^{S-C} [u^{p-1} t] \prod_i (G_i(s) + u F_i(s) + t H_i(s)).$$

对于第 p 个任务完成时刻 T_p ，

$$\Pr(T_p = n + 1) = n! [z^n] Q_p(e^{z/S}).$$

只考虑所有所需类别概率为正且 $1 \leq p \leq$ 可完成任务数的情况，此时最终达到目标的概率为 1。又有 $\deg H_i < d_i$ ，所以 $\deg Q_p < S$ ，下面的几何级数均收敛。写 $Q_p = \sum_a q_{p,a} s^a$ ，则

$$\sum_a q_{p,a} \frac{S}{S-a} = 1,$$

$$\mathbb{E}[T_p] = \sum_a q_{p,a} \frac{S^2}{(S-a)^2} = 1 + \sum_a q_{p,a} \frac{aS}{(S-a)^2}.$$

第一式可检查概率是否归一化；期望式来自 $\sum_{n \geq 0} (n+1)r^n = (1-r)^{-2}$ 。若构造的反而是尾概率 $\Pr(T > n)$ 的 EGF，多项式各项应乘 $S/(S-a)$ 后求和，不能混用权重。尾和公式见 **34-probability-expectation.md**。

以上先是实数概率与有理数恒等式。模质数 M 计算时，再将除法改成逆元；必须保证 S 及用到的 $S-a$ 可逆。 $S < M$ 是本模型中一个足够条件。 $q_{p,a}$ 可以为负，单项式系数本身不是概率。

稀疏因子与一次标记

构造一个任务的 F, H 时，初始 $F = 1, H = 0$ 。加入权重 c ，使用更新前的两个多项式计算

$$H_{\text{new}} = H_{\text{old}}(s^c - 1) + \frac{c}{S} F_{\text{old}}, \quad F_{\text{new}} = F_{\text{old}}(s^c - 1).$$

乘 $s^c - 1$ 只需移位相减。任务有 m 类、总权重 d ，朴素实现上界 $O(md)$ ，无需每个因子都调用 NTT。

更一般地，求 $[t] \prod_i (A_i + tB_i)$ 时，只保留 t^0, t^1 。用一对多项式 (P, Q) 表示 $P + tQ \pmod{t^2}$ ，合并为

$$(P_L, Q_L)(P_R, Q_R) = (P_L P_R, Q_L P_R + P_L Q_R).$$

这就是乘法法则，也是“恰好选一个特殊位置”的通用维护方式；不必计算全部 t 次数，也不需要 B_i/A_i 的除法。要标记恰好 r 个位置，则改为模 t^{r+1} 截断。

二元多项式压成一元卷积

两个多项式相乘后，内层变量 s 的次数上界为 D ，取 $B = D + 1$ ，编码

$$u^i s^j \mapsto X^{iB+j}.$$

乘法中的 s 次数之和小于 B ，所以不会进位到 u 的位置，一次普通卷积就是一次二元卷积。这称为 Kronecker 代换。两个输入必须按本次结果的同一个 B 重新打包，不能直接拿各自原来的二维数组展平后相乘。

已有实现见 `23-polynomial-advanced.md` 中的 `BiPoly` 与 `multiply(const BiPoly&, const BiPoly&)`；外层变量对应 `dx`，内层变量对应 `dy`。

若外层次数为 L ，打包规模为 $O((L+1)(D+1))$ ，还要向上补齐 NTT 长度，并检查不超过 2^{23} 。对于按任务数平衡的分治，节点含 l 个任务、总权重 D ，规模为 $O(lD)$ ；每层权重和至多 S ，块长按倍数增长，所有层总规模为 $O(kS)$ ，总时间上界 $O(kS \log(kS))$ 。该估计依赖分治平衡及各任务权重可加；依次乘入所有任务不能直接套这个复杂度。

标记变量与阶乘矩

若 $A(z, u)$ 中 u 的次数记录某个统计量 K ，则

$$[u^r]A \text{ 选出 } K = r, \quad \left. \frac{\partial A}{\partial u} \right|_{u=1} \text{ 给每个对象乘 } K,$$

$$\left. \frac{\partial^r A}{\partial u^r} \right|_{u=1} \text{ 给每个对象乘 } K(K-1)\cdots(K-r+1).$$

因此如果只问统计量的均值或二阶矩，往往只需维护少数导数，避免保留完整的 u 分布。例如固定大小为 n 的等概率对象，其平均 K 为 $[z^n]A_u(z, 1)/[z^n]A(z, 1)$ ；使用 EGF 时，分子分母中的 $n!$ 相消。概率生成函数的对应公式见 `34-probability-expectation.md`。

从递推到有理生成函数

若从某处开始满足

$$a_n = c_1 a_{n-1} + \cdots + c_d a_{n-d},$$

则 OGF 的分母是

$$Q(x) = 1 - c_1 x - \cdots - c_d x^d.$$

计算 $P(x) = A(x)Q(x)$ 的前 d 项即可确定分子。求很远的单项系数可用 `14-分式求系数.md` 的 Bostan-Mori；递推本身未知时可用 `20-linear-recurrence.md` 的 Berlekamp-Massey。

系数提取技巧

- 平移: $[x^n]x^k A(x) = [x^{n-k}]A(x)$;
- 差分: $(1-x)A(x)$ 的第 n 项是 $a_n - a_{n-1}$; 前缀和对应除以 $1-x$;
- 奇偶项: $(A(x) + A(-x))/2$ 保留偶次项, $(A(x) - A(-x))/2$ 保留奇次项;
- 模 m 的下标筛选可使用单位根滤子, 但模意义下必须存在所需的 m 次单位根且 m 可逆;
- 若式子是 $P(x)/Q(x)$, 先看能否约分或降低分母次数, 再调用通用算法;
- 二元生成函数中可先固定一个变量取系数, 把问题化成另一个变量的一元多项式。

两个建模例子

用 1, 2, 3 组成总和为 n 的有序序列, 每个数可重复使用。单个位置贡献 $x + x^2 + x^3$, 序列 OGF 为

$$\frac{1}{1 - (x + x^2 + x^3)},$$

从而得到递推 $a_n = a_{n-1} + a_{n-2} + a_{n-3}$ 。

把 n 个不同球放进 k 个有标号盒子且每盒非空: 一个非空盒子的 EGF 是 $e^x - 1$, k 个有标号盒子给出 $(e^x - 1)^k$, 答案为

$$n![x^n](e^x - 1)^k = k! \left\{ \begin{matrix} n \\ k \end{matrix} \right\}.$$

计算所需的多项式逆、对数、指数和幂见 `16-polynomial.md`。截断到 x^N 时, 每一步操作都应立即截断, 避免无意义的高次系数膨胀。

结构化万能欧几里得

这里的“万能欧几里得”不是把几个求和式放进同一个递归, 而是把直线对应的格路编码成 `U/R` 字符串, 再用任意幺半群维护这个字符串。

设

$$y_i = \left\lfloor \frac{ai + b}{c} \right\rfloor, \quad a, b, n \geq 0, \quad c > 0.$$

从 $x = 0$ 走到 $x = n$: 每跨过一条水平网格线记 `U`, 每跨过一条竖直网格线记 `R`; 若同时经过整点, 约定先 `U` 后 `R`。于是第 i 个 `R` 之前恰有 y_i 个 `U`。例如 $n = 6, a = 3, b = 2, c = 5$ 时

```
y_1, ..., y_6 = 1, 1, 2, 2, 3, 4
word          = U R R U R R U R U R
```

算法只要求状态满足:

- `Info{}` 是空串;
- `lhs * rhs` 是两段路径按顺序拼接, 必须满足结合律;
- `U`、`R` 分别是单个字符的状态。

不要求乘法交换。矩阵、哈希、自动机转移以及各种路径统计都可以作为 `Info`。

通用模板

`euclidean_word(n,a,b,c,up,right)` 返回区间 $(0,n]$ 生成的完整路径状态；当 $n > 0$ 时，它包含初始高度 $\lfloor b/c \rfloor$ 个 U，并且最后一个字符是第 n 个 R；当 $n = 0$ 时返回空串。参数和中间乘法使用无符号 64 位与 u128，以免 $a*n$ 溢出。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using u64 = unsigned long long;
using u128 = __uint128_t;
using i64 = long long;
using i128 = __int128_t;

template<class Info>
Info monoid_power(Info base, u64 exponent) {
    Info result;
    while (exponent > 0) {
        if (exponent & 1ULL) result = result * base;
        base = base * base;
        exponent >>= 1;
    }
    return result;
}

template<class Info>
Info euclidean_word(u64 n, u64 a, u64 b, u64 c,
                    const Info& up, const Info& right) {
    assert(c > 0);
    if (n == 0) return Info{};

    if (b >= c) {
        return monoid_power(up, b / c) *
            euclidean_word(n, a, b % c, c, up, right);
    }
    if (a >= c) {
        Info new_right = monoid_power(up, a / c) * right;
        return euclidean_word(n, a % c, b, c, up, new_right);
    }

    // 此时  $0 \leq a, b < c$ 。
    u64 up_count = (u64)((u128)a * n + b) / c;
    if (up_count == 0) return monoid_power(right, n);

    // 转置坐标轴。头尾两段必须保留，不能只交换  $a$  与  $c$ 。
    u64 first_rights = (c - b - 1) / a;
    u64 last_up_position =
        (u64)((u128)c * up_count - b - 1) / a;
    assert(last_up_position <= n);

    return monoid_power(right, first_rights) * up *
        euclidean_word(up_count - 1, c, (c - b - 1) % a, a,
            right, up) *
        monoid_power(right, n - last_up_position);
}
```


}

递归深度与 Euclid 算法相同。若一次状态合并代价为 T ，复杂度为 $O(T \log \max(a, c, n))$ ；额外的 $\log n$ 来自幺半群快速幂。很多状态可为纯段构造闭式幂，但通用模板保留快速幂最稳妥。

为什么递归成立

以下顺序很重要，因为 `operator*` 不一定交换。

1. 若 $b \geq c$ ，整条线先整体升高 $\lfloor b/c \rfloor$ ，答案前缀是 $U^{(b/c)}$ 。
2. 若 $a \geq c$ ，每次向右前固定多走 $\lfloor a/c \rfloor$ 次 U ，故把原子 R 替换为 $U^{(a/c)} R$ 。
3. 若 $0 < a < c$ ，令 $m = \lfloor (an + b)/c \rfloor$ 。交换坐标轴后，第 i 个 U 前的 R 数为

$$\left\lfloor \frac{ci - b - 1}{a} \right\rfloor.$$

-1 正是“整点先 U 后 R ”的边界修正。提出第一个 U 之前与最后一个 U 之后的 R ，中间部分就是交换 U/R 后的同类子问题：

$$R^{\lfloor (c-b-1)/a \rfloor} U F(m-1, c, (c-b-1) \bmod a, a, R, U) R^{n-\lfloor (cm-b-1)/a \rfloor}.$$

若 $m = 0$ ，路径只有 R^n 。这四种情况就是完整的结构化算法。

示例：怎样从目标式设计结构体

假设最终要求

$$\sum_{i=0}^n y_i, \quad \sum_{i=0}^n y_i^2, \quad \sum_{i=0}^n i y_i.$$

重点不是记住下面的代码，而是学会从目标式推出一个对“字符串拼接”封闭的结构体。

第一步：规定每个字段的局部含义

任取一段 U/R 串 S ，都从局部原点 $(0, 0)$ 开始执行。每遇到一次 U 令 $y \leftarrow y + 1$ ；每遇到一次 R ，先令 $x \leftarrow x + 1$ ，再记录此时的 (x, y) 。

结构体维护：

	字段	含义
x	串中 R 的数量，即终点横坐标	
y	串中 U 的数量，即终点纵坐标	
s_x	每个 R 处的横坐标之和	$\sum x$
s_y	每个 R 处的纵坐标之和	$\sum y$
s_{y^2}		$\sum y^2$
s_{xy}		$\sum xy$

为什么还要维护看似不是答案的 x, y, s_x ？因为把右段接到左段后，右段的每个局部坐标 (x', y') 都会变成

$$(x_L + x', y_L + y').$$

展开目标中的 xy 会出现 $x_L y'$ 、 $y_L x'$ 和 $x_L y_L$ ，所以必须同时知道右段的 s_x, s_y 以及两段的 R/U 数量。设计结构体的一般方法正是：先放入答案，再反复展开拼接式，把缺失的辅助统计量补到不再产生新量为止。

第二步：推导左右段的合并式

设整串是 $L + R$ 。左段里的坐标不变；右段的坐标整体平移 (x_L, y_L) 。于是

$$\begin{aligned}x &= x_L + x_R, \\y &= y_L + y_R, \\s_x &= s_{x,L} + s_{x,R} + x_L x_R, \\s_y &= s_{y,L} + s_{y,R} + y_L x_R, \\s_{y^2} &= s_{y^2,L} + s_{y^2,R} + 2y_L s_{y,R} + y_L^2 x_R, \\s_{xy} &= s_{xy,L} + s_{xy,R} + x_L s_{y,R} + y_L s_{x,R} + x_L y_L x_R.\end{aligned}$$

最后一行直接来自

$$(x_L + x')(y_L + y') = x'y' + x_L y' + y_L x' + x_L y_L.$$

这一步也说明拼接通常不交换： $L * R$ 和 $R * L$ 的平移量不同。

第三步：写空串与两个原子

- 空串：六个字段全为 0，它就是乘法单位元。
- 单个 U：终点为 $(0,1)$ ，还没有遇到 R，所以只有 $y = 1$ 。
- 单个 R：执行后记录点 $(1,0)$ ，所以 $x = 1, s_x = 1$ ，其余为 0。

例如把 URR 与 UR 拼接：

$$L = (2, 1, 3, 2, 2, 3), \quad R = (1, 1, 1, 1, 1, 1),$$

字段顺序为 $(x, y, s_x, s_y, s_{y^2}, s_{xy})$ 。代入合并式得到

$$L + R = (3, 2, 6, 4, 6, 9),$$

与直接扫描 URRUR 时三个 R 上的坐标 $(1,1), (2,1), (3,2)$ 完全一致。先用这种短串手算，是检查合并式最有效的方法。

第四步：把推导翻译成结构体

下面的代码只是上述六条公式的逐项翻译，所有量对 998244353 取模；Euclid 模板本身完全不知道这些字段的含义。

```
struct FloorPathInfo {
    static constexpr i64 mod = 998244353;

    i64 x = 0;
    i64 y = 0;
    i64 sum_x = 0;
    i64 sum_y = 0;
    i64 sum_y_squared = 0;
    i64 sum_xy = 0;
```

```

friend FloorPathInfo operator*(const FloorPathInfo& left,
                               const FloorPathInfo& right) {
    auto norm = [](i128 value) -> i64 {
        value %= mod;
        if (value < 0) value += mod;
        return (i64)value;
    };

    FloorPathInfo result;
    result.x = (left.x + right.x) % mod;
    result.y = (left.y + right.y) % mod;
    result.sum_x = norm((i128)left.sum_x + right.sum_x +
                        (i128)left.x * right.x);
    result.sum_y = norm((i128)left.sum_y + right.sum_y +
                        (i128)left.y * right.x);
    result.sum_y_squared = norm(
        (i128)left.sum_y_squared + right.sum_y_squared +
        (i128)right.x * left.y % mod * left.y +
        (i128)2 * left.y * right.sum_y);
    result.sum_xy = norm(
        (i128)left.sum_xy + right.sum_xy +
        (i128)right.x * left.x % mod * left.y +
        (i128)left.x * right.sum_y +
        (i128)left.y * right.sum_x);
    return result;
}

static FloorPathInfo up() {
    FloorPathInfo result;
    result.y = 1;
    return result;
}

static FloorPathInfo right() {
    FloorPathInfo result;
    result.x = 1;
    result.sum_x = 1;
    return result;
}
};

struct FloorMoments {
    i64 floor_sum = 0;
    i64 square_sum = 0;
    i64 index_times_floor = 0;
};

FloorMoments floor_moments(u64 n, u64 a, u64 b, u64 c) {
    assert(c > 0);
    FloorPathInfo path = euclidean_word(
        n, a, b, c, FloorPathInfo::up(), FloorPathInfo::right());

    constexpr i64 mod = FloorPathInfo::mod;

```

```
i64 initial_height = (i64)((b / c) % (u64)mod);
return {
    (path.sum_y + initial_height) % mod,
    (path.sum_y_squared +
    (i64)((i128)initial_height * initial_height % mod)) % mod,
    path.sum_xy
};
}
```

floor_moments(n,a,b,c) 计算闭区间 $0 \leq i \leq n$ 上的

$$\sum y_i, \quad \sum y_i^2, \quad \sum iy_i.$$

之所以额外补一次初始高度，是因为路径中的每个 R 记录 $i = 1, \dots, n$ ，而 $i = 0$ 没有对应的 R。

例如 floor_moments(5,3,2,5) 对应

$$(y_0, \dots, y_5) = (0, 1, 1, 2, 2, 3),$$

结果为 floor_sum=9、square_sum=19、index_times_floor=32。

如何迁移到别的题

先完全忘掉 Euclid 递归，只按题意设计一个能表示任意 U/R 串的结构：

- 1. 写出单个 U 与单个 R 的状态；
- 2. 推导 left + right 的拼接公式，特别检查顺序；
- 3. 确认空状态是单位元、拼接满足结合律；
- 4. 把三个对象直接传给 euclidean_word。

常见应用包括 $\sum i^p y_i^q$ 、 $\sum A^i B^{y_i}$ 、路径哈希以及在格路上复合转移。所谓“万能”，指的是 Euclid 部分无需随统计量重推；真正变化的只有幺半群状态。

完整多项式与形式幂级数库

这是自包含的高级版本，固定模数为 998244353。若只需要卷积、逆、对数、指数和普通整数幂，使用 16-polynomial.md 更短；需要下表中的高级操作时使用本文件。

接口	含义	主要前提
fps_inv	形式幂级数逆	$f_0 \neq 0$
fps_ln	形式对数	$f_0 = 1$
fps_exp	形式指数	$f_0 = 0$
fps_pow	十进制大指数幂	指数非负
fps_sqrt	形式平方根	前导零次数为偶数且首项有平方根
divmod	多项式商和余数	除式非零
multipoint_eval	多点求值	无
fast_interpolate	多点插值	横坐标两两不同
bostan_mori	$[x^k]P/Q$	$Q(0) \neq 0$
taylor_shift	$f(x+c)$	次数小于模数
compose	$f(g(x))$	返回模 $x^{\deg f+1}$
power_projection	同时求 $[x^k]f(x)^i$	给定目标次数与项数

接口	含义	主要前提
<code>compositional_inverse</code>	复合逆	$f(0) = 0, f'(0) \neq 0$

多项式均按升幂存储。代码中的二维 Bostan-Mori 使用 Kronecker 代换，复合、幂投影和复合逆不是朴素占位实现。

方法、推导与复杂度

下面统一记 $M(n)$ 为两个 n 次以内多项式的卷积复杂度；在本模数下使用 NTT，故 $M(n) = O(n \log n)$ 。所有 FPS 接口的第二个参数 n 都表示只保留模 x^n 的前 n 项。

NTT 与卷积

DFT 把卷积变成逐点乘法。模 998244353 中存在最大阶为 2^{23} 的单位根，因此可把复数单位根替换为有限域单位根：

$$\hat{a}_k = \sum_{j=0}^{N-1} a_j \omega_N^{jk}, \quad a_j = N^{-1} \sum_{k=0}^{N-1} \hat{a}_k \omega_N^{-jk}.$$

Cooley-Tukey 每层按下标奇偶拆分，代码用位逆序迭代实现，并缓存每种长度的旋转因子。卷积先补零到不小于 $|A| + |B| - 1$ 的二次幂，正变换后逐点相乘再逆变换。复杂度 $O(M(n))$ ，长度不得超过 2^{23} 。

FPS Newton 迭代的一般形式

令 $h(x)$ 为已知 FPS、 $f(x)$ 为待求 FPS。先把目标关系写成

$$g(f) = F(f) - h = 0,$$

然后把 h 当作常量，对未知量 f 求导。对当前近似值加入修正量 Δ ：

$$g(f + \Delta) = g(f) + \frac{\partial g}{\partial f} \Delta + O(\Delta^2).$$

令一次项抵消当前误差，得到

$$\Delta = - \left(\frac{\partial g}{\partial f} \right)^{-1} g(f), \quad f_{\text{new}} = f + \Delta.$$

下面的求逆、指数和平方根都从这一步代入对应的 $g(f)$ ，而不是直接背最终更新式。这里涉及的导数都是对未知 FPS f 的形式导数；在交换的 FPS 环中，下列导数算子都是乘法算子，所以可以写成通常的分式形式。

FPS 逆：牛顿迭代从哪里来

已知 h ，要求 $f = h^{-1} \pmod{x^n}$ 。写成

$$g(f) = f^{-1} - h = 0, \quad \frac{\partial g}{\partial f} = -f^{-2}.$$

代入 Newton 公式：

$$f_{\text{new}} = f - \frac{f^{-1} - h}{-f^{-2}} = f(2 - hf).$$

也可直接看误差：若 $hf = 1 - e$ 且 $e \equiv 0 \pmod{x^m}$ ，则

$$hf(2 - hf) = (1 - e)(1 + e) = 1 - e^2 \equiv 1 \pmod{x^{2m}}.$$

所以每轮正确长度由 m 翻倍到 $2m$ 。初值为 $f_0 = h_0^{-1}$ ，故要求 $h_0 \neq 0$ ；总复杂度 $O(M(n))$ 。

求导、积分与形式对数

求导、积分逐项进行。积分的第 i 项要除以 i ，所以截断次数必须小于模数。由

$$(\ln f)' = \frac{f'}{f}$$

得到

$$\ln f = \int f' f^{-1} dx.$$

实现约定积分常数为 0，因此标准接口要求 $f_0 = 1$ 。复杂度由一次求逆和卷积主导，为 $O(M(n))$ 。

FPS 指数：对 $\ln f - h = 0$ 做牛顿迭代

已知 h ，要求 $f = \exp(h)$ 。写成

$$g(f) = \ln f - h = 0, \quad \frac{\partial g}{\partial f} = f^{-1}.$$

Newton 更新为

$$f_{\text{new}} = f - \frac{\ln f - h}{f^{-1}} = f(1 + h - \ln f).$$

若 f 已在模 x^m 下正确，新值就在模 x^{2m} 下正确。形式指数的常数项固定为 1，故输入要求 $h_0 = 0$ 。复杂度 $O(M(n))$ 。

FPS 幂

先找到最低非零项 $f_t x^t$ ，写成

$$f(x) = x^t f_t h(x), \quad h(0) = 1.$$

于是对非负整数 k ，

$$f(x)^k = x^{tk} f_t^k \exp(k \ln h(x)).$$

这里必须把 k 的三种用途分开处理，不能笼统地说“指数取模”。设模数为质数 $p = 998244353$ ：

1. 位移 x^{tk} ：使用原始整数 k 。指数代表次数，不是有限域元素，不能对 p 或 $p-1$ 取模。算法只需判断 $tk < n$ ；若 $tk \geq n$ ，模 x^n 的答案全为零。

2. 非零常数 f_t^k : 指数对 $p-1$ 取模。因为 $f_t \in \mathbb{F}_p^*$, 由 Fermat 定理 $f_t^{p-1} = 1$, 所以这里只需 $k \bmod (p-1)$ 。
3. 单位 FPS 的 $\exp(k \ln h)$: 标量 k 对 p 取模。 $\ln h$ 的每个系数都在 $\mathbb{F}_p$ 中, 乘整数 k 就是把 k 映射到域内, 因此只取 $k \bmod p$ 。

第三点也可直接用 Frobenius 解释。因为 $h(0) = 1$, 在特征 p 的域中

$$h(x)^p = h(x^p) = 1 + O(x^p).$$

当只保留模 x^n 且 $n \leq p$ 的项时, $h(x)^p \equiv 1 \pmod{x^n}$, 于是

$$h(x)^{k+p} \equiv h(x)^k \pmod{x^n}.$$

这就是单位部分的指数周期为 p 的直接原因。例如在 $\mathbb{F}_7$ 中, 模 x^7 有

$$(1+x)^8 = (1+x)(1+x)^7 \equiv 1+x,$$

与指数 $8 \bmod 7 = 1$ 一致; 但若保留到 x^7 项, 即模 x^8 , 则 $(1+x)^7 = 1+x^7 \not\equiv 1$, 此时便不能把指数只对 7 取模。这也说明模板为什么要求截断长度 $n \leq p$ 。

综上, 代码分别计算: 用于位移判断的有界原始 k 、 $k_mod = k \bmod p$ 、 $k_phi = k \bmod (p-1)$ 。三者不能互换。复杂度 $O(M(n))$ 。

FPS 平方根

先删除前导零。最低非零次数必须为偶数, 否则不可能是平方; 首项还必须在模意义下有平方根, 代码用 Tonelli-Shanks 求它。

记去掉前导项后的已知 FPS 为 h , 待求平方根为 f 。从

$$g(f) = f^2 - h = 0, \quad \frac{\partial g}{\partial f} = 2f$$

开始, Newton 更新为

$$f_{\text{new}} = f - \frac{f^2 - h}{2f} = \frac{1}{2} \left(f + \frac{h}{f} \right).$$

每轮同样把正确精度翻倍, 复杂度 $O(M(n))$ 。平方根通常有正负两支, 接口返回 Tonelli-Shanks 选出的其中一支; 无解返回空数组。

多项式除法

若 $A = BQ + R$ 且 $\deg R < \deg B$, 设商长为 $k = \deg A - \deg B + 1$ 。把系数反转后, 高次项变成低次项, 而余数落到不影响前 k 项的位置:

$$\text{rev}(Q) \equiv \text{rev}(A) \text{rev}(B)^{-1} \pmod{x^k}.$$

求出商后再算 $R = A - BQ$ 。复杂度 $O(M(n))$, 除式必须非零。

多点求值

对点 x_i 建乘积树：叶子是 $M_i(x) = x - x_i$ ，结点维护

$$M_{[l,r)}(x) = \prod_{i=l}^{r-1} (x - x_i).$$

根处先求 $f \bmod M$ ，再向下分别对两个儿子的 M 取模；叶子的常数余数就是 $f(x_i)$ 。模板把乘积树存成反转形式，使每层取模归约为卷积，复杂度 $O(M(n) \log n)$ 。

快速插值

令 $M(x) = \prod_i (x - x_i)$ 。Lagrange 公式写成

$$f(x) = \sum_i \frac{y_i}{M'(x_i)} \frac{M(x)}{x - x_i}.$$

先用多点求值算出全部 $M'(x_i)$ ，叶子权值为 $w_i = y_i / M'(x_i)$ 。乘积树上合并两个区间时使用

$$F_{L \cup R} = F_L M_R + F_R M_L.$$

根结点即插值多项式。批量逆元只做一次快速幂；横坐标必须两两不同。复杂度 $O(M(n) \log n)$ 。

Bostan-Mori

要求 $[x^k]P(x)/Q(x)$ 。构造 $Q(-x)$ 后，

$$Q(x)Q(-x)$$

只含偶次项。若 k 为偶数， $P(x)Q(-x)$ 只保留偶次项；若 k 为奇数则只保留奇次项。随后令 $x^2 \leftarrow x$ 、 $k \leftarrow \lfloor k/2 \rfloor$ ，目标下标每轮减半。复杂度 $O(M(d) \log k)$ ，其中 d 是分母次数，并要求 $Q(0) \neq 0$ 。

Taylor shift

由二项式定理，

$$[x^j]f(x+c) = \frac{1}{j!} \sum_{i \geq j} (f_i i!) \frac{c^{i-j}}{(i-j)!}.$$

把序列 $f_i i!$ 反转后，右侧就是一次普通卷积，故可在 $O(M(n))$ 内求出全部系数。

多项式复合

目标是 $f(g(x)) \bmod x^{n+1}$ 。当 $g(0) \neq 0$ 时，先做 Taylor shift：把常数项吸收到 $f(x+g_0)$ ，再令 $g_0 = 0$ 。

对 $g(0) = 0$ ，使用二元恒等式

$$\sum_{i \geq 0} g(x)^i y^i = \frac{1}{1 - yg(x)}.$$

于是复合可转化为关于 x, y 的二维 Bostan-Mori。每轮用 $V(-x, y)$ 消掉分母的奇数次 x ，再令 $x^2 \leftarrow x$ 。实现先保存每轮分母核，再反向应用转置乘法；二维卷积通过 Kronecker 代换

$$x^i y^j \mapsto z^{i \cdot \text{stride} + j}$$

压成一次 NTT。每轮 x 次数约减半、 y 次数约翻倍，打包后的一维有效长度保持在线性量级；总复杂度为 $O(M(n) \log n)$ 。这个实现面向大规模复合，不是朴素 $O(n^2)$ 版本。

转置原理：乘法、除法和复合的转置

在有限维系数空间中取标准内积

$$\langle a, b \rangle = \sum_i a_i b_i.$$

若线性映射 L 满足

$$\langle L(a), b \rangle = \langle a, L^\top(b) \rangle,$$

则 $L^\top$ 称为 L 的转置。转置原理说：一段只包含加法、常数乘法和复制的直线程序，倒序执行并把“取值”改成“累加贡献”，就能以同阶复杂度计算转置映射。这里的转置是线性代数意义，不是简单 **reverse**；反转系数只是实现卷积转置时出现的工具。

乘法的转置：middle product 固定 f ，考虑截断乘法

$$h_k = [x^k]f(x)g(x) = \sum_i f_i g_{k-i}.$$

若输出端给定权值 w_k ，则

$$\sum_k w_k h_k = \sum_j g_j \left(\sum_i f_i w_{i+j} \right).$$

所以转置后第 j 项是 $\sum_i f_i w_{i+j}$ 。它等价于把 f 或 w 反转后做一次卷积，再截取中间连续的一段，因此通常称为 **middle product**。完整代码中的 `multiply_transpose` 正是在二维 Kronecker 编码下做这个操作。

固定除式时，除法的转置 FPS 除法在除式 q 固定时是关于被除式 p 的线性映射：

$$D_q(p) = p q^{-1} \pmod{x^n}.$$

因此

$$D_q^\top = M_{q^{-1}}^\top,$$

即先求一次 q^{-1} ，再使用上面的 middle product。若讨论普通多项式长除法的“取商”映射，则正向过程为

$$p \xrightarrow{\text{反转、截取}} \text{rev}(p) \xrightarrow{\times \text{rev}(q)^{-1}} \text{rev}(\text{quotient}),$$

转置时必须把这三步反向执行：先把输出权值放回对应截取区间，再做乘法的转置，最后反转回来。不能只写成“再除一次 q ”。

例如在模 x^4 下固定

$$q^{-1}(x) = r_0 + r_1x + r_2x^2 + r_3x^3.$$

正向输出 $h_k = \sum_{i=0}^k p_i r_{k-i}$ 。给定输出权值 $w_0, \dots, w_3$ ，转置输出为

$$p_j \text{ 的权值} = \sum_{k=j}^3 w_k r_{k-j},$$

这正是一段相关运算，也就是反转卷积后的中间段。

复合的转置就是幂投影 固定内层多项式 g ，映射

$$C_g : f \mapsto f(g(x)) \pmod{x^n}$$

对 f 的系数是线性的，因为

$$f(g(x)) = \sum_i f_i g(x)^i.$$

设输出端权值多项式为 w ，则

$$\begin{aligned} \langle C_g(f), w \rangle &= \sum_j w_j [x^j] \sum_i f_i g(x)^i \\ &= \sum_i f_i \left(\sum_j w_j [x^j] g(x)^i \right). \end{aligned}$$

故

$$(C_g^T(w))_i = \sum_j w_j [x^j] g(x)^i.$$

若 $w = x^k$ ，转置输出恰为

$$([x^k]g(x)^0, [x^k]g(x)^1, \dots),$$

这就是幂投影。换句话说，`power_projection(g,k,m)` 是“复合映射的转置作用在第 k 个坐标基向量上”，而不是另一个无关技巧。

例如 $g = x + x^2$ ，取 $k = 2$ ，则

$$[x^2]g^0 = 0, \quad [x^2]g^1 = 1, \quad [x^2]g^2 = 1, \quad [x^2]g^i = 0 \quad (i \geq 3).$$

因此前四项幂投影是 $(0, 1, 1, 0)$ 。它与任意 $f = f_0 + f_1x + f_2x^2 + f_3x^3$ 做内积，得到 $f_1 + f_2$ ，也正是 $[x^2]f(x + x^2)$ 。

需要注意： C_g 只在固定 g 、变化 f 时是线性的；把 f 固定而改变 g 并不是线性映射，不能直接套转置原理。

幂投影：复合转置的专用实现

要求同时得到

$$c_i = [x^k]f(x)^i, \quad 0 \leq i < m.$$

仍由

$$\sum_{i \geq 0} f(x)^i y^i = \frac{1}{1 - yf(x)}$$

可知答案是该二元有理式在 x^k 处、关于 y 的前 m 项。直接执行二维 Bostan-Mori，每轮按当前 k 的奇偶抽取分子、只抽取分母偶次项即可。

打包卷积部分复杂度为 $O(M(k) \log k)$ ，最后还需一次长度 m 的 FPS 逆，合计 $O(M(k) \log k + M(m))$ 。

复合逆

要求 $G(F(x)) = x \pmod{x^n}$ ，前提是 $F(0) = 0, F'(0) \neq 0$ 。先除以线性项，把 $A'(0)$ 归一为 1。设 $H = A^{(-1)}$ ，固定 $N = n - 1$ ；Lagrange 反演把 H 的系数转化为 $[x^N]A(x)^k$ ，这些值可由一次幂投影同时得到。构造

$$S(x) = \sum_{k=1}^N \frac{N}{k} [x^N]A(x)^k x^{N-k} = \left(\frac{x}{H(x)} \right)^N,$$

便有

$$H(x) = xS(x)^{-1/N}.$$

最后缩放变量恢复原来的线性系数。代码用 `ln/exp` 计算 $S^{-1/N}$ 。

它由一次规模 n 的幂投影和常数次 FPS 操作组成，复杂度为 $O(M(n) \log n)$ 。

完整实现

```
#include<vector>
#include<array>
#include<iostream>
#include<algorithm>
#include<limits>
#include<cassert>
#include<optional>
#include<string>
#include<random>
using namespace std;

// =====
// 1. NTT
// =====

template <int modulus_, int root_, int inverse_root_>
struct NTT {
    static constexpr int mod = modulus_;
```

```

static constexpr int root = root_;
static constexpr int iroot = inverse_root_;

static int qpow(int a, long long e) {
    int r = 1;
    while (e) {
        if (e & 1) r = (int)(1LL * r * a % mod);
        a = (int)(1LL * a * a % mod);
        e >>= 1;
    }
    return r;
}

static int add(int a, int b) {
    a += b;
    return a >= mod ? a - mod : a;
}

static int sub(int a, int b) {
    a -= b;
    return a < 0 ? a + mod : a;
}

private:
    inline static vector<vector<int>> rev_cache;
    inline static vector<vector<int>> roots, iroots;
    inline static int built_level = 0;

    static const vector<int>& get_rev(int level) {
        if ((int)rev_cache.size() <= level)
            rev_cache.resize(level + 1);

        auto& rev = rev_cache[level];
        if (!rev.empty()) return rev;

        int n = 1 << level;
        rev.resize(n);
        for (int i = 1; i < n; ++i)
            rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (level - 1));
        return rev;
    }

    static void ensure_roots(int level) {
        if (level <= built_level) return;

        roots.resize(level + 1);
        iroots.resize(level + 1);

        for (int t = built_level + 1; t <= level; ++t) {
            int half = 1 << (t - 1);
            roots[t].resize(half);
            iroots[t].resize(half);
        }
    }

```

```

        int w = qpow(root, (mod - 1) >> t);
        int iw = qpow(iroot, (mod - 1) >> t);
        roots[t][0] = iroots[t][0] = 1;

        for (int i = 1; i < half; ++i) {
            roots[t][i] = (int)(1LL * roots[t][i - 1] * w % mod);
            iroots[t][i] = (int)(1LL * iroots[t][i - 1] * iw % mod);
        }
    }
    built_level = level;
}

public:
    // a.size() 必须是 2 的幂；正逆变换均为自然顺序输入、自然顺序输出。
    static void transform(vector<int>& a, bool inverse = false) {
        int n = (int)a.size();
        assert(n > 0 && (n & (n - 1)) == 0);
        assert((mod - 1) % n == 0); // 防止 NTT 长度超过模数支持范围

        int level = __builtin_ctz((unsigned)n);
        ensure_roots(level);
        const auto& rev = get_rev(level);

        // 同一 NTT 长度的位逆序表只构造一次。
        for (int i = 0; i < n; ++i)
            if (i < rev[i]) swap(a[i], a[rev[i]]);

        const auto& table = inverse ? iroots : roots;
        for (int half = 1, t = 1; half < n; half <= 1, ++t) {
            int block = half << 1;
            const auto& w = table[t];
            for (int l = 0; l < n; l += block) {
                for (int j = 0; j < half; ++j) {
                    int u = a[l + j];
                    int v = (int)(1LL * a[l + j + half] * w[j] % mod);
                    a[l + j] = add(u, v);
                    a[l + j + half] = sub(u, v);
                }
            }
        }

        if (!inverse) return;
        int inv_n = qpow(n, mod - 2);
        for (int& x : a) x = (int)(1LL * x * inv_n % mod);
    }
};

// =====
// 2. 多项式基础设施
// =====

namespace polyops {

```

```

using poly = vector<int>;
using Ntt = NTT<998244353, 3, 332748118>;
static constexpr int mod = Ntt::mod;

// 取前  $n$  项, 不足补 0。
poly prefix(const poly& f, int n) {
    assert(n >= 0);
    poly r(f.begin(), f.begin() + min(n, (int)f.size()));
    r.resize(n);
    return r;
}

// 删除高位 0; 零多项式统一保留为 {0}。
void normalize(poly& f) {
    while (f.size() > 1 && f.back() == 0) f.pop_back();
    if (f.empty()) f.push_back(0);
}

// 多项式乘法。
// 参数按值传递: 调用方若不再需要原对象, 可显式 std::move 以避免复制。
poly mul(poly a, poly b) {
    if (a.empty() || b.empty()) return {};

    int na = (int)a.size();
    int nb = (int)b.size();

    if (min(na, nb) <= 32) {
        poly c(na + nb - 1);
        for (int i = 0; i < na; ++i) {
            long long ai = a[i];
            for (int j = 0; j < nb; ++j)
                c[i + j] = (int)((c[i + j] + ai * b[j]) % mod);
        }
        return c;
    }

    int need = na + nb - 1;
    int ntt_len = 1;
    while (ntt_len < need) ntt_len <<= 1;

    a.resize(ntt_len);
    b.resize(ntt_len);
    Ntt::transform(a);
    Ntt::transform(b);

    for (int i = 0; i < ntt_len; ++i)
        a[i] = (int)(1LL * a[i] * b[i] % mod);

    Ntt::transform(a, true);
    a.resize(need);
    return a;
}

```

```

// =====
// 3. FPS: 逆 / 导数 / 积分 / Ln / exp
// =====

//  $f^{-1} \bmod x^n$ , 要求  $f[0] \neq 0$ 。
//
// 设当前  $g$  已满足  $f * g = 1 \pmod{x^m}$ , 误差  $e = 1 - f * g$  含有因子  $x^m$ 。
// 令  $g_{\text{new}} = g * (2 - f * g) = g * (1 + e)$ , 则
//  $1 - f * g_{\text{new}} = 1 - (1 - e)(1 + e) = e^2 = 0 \pmod{x^{2m}}$ 。
// 因而每轮把正确项数翻倍, 这就是下面牛顿迭代的来源。
poly fps_inv(const poly& f, int n) {
    assert(n >= 0);
    if (n == 0) return {};
    assert(!f.empty() && f[0] != 0);

    poly g{Ntt::qpow(f[0], mod - 2)};
    if (n == 1) return g;

    for (int len = 2; len <= n) {
        int cur = min(len, n);
        poly f_cut = prefix(f, cur);

        // 在频域一次完成  $g * (2 - f * g)$ , 比写两次 convolution 常数更小。
        int ntt_len = 1;
        while (ntt_len < cur * 2) ntt_len <= 1;

        g.resize(ntt_len);
        f_cut.resize(ntt_len);
        Ntt::transform(g);
        Ntt::transform(f_cut);

        for (int i = 0; i < ntt_len; ++i) {
            int fg = (int)(1LL * f_cut[i] * g[i] % mod);
            g[i] = (int)(1LL * g[i] * ((2 - fg + mod) % mod) % mod);
        }

        Ntt::transform(g, true);
        g.resize(cur);
        if (cur == n) break;
    }
    return g;
}

poly derivative(const poly& f) {
    if (f.size() <= 1) return {};
    poly g(f.size() - 1);
    for (int i = 1; i < (int)f.size(); ++i)
        g[i - 1] = (int)(1LL * f[i] * i % mod);
    return g;
}

namespace fps_detail {

```

```

// 积分会频繁用到  $1/i$ , 线性预处理比每项快速幂更清楚也更快。
inline vector<int> inv_int{0, 1};

void ensure_inv_int(int n) {
    assert(n < mod);
    if ((int)inv_int.size() > n) return;

    int old = (int)inv_int.size();
    inv_int.resize(n + 1);
    for (int i = max(2, old); i <= n; ++i)
        inv_int[i] =
            (int)(mod - 1LL * (mod / i) * inv_int[mod % i] % mod);
}

} // namespace fps_detail

poly integral(const poly& f) {
    fps_detail::ensure_inv_int((int)f.size());
    poly g(f.size() + 1);
    for (int i = 0; i < (int)f.size(); ++i)
        g[i + 1] =
            (int)(1LL * f[i] * fps_detail::inv_int[i + 1] % mod);
    return g;
}

//  $\ln(f) \bmod x^n = \text{integral}(f' / f)$ , 要求  $f[0] \neq 0$ 。
// 这是形式幂级数恒等式  $(\ln f)' = f'/f$ ; 积分常数约定为 0。
poly fps_ln(const poly& f, int n) {
    assert(n >= 0);
    if (n == 0) return {};
    assert(!f.empty() && f[0] != 0);

    poly df = derivative(f);
    poly inv_f = fps_inv(f, n);
    poly g = mul(df, inv_f);
    g.resize(max(0, n - 1));
    g = integral(g);
    g.resize(n);
    return g;
}

//  $\exp(f) \bmod x^n$ , 要求  $f[0] = 0$ 。
//
// 把目标写成方程  $\Phi(g) = \ln(g) - f = 0$ 。由于  $\Phi'(g) = 1/g$ , 牛顿迭代为
//  $g_{\text{new}} = g - (\ln g - f) / (1/g) = g(1 + f - \ln g)$ 。
// 若  $g$  在模  $x^m$  下正确, 新值就在模  $x^{2m}$  下正确。
poly fps_exp(const poly& f, int n) {
    assert(n >= 0);
    if (n == 0) return {};
    assert(f.empty() || f[0] == 0);

    poly g{1};
    for (int len = 2;; len <= 1) {

```



```

    int cur = min(len, n);

    poly delta = prefix(f, cur);
    poly log_g = fps_ln(g, cur);
    for (int i = 0; i < cur; ++i)
        delta[i] = (delta[i] - log_g[i] + mod) % mod;
    delta[0] = (delta[0] + 1) % mod;

    g = mul(g, delta);
    g.resize(cur);
    if (cur == n) break;
}
return g;
}

// =====
// 4. FPS 幂
// =====

namespace power_detail {

long long decimal_mod(const string& s, long long m) {
    long long r = 0;
    for (char ch : s) r = (r * 10 + ch - '0') % m;
    return r;
}

bool is_zero(const string& s) {
    for (char ch : s)
        if (ch != '0') return false;
    return true;
}

// 若十进制整数  $\leq limit$ , 返回其值; 否则返回 nullopt。
optional<long long> parse_bounded(const string& s, long long limit) {
    long long x = 0;
    for (char ch : s) {
        int d = ch - '0';
        if (x > limit / 10 || (x == limit / 10 && d > limit % 10))
            return nullopt;
        x = x * 10 + d;
    }
    return x;
}

} // namespace power_detail

//  $f^k \bmod x^n$ ,  $k$  用十进制字符串表示, 可非常大。
poly fps_pow(const poly& f, const string& k_str, int n) {
    assert(n >= 0);
    assert(n <= mod);
    if (n == 0) return {};

```

```

if (power_detail::is_zero(k_str)) {
    poly ans(n);
    ans[0] = 1;
    return ans;
}

int valuation = 0;
while (valuation < (int)f.size() && f[valuation] == 0) ++valuation;
if (valuation == (int)f.size()) return poly(n, 0);

//  $x^{\text{valuation} * k}$  若已经达到  $x^n$ , 答案直接为 0。
long long shift = 0;
if (valuation > 0) {
    long long max_k = (n - 1) / valuation;
    auto k_small = power_detail::parse_bounded(k_str, max_k);
    if (!k_small) return poly(n, 0);
    shift = 1LL * valuation * *k_small;
}

long long k_mod = power_detail::decimal_mod(k_str, mod);
long long k_phi = power_detail::decimal_mod(k_str, mod - 1);

int lead = f[valuation];
int inv_lead = Ntt::qpow(lead, mod - 2);
int need = n - (int)shift;

// 分解  $f = x^{\text{valuation}} * \text{lead} * \text{unit}$ , 其中  $\text{unit}[0] = 1$ , 于是
//  $f^k = x^{\text{valuation} * k} * \text{lead}^k * \exp(k * \ln(\text{unit}))$ 。
//  $\text{unit}[0]=1$  且  $\text{need} \leq \text{mod}$ , 所以  $\text{unit}^{\text{mod}} = 1 \pmod{x^{\text{need}}}$ :
// 单位部分的标量  $k$  取模  $\text{mod}$ ; 非零常数  $\text{lead}$  的指数取模  $\text{mod}-1$ ;
// 位移  $\text{valuation} * k$  已在上面用原始整数单独判断, 三者不能混用。
poly unit(f.begin() + valuation, f.end());
unit.resize(need);
for (int& x : unit) x = (int)(1LL * x * inv_lead % mod);

poly log_unit = fps_ln(unit, need);
for (int& x : log_unit) x = (int)(1LL * x * k_mod % mod);
poly unit_pow = fps_exp(log_unit, need);

int lead_pow = Ntt::qpow(lead, k_phi);
poly ans(n);
for (int i = 0; i < need; ++i)
    ans[i + shift] = (int)(1LL * unit_pow[i] * lead_pow % mod);
return ans;
}

// =====
// 5. FPS 开方
// =====

namespace sqrt_detail {

// Tonelli-Shanks: 求模  $\text{mod}$  的平方根; 无解返回 -1。

```

```

int mod_sqrt(int x) {
    if (x == 0) return 0;
    if (Ntt::qpow(x, (mod - 1) / 2) != 1) return -1;

    int q = mod - 1, s = 0;
    while ((q & 1) == 0) q >>= 1, ++s;

    int z = 2;
    while (Ntt::qpow(z, (mod - 1) / 2) != mod - 1) ++z;

    int c = Ntt::qpow(z, q);
    int t = Ntt::qpow(x, q);
    int r = Ntt::qpow(x, (q + 1) / 2);
    int m = s;

    while (t != 1) {
        int i = 0;
        int tt = t;
        while (tt != 1) {
            tt = (int)(1LL * tt * tt % mod);
            ++i;
        }

        int b = Ntt::qpow(c, 1LL << (m - i - 1));
        int bb = (int)(1LL * b * b % mod);
        r = (int)(1LL * r * b % mod);
        t = (int)(1LL * t * bb % mod);
        c = bb;
        m = i;
    }
    return r;
}

} // namespace sqrt_detail

// sqrt(f) mod x^n: 不存在时返回空 vector。
// 去掉偶数阶前导零后, 对 g^2=f 使用牛顿迭代:
//      g_new = (g + f/g) / 2。
// 与求逆相同, 每轮把正确精度从 x^m 提升到 x^{2m}。
poly fps_sqrt(const poly& f, int n) {
    assert(n >= 0);
    if (n == 0) return {};

    int valuation = 0;
    while (valuation < (int)f.size() && f[valuation] == 0) ++valuation;
    if (valuation == (int)f.size()) return poly(n, 0);
    if (valuation & 1) return {};

    int shift = valuation / 2;
    if (shift >= n) return poly(n, 0);

    int root = sqrt_detail::mod_sqrt(f[valuation]);
    if (root == -1) return {};

```

```

int need = n - shift;
poly unit(f.begin() + valuation, f.end());
unit.resize(need);

static const int inv2 = (mod + 1) / 2;
poly g{root};

for (int len = 2;; len <= 1) {
    int cur = min(len, need);
    poly inv_g = fps_inv(g, cur);
    poly q = mul(prefix(unit, cur), inv_g);
    q.resize(cur);
    g.resize(cur);

    for (int i = 0; i < cur; ++i)
        g[i] = (int)(1LL * (g[i] + q[i]) * inv2 % mod);

    if (cur == need) break;
}

poly ans(n);
for (int i = 0; i < need; ++i) ans[i + shift] = g[i];
return ans;
}

// =====
// 6. 多项式除法
// =====

// 返回 {商, 余数}。输入按普通多项式解释, 会自动去掉高位 0。
//
// 若  $A=BQ+R$  且  $\deg R < \deg B$ , 把系数反转后,  $R$  会落在不影响前  $q\_len$  项的
// 高次位置, 因此有  $rev(Q)=rev(A)/rev(B) \pmod{x^{q\_len}}$ 。求出商后再用
//  $R=A-BQ$  恢复余数。这样多项式除法就归约成一次 FPS 求逆和两次乘法。
pair<poly, poly> divmod(poly a, poly b) {
    normalize(a);
    normalize(b);
    assert(!(b.size() == 1 && b[0] == 0));

    int n = (int)a.size();
    int m = (int)b.size();
    if (n < m) return {{0}, a};

    int q_len = n - m + 1;
    poly ra = a, rb = b;
    reverse(ra.begin(), ra.end());
    reverse(rb.begin(), rb.end());
    ra.resize(q_len);
    rb.resize(q_len);

    poly q = mul(ra, fps_inv(rb, q_len));
    q.resize(q_len);
    reverse(q.begin(), q.end());

```

```

poly prod = mul(q, b);
poly r(max(0, m - 1));
for (int i = 0; i < m - 1; ++i) {
    int pi = i < (int)prod.size() ? prod[i] : 0;
    r[i] = (a[i] - pi + mod) % mod;
}

normalize(q);
if (r.empty()) r = {0};
else normalize(r);
return {q, r};
}

// =====
// 7. 多点求值
// =====

namespace multipoint_detail {

struct Evaluator {
    vector<int> points;
    // 对区间  $[l, r)$ , 令  $M_p(x) = \prod_{i=l}^{r-1} (x - x_i)$ 。
    //  $tree[p]$  存的是固定长度  $r-l+1$  的  $rev(M_p)$ , 即  $prod(1-x_i*x)$ 。
    // 翻转表示既适合下面的快速取模, 也能被插值的自底向上合并直接复用。
    vector<poly> tree;
    vector<int> answer;

    void build(int p, int l, int r) {
        if (r - l == 1) {
            tree[p] = {1, points[l] ? mod - points[l] : 0};
            return;
        }

        int mid = (l + r) >> 1;
        build(p << 1, l, mid);
        build(p << 1 | 1, mid, r);
        tree[p] = mul(tree[p << 1], tree[p << 1 | 1]);
    }

    //  $q$  是当前区间余数的翻转表示。
    //
    // 从父亲向左儿子取模, 相当于在翻转域中乘上右儿子的  $rev(M_R)$ ,
    // 再截取中间连续的  $left\_len$  项; 向右同理。这个写法把每层的多项式
    // 除法变成两次卷积, 不必在每个结点各做一次  $FPS$  求逆, 常数明显更小。
    void descend(int p, int l, int r, const poly& q) {
        if (r - l == 1) {
            answer[l] = q[0];
            return;
        }

        int mid = (l + r) >> 1;
        int left_len = mid - l;
        int right_len = r - mid;

```

```

    poly to_left = mul(q, tree[p << 1 | 1]);
    poly left_q(left_len);
    for (int i = 0; i < left_len; ++i)
        left_q[i] = to_left[i + right_len];
    descend(p << 1, l, mid, left_q);

    poly to_right = mul(q, tree[p << 1]);
    poly right_q(right_len);
    for (int i = 0; i < right_len; ++i)
        right_q[i] = to_right[i + left_len];
    descend(p << 1 | 1, mid, r, right_q);
}

void init(const vector<int>& xs) {
    assert(!xs.empty());
    points.resize(xs.size());
    for (int i = 0; i < (int)xs.size(); ++i) {
        int x = xs[i] % mod;
        points[i] = x < 0 ? x + mod : x;
    }
    int m = (int)points.size();
    tree.assign(m << 2, {});
    answer.assign(m, 0);
    build(1, 0, m);
}

vector<int> evaluate(const poly& f) {
    int m = (int)points.size();
    assert(m > 0);

    // 根结点是  $rev(M)$ 。 $rev(f) / rev(M)$  的低位给出商；这里把除法与
    // 余数树合并整理后，只需根结点的一次逆，再由 descend 逐层下传。
    int work_len = max(m, (int)f.size());
    poly inv_root = fps_inv(tree[1], work_len);

    poly rev_f = f;
    rev_f.resize(work_len);
    reverse(rev_f.begin(), rev_f.end());
    poly q = mul(rev_f, inv_root);

    poly root_q(m);
    for (int i = 0; i < m; ++i)
        root_q[i] = q[work_len - m + i];

    descend(1, 0, m, root_q);
    return answer;
}

vector<int> solve(const poly& f, const vector<int>& xs) {
    if (xs.empty()) return {};
    init(xs);
    return evaluate(f);
}

```

```

};

} // namespace multipoint_detail

vector<int> multipoint_eval(const poly& f, const vector<int>& xs) {
    return multipoint_detail::Evaluator{}.solve(f, xs);
}

// =====
// 8. 多项式快速插值
// =====

namespace interpolation_detail {

// 批量求非零元素的逆：前缀积 + 一次快速幂 + 反向扫描。
// 与逐个 qpow 相比，把 n 次  $O(\log \text{mod})$  求逆降为一次，插值时常数更小。
vector<int> batch_inverse_nonzero(const vector<int>& a) {
    int n = (int)a.size();
    vector<int> pref(n + 1, 1), inv(n);
    for (int i = 0; i < n; ++i) {
        assert(a[i] != 0);
        pref[i + 1] = (int)(1LL * pref[i] * a[i] % mod);
    }

    int suffix_inv = Ntt::qpow(pref[n], mod - 2);
    for (int i = n - 1; i >= 0; --i) {
        inv[i] = (int)(1LL * pref[i] * suffix_inv % mod);
        suffix_inv = (int)(1LL * suffix_inv * a[i] % mod);
    }
    return inv;
}

// 已知叶子值  $\text{weight}_i$ ，自底向上构造
//  $F_p = F_L * M_R + F_R * M_L$ 。
// 若  $F_L$ 、 $F_R$  也使用固定长度翻转表示，则翻转乘积仍是普通卷积，
// 所以可以直接与  $\text{product\_tree}$  中的  $\text{rev}(M_L)$ 、 $\text{rev}(M_R)$  相乘，无需来回  $\text{reverse}$ 。
poly interpolate_up(
    int p,
    int l,
    int r,
    const vector<int>& weight,
    const vector<poly>& product_tree
) {
    if (r - l == 1) return {weight[l]};

    int mid = (l + r) >> 1;
    poly left = interpolate_up(p << 1, l, mid, weight, product_tree);
    poly right = interpolate_up(p << 1 | 1, mid, r, weight, product_tree);

    poly a = mul(std::move(left), product_tree[p << 1 | 1]);
    poly b = mul(std::move(right), product_tree[p << 1]);
    int len = r - l;
    a.resize(len);
}

```

```

    b.resize(len);
    for (int i = 0; i < len; ++i) a[i] = Ntt::add(a[i], b[i]);
    return a;
}

// 给定两两不同的点 (xs[i], ys[i]), 返回唯一的次数 < n 的插值多项式。
// 空点集返回零多项式 {0}; 重复横坐标会触发 assert。
//
// 令  $M(x)=\prod_i(x-x_i)$ 。拉格朗日公式可写成
//  $f(x) = \sum_i y_i / M'(x_i) * M(x)/(x-x_i)$ 。
// 先用同一棵乘积树求出所有  $M'(x_i)$ , 得到叶子权值
//  $w_i = y_i / M'(x_i)$ 。
// 对区间分治合并:
//  $F_{[L,r]} = F_L * M_R + F_R * M_L$ ,
// 展开后恰好是该区间所有  $w_i * M_{[L,r]} / (x-x_i)$  的和。根结点即答案。
//
// 复杂度  $O(M(n) \log n)$ , 额外只有一次模逆; 乘积树同时服务于求值和合并。
poly solve(const vector<int>& xs, const vector<int>& ys) {
    assert(xs.size() == ys.size());
    int n = (int)xs.size();
    if (n == 0) return {0};
    assert(n < mod);

    multipoint_detail::Evaluator evaluator;
    evaluator.init(xs);

    // tree[1]=rev(M), 先翻转回普通系数顺序再求导。
    poly product = evaluator.tree[1];
    reverse(product.begin(), product.end());
    poly derivative_values_poly = derivative(product);
    vector<int> derivative_values = evaluator.evaluate(derivative_values_poly);
    vector<int> inv_derivative = batch_inverse_nonzero(derivative_values);

    vector<int> weight(n);
    for (int i = 0; i < n; ++i) {
        int y = ys[i] % mod;
        if (y < 0) y += mod;
        weight[i] = (int)(1LL * y * inv_derivative[i] % mod);
    }

    // interpolate_up 返回固定长度 n 的翻转系数, 最后翻转回升幂顺序。
    poly result = interpolate_up(1, 0, n, weight, evaluator.tree);
    reverse(result.begin(), result.end());
    normalize(result);
    return result;
}

} // namespace interpolation_detail

// 给定两两不同的点 (xs[i], ys[i]), 返回唯一的次数 < n 的插值多项式。
poly fast_interpolate(const vector<int>& xs, const vector<int>& ys) {
    return interpolation_detail::solve(xs, ys);
}

```



```

// =====
// 9. Bostan-Mori
// =====

// 求  $[x^k] P(x)/Q(x)$ , 要求  $Q(0) \neq 0$ 。
//
// 乘  $Q(-x)$  后,  $Q(x)Q(-x)$  只含偶次项。若  $k$  为偶数, 分子只保留偶次项;
// 若  $k$  为奇数, 只保留奇次项。随后令  $x^2 \rightarrow x$ 、 $k = \text{floor}(k/2)$ , 问题规模减半。
// 重复  $O(\log k)$  轮后只需求  $P(0)/Q(0)$ 。
int bostan_mori(poly p, poly q, long long k) {
    assert(!q.empty() && q[0] != 0);
    if (p.empty()) return 0;

    while (k > 0) {
        poly q_neg = q;
        for (int i = 1; i < (int)q_neg.size(); i += 2)
            if (q_neg[i]) q_neg[i] = mod - q_neg[i];

        poly pq = mul(p, q_neg);
        poly qq = mul(q, q_neg);

        poly next_p, next_q;
        next_p.reserve((pq.size() + 1) / 2);
        next_q.reserve((qq.size() + 1) / 2);

        for (int i = (int)(k & 1); i < (int)pq.size(); i += 2)
            next_p.push_back(pq[i]);
        for (int i = 0; i < (int)qq.size(); i += 2)
            next_q.push_back(qq[i]);

        p.swap(next_p);
        q.swap(next_q);
        k >>= 1;
    }

    int p0 = p.empty() ? 0 : p[0];
    return (int)(1LL * p0 * Ntt::qpow(q[0], mod - 2) % mod);
}

// =====
// 10. 多项式复合
// =====

namespace composition_detail {

// 复合的核心恒等式:
//  $f(g(x)) = [y^n] f_{\text{rev}}(y) / (1 - y * g(x))$ ,
// 其中  $f_{\text{rev}}(y) = \sum_{i=0}^n f_i * y^{n-i}$  (实现通过转置过程隐式放入  $f_i$ )。
// 因而复合可以转化为一个关于  $x, y$  的二维 Bostan-Mori 问题: 每轮把分母
//  $V(x, y)$  乘  $V(-x, y)$ , 消掉  $x$  的奇次项, 再令  $x^2 \rightarrow x$ , 使  $x$  次数减半。
//
// 直接正向维护分子仍然偏慢。这里先只跑分母并保存每轮 kernel, 最后从
// 目标线性泛函反向应用每步的转置 (transposed multiplication)。这样总复杂度

```

```

// 由二维卷积主导；二维卷积再通过 Kronecker 代换压成一次普通 NTT。

// 二元多项式：dx / dy 永远表示“最高次数”，不是 size。
//  $P(i, j) = [x^i y^j] P(x, y)$ 。
struct BiPoly {
    int dx = -1, dy = -1;
    // 按 x 主序连续存储：(i, j) 位于 coef[i*(dy+1)+j]。
    // 相比 vector<vector<int>>，这会把  $O(dx)$  次小块堆分配降为一次，
    // 也让 Kronecker 打包和奇偶抽取保持顺序访存。
    vector<int> coef;

    BiPoly(int _dx = -1, int _dy = -1)
        : dx(_dx), dy(_dy), coef((_dx + 1) * (_dy + 1)) {}

    int& operator()(int i, int j) { return coef[i * (dy + 1) + j]; }
    const int& operator()(int i, int j) const { return coef[i * (dy + 1) + j]; }

    poly row(int i) const {
        auto first = coef.begin() + i * (dy + 1);
        return poly(first, first + dy + 1);
    }

    void swap(BiPoly& other) noexcept {
        std::swap(dx, other.dx);
        std::swap(dy, other.dy);
        coef.swap(other.coef);
    }
};

// Kronecker 代换： $x^i y^j \rightarrow z^{(i * stride + j)}$ 。
// stride 取结果 y 次数上界 dy+1，保证不同的 (i, j) 不会映射到同一 z 次数。
BiPoly multiply(const BiPoly& left, const BiPoly& right) {
    int dx = left.dx + right.dx;
    int dy = left.dy + right.dy;
    int stride = dy + 1;

    poly a(left.dx * stride + left.dy + 1);
    poly b(right.dx * stride + right.dy + 1);

    for (int i = 0; i <= left.dx; ++i)
        for (int j = 0; j <= left.dy; ++j)
            a[i * stride + j] = left(i, j);

    for (int i = 0; i <= right.dx; ++i)
        for (int j = 0; j <= right.dy; ++j)
            b[i * stride + j] = right(i, j);

    // a、b 到这里都是最后一次使用，转移给 mul。
    poly c = mul(std::move(a), std::move(b));

    // c 的长度恰为 (dx+1)*(dy+1)，且 Kronecker 的 stride 就是 dy+1，
    // 所以它已经是 BiPoly 所需的连续行主序，无需再分配并逐项复制。
    BiPoly result;

```

```

    result.dx = dx;
    result.dy = dy;
    result.coef = std::move(c);
    return result;
}

BiPoly negate_x(BiPoly value) {
    for (int i = 1; i <= value.dx; i += 2)
        for (int j = 0; j <= value.dy; ++j)
            if (value(i, j)) value(i, j) = mod - value(i, j);
    return value;
}

// 正向 parity_s: 取  $x^{2i+s}$  并把指数除 2。
// 转置就是把系数重新塞回  $2i+s$  位置。
BiPoly inject_parity(const BiPoly& value, int old_dx, int parity) {
    BiPoly result(old_dx, value.dy);
    for (int i = 0; i <= value.dx; ++i) {
        int p = 2 * i + parity;
        if (p > old_dx) break;
        for (int j = 0; j <= value.dy; ++j)
            result(p, j) = value(i, j);
    }
    return result;
}

// “乘固定  $W$ ”的转置: 与  $reverse(W)$  卷积, 再取中间块。
// 直接把  $reverse(W)$  打包进一维数组, 并从卷积结果提取所需区间, 避免真的
// 构造  $reverse(W)$  和完整  $BiPoly$  卷积结果: 公式仍与朴素转置完全一致。
BiPoly multiply_transpose(
    const BiPoly& value,
    const BiPoly& kernel,
    int old_dx,
    int old_dy
) {
    int result_dy = value.dy + kernel.dy;
    int stride = result_dy + 1;
    poly a(value.dx * stride + value.dy + 1);
    poly b(kernel.dx * stride + kernel.dy + 1);

    for (int i = 0; i <= value.dx; ++i)
        for (int j = 0; j <= value.dy; ++j)
            a[i * stride + j] = value(i, j);

    for (int i = 0; i <= kernel.dx; ++i)
        for (int j = 0; j <= kernel.dy; ++j)
            b[(kernel.dx - i) * stride + kernel.dy - j] = kernel(i, j);

    poly c = mul(std::move(a), std::move(b));
    BiPoly result(old_dx, old_dy);

    for (int i = 0; i <= old_dx; ++i)
        for (int j = 0; j <= old_dy; ++j)

```

```

        result(i, j) =
            c[(i + kernel.dx) * stride + j + kernel.dy];
    return result;
}

struct Stage {
    int x_deg;           // 本轮 BM 前的 x 次数
    int old_y_deg;       // 本轮分子变换前的 y 次数
    BiPoly kernel;       //  $W = V(-x, y)$ 
};

struct Plan {
    vector<Stage> stages;
    int final_y_deg = 0;
};

// 只正向跑“分母 BM”，保存每轮分子转置需要的 kernel。
Plan build_stages(const poly& g, int n) {
    BiPoly denominator(n, 1);           //  $V_0 = 1 - y g(x)$ 
    denominator(0, 0) = 1;
    for (int i = 0; i <= n; ++i)
        if (g[i]) denominator(i, 1) = mod - g[i];

    Plan plan;
    int x_deg = n;
    int u_y_deg = 0;

    while (x_deg > 0) {
        BiPoly kernel = negate_x(denominator); //  $W = V(-x, y)$ 
        int next_x_deg = x_deg >> 1;
        int next_u_y_deg = min(n, u_y_deg + kernel.dy);

        //  $V_{next} = Even_x(V * V(-x, y))$ 。
        BiPoly product = multiply(denominator, kernel);
        int next_v_y_deg = min(n, denominator.dy + kernel.dy);
        BiPoly next_denominator(next_x_deg, next_v_y_deg);

        for (int i = 0; i <= next_x_deg; ++i)
            for (int j = 0; j <= next_v_y_deg; ++j)
                next_denominator(i, j) = product(2 * i, j);

        //  $W$  到这里已经最后一次使用，可以安全转移到 stage。
        plan.stages.push_back({x_deg, u_y_deg, std::move(kernel)});
        denominator.swap(next_denominator);
        x_deg = next_x_deg;
        u_y_deg = next_u_y_deg;
    }

    plan.final_y_deg = u_y_deg;
    return plan;
}

poly compose_zero_constant(const poly& f, const poly& g) {

```

```

assert(!g.empty() && g[0] == 0);
int n = (int)f.size() - 1;
if (n == 0) return {f[0]};

Plan plan = build_stages(g, n);

// 转置的起点: 最终  $U(0,y)$  的各个  $y$  系数对应  $f_i$ 。
BiPoly adjoint(0, plan.final_y_deg);
for (int i = 0; i <= n; ++i) adjoint(0, i) = f[i];

// 正向:  $U_{next} = \text{parity}(U * W)$ 
// 反向:  $U_{bar} = \text{Mul}_W^T(\text{parity}^T(U_{next\_bar}))$ 
for (int t = (int)plan.stages.size() - 1; t >= 0; --t) {
    const Stage& st = plan.stages[t];
    adjoint = inject_parity(adjoint, st.x_deg, st.x_deg & 1);
    adjoint = multiply_transpose(
        adjoint, st.kernel, st.x_deg, st.old_y_deg);
}

// 初始分子是  $b_0 x^n + b_1 x^{n-1} + \dots + b_n$ , 因此最后反转  $x$  下标。
poly ans(n + 1);
for (int i = 0; i <= n; ++i) ans[i] = adjoint(n - i, 0);
return ans;
}

} // namespace composition_detail

// Taylor shift:  $f(x) \rightarrow f(x+c)$ 。
// 由二项式定理
//  $[x^j]f(x+c) = 1/j! * \sum_{i \geq j} (f_i i!) * c^{i-j}/(i-j)!$ ,
// 反转  $i$  这一维后, 右侧正好是一段卷积。
poly taylor_shift(const poly& f, int c) {
    if (f.empty()) return {};
    int n = (int)f.size() - 1;

    poly fac(n + 1), ifac(n + 1);
    fac[0] = 1;
    for (int i = 1; i <= n; ++i)
        fac[i] = (int)(1LL * fac[i - 1] * i % mod);

    ifac[n] = Ntt::qpow(fac[n], mod - 2);
    for (int i = n; i >= 1; --i)
        ifac[i - 1] = (int)(1LL * ifac[i] * i % mod);

    //  $h_j = 1/j! * \sum_{i \geq j} (f_i i!) * c^{i-j}/(i-j)!$ 。
    // 反转第一列后就是普通卷积。
    poly weighted_coefficients(n + 1), shift_powers(n + 1);
    for (int i = 0; i <= n; ++i)
        weighted_coefficients[n - i] =
            (int)(1LL * f[i] * fac[i] % mod);

    long long pw = 1;
    for (int i = 0; i <= n; ++i) {

```

```

        shift_powers[i] = (int)(pw * ifac[i] % mod);
        pw = pw * c % mod;
    }

    poly convolution = mul(std::move(weighted_coefficients),
                          std::move(shift_powers));

    poly h(n + 1);
    for (int j = 0; j <= n; ++j)
        h[j] = (int)(1LL * convolution[n - j] * ifac[j] % mod);
    return h;
}

// f(g(x)) mod x^{deg(f)+1}.
// g 可以比 f 长或短: 只保留需要的前 deg(f)+1 项。
poly compose(poly f, poly g) {
    if (f.empty()) return {};
    int n = (int)f.size() - 1;
    if (n == 0) return {f[0]};

    g.resize(n + 1);
    if (g[0] != 0) {
        int c = g[0];
        f = taylor_shift(f, c);
        g[0] = 0;
    }
    return composition_detail::compose_zero_constant(f, g);
}

// =====
// 11. 幂投影: [x^k] f(x)^i
// =====

namespace power_projection_detail {

using composition_detail::BiPoly;

// 从 P(x,y) 中抽取 x 次数与 parity 同奇偶的项, 并执行 x^2 -> x。
// x 次数只保留到下一轮目标 next_x_deg, y 次数只保留前 y_limit+1 项。
BiPoly parity_reduce(
    const BiPoly& value,
    int next_x_deg,
    int parity,
    int y_limit
) {
    BiPoly result(next_x_deg, min(y_limit, value.dy));
    for (int i = 0; i <= result.dx; ++i) {
        int source_x = 2 * i + parity;
        if (source_x > value.dx) break;
        for (int j = 0; j <= result.dy; ++j)
            result(i, j) = value(source_x, j);
    }
    return result;
}
}

```

```

// 返回  $c_i = [x^{\text{target}}]f(x)^i$  ( $0 \leq i < \text{count}$ )。
//
// 关于幂次数  $i$  再引入变量  $y$ :
//  $\sum_{i \geq 0} f(x)^i y^i = 1/(1-yf(x)) = U(x,y)/V(x,y)$ 。
// 因此答案就是  $[x^{\text{target}}]U/V$  作为  $y$  的形式幂级数的前  $\text{count}$  项。
//
// 二维 Bostan-Mori 的一轮为
//  $W(x,y) = V(-x,y)$ ,
//  $U \leftarrow \text{parity\_target}(U*W)$ ,
//  $V \leftarrow \text{even}(V*W)$ ,
//  $\text{target} \leftarrow \text{floor}(\text{target}/2)$ 。
// 因为  $V(x,y)V(-x,y)$  的  $x$  奇次项全部抵消, 所以可令  $x^2 \rightarrow x$ 。
// 每轮  $x$  上界减半、 $y$  上界至多翻倍, 二维有效面积始终为  $O(\text{target}*\text{count})$ ;
// BiPoly::multiply 再用 Kronecker 代换把二维卷积压成一次普通 NTT。
poly solve(poly f, int target, int count) {
    assert(target >= 0 && count >= 0);
    if (count == 0) return {};

    f.resize(target + 1);

    BiPoly numerator(0, 0);
    numerator(0, 0) = 1;

    BiPoly denominator(target, min(1, count - 1));
    denominator(0, 0) = 1;
    if (count > 1) {
        for (int i = 0; i <= target; ++i)
            if (f[i]) denominator(i, 1) = mod - f[i];
    }

    int current_target = target;
    while (current_target > 0) {
        BiPoly kernel = composition_detail::negate_x(denominator);
        BiPoly numerator_product =
            composition_detail::multiply(numerator, kernel);
        BiPoly denominator_product =
            composition_detail::multiply(denominator, kernel);

        int parity = current_target & 1;
        int next_target = current_target >> 1;
        numerator = parity_reduce(
            numerator_product, next_target, parity, count - 1);
        denominator = parity_reduce(
            denominator_product, next_target, 0, count - 1);
        current_target = next_target;
    }

    //  $x$  已经被完全消去, 此时所求  $y$  级数就是  $U(0,y)/V(0,y)$ 。
    poly numerator_row = numerator.row(0);
    poly denominator_row = denominator.row(0);
    poly result = mul(std::move(numerator_row),
        fps_inv(denominator_row, count));
    result.resize(count);
}

```

```

    return result;
}

} // namespace power_projection_detail

// 返回  $[x^{\text{target}}]f(x)^i$  ( $0 \leq i < \text{count}$ )。
poly power_projection(const poly& f, int target, int count) {
    return power_projection_detail::solve(f, target, count);
}

// =====
// 12. 多项式复合逆
// =====

namespace compositional_inverse_detail {

// 求  $G(F(x))=x \pmod{x^n}$ 。要求  $F(0)=0$  且  $F'(0) \neq 0$ 。
//
// 先令  $v=F'(0)$ ,  $A=F/v$ , 则  $A'(0)=1$ 。设  $H$  是  $A$  的复合逆。
// 固定  $N=n-1$ , 对  $A$  做幂投影  $c_k=[x^N]A(x)^k$ 。交换拉格朗日反演中的
// 原函数与逆函数可得
//  $c_k = k/N * [x^{N-k}](x/H(x))^N$ 。
// 因而令
//  $S(x)=\sum_{k=1}^N N/k * c_k * x^{N-k}$ ,
// 就有  $S=(x/H)^N \pmod{x^N}$ , 常数项为 1, 故
//  $H(x)=x*S(x)^{-1/N}$ 。
// 最后  $F(G)=x$  等价于  $A(G)=x/v$ , 所以  $G(x)=H(x/v)$ 。
poly solve(const poly& f, int n) {
    assert(n >= 2);
    assert((int)f.size() >= n);
    assert(f[0] == 0 && f[1] != 0);
    assert(n < mod);

    int degree = n - 1;
    int linear = f[1];
    int inv_linear = Ntt::qpow(linear, mod - 2);

    poly normalized = prefix(f, n);
    for (int& coefficient : normalized)
        coefficient = (int)(1LL * coefficient * inv_linear % mod);

    poly projected = power_projection_detail::solve(
        std::move(normalized), degree, degree + 1
    );

    fps_detail::ensure_inv_int(degree);
    poly lagrange_series(degree);
    for (int k = 1; k <= degree; ++k) {
        int index = degree - k;
        lagrange_series[index] = (int)(
            1LL * degree * fps_detail::inv_int[k] % mod * projected[k] % mod);
    }
    assert(lagrange_series[0] == 1);
}

```



```

// S 的常数项为 1，因此任意模意义指数都可定义为  $\exp(\alpha \ln S)$ 。
int exponent = mod - fps_detail::inv_int[degree]; // -1/degree
poly log_series = fps_ln(lagrange_series, degree);
for (int& coefficient : log_series)
    coefficient = (int)(1LL * coefficient * exponent % mod);
poly root = fps_exp(log_series, degree);

poly inverse(n);
long long scale = inv_linear;
for (int i = 1; i < n; ++i) {
    inverse[i] = (int)(root[i - 1] * scale % mod);
    scale = scale * inv_linear % mod;
}
return inverse;
}

} // namespace compositional_inverse_detail

poly compositional_inverse(const poly& f, int n) {
    return compositional_inverse_detail::solve(f, n);
}

// =====
// 13. 兼容旧模板命名（可删除）
// =====

poly poly_mul(poly a, poly b) { return mul(std::move(a), std::move(b)); }
poly poly_inv(const poly& f, int n) { return fps_inv(f, n); }
poly poly_derivative(const poly& f) { return derivative(f); }
poly poly_integral(const poly& f) { return integral(f); }
poly poly_ln(const poly& f, int n) { return fps_ln(f, n); }
poly poly_exp(const poly& f, int n) { return fps_exp(f, n); }
poly poly_pow(const poly& f, const string& k, int n) { return fps_pow(f, k, n); }
poly poly_sqrt(const poly& f, int n) { return fps_sqrt(f, n); }
pair<poly, poly> poly_divmod(poly a, poly b) { return divmod(std::move(a), std::move(b)); }
poly poly_interpolate(const vector<int>& xs, const vector<int>& ys) {
    return fast_interpolate(xs, ys);
}

poly poly_power_projection(const poly& f, int target, int count) {
    return power_projection(f, target, count);
}

poly poly_compositional_inverse(const poly& f, int n) {
    return compositional_inverse(f, n);
}

} // namespace polyops

```

计数公式、反演与生成函数速查

本节集中保存容易在推式子时用到的公式。默认越界组合数为 0，空和为 0，空积为 1。第一类 Stirling 数的正负号严格沿用 18-stirling.md：无符号第一类记 $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ ，带符号第一类记

$$s(n, k) = (-1)^{n-k} \begin{bmatrix} n \\ k \end{bmatrix}.$$

组合数与二项式

基本变形

$$\binom{n}{k} = \binom{n}{n-k}, \quad \binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1},$$

$$k \binom{n}{k} = n \binom{n-1}{k-1}, \quad (n-k) \binom{n}{k} = n \binom{n-1}{k},$$

$$\binom{j}{i} \binom{i}{k} = \binom{j}{k} \binom{j-k}{i-k}.$$

最后一式可理解为：先从 j 个元素中选 i 个、再从中标记 k 个，等价于先选出被标记的 k 个，再从剩余 $j-k$ 个中补 $i-k$ 个。

二项式定理与广义二项式

$$(a+b)^n = \sum_{k=0}^n \binom{n}{k} a^{n-k} b^k.$$

对任意形式参数 α ，定义

$$\binom{\alpha}{n} = \frac{\alpha(\alpha-1)\cdots(\alpha-n+1)}{n!},$$

则形式幂级数意义下

$$(1+x)^\alpha = \sum_{n \geq 0} \binom{\alpha}{n} x^n.$$

特别地，对正整数 r ，

$$\frac{1}{(1-x)^r} = \sum_{n \geq 0} \binom{n+r-1}{r-1} x^n.$$

不要把最后一式的系数误写成 $\binom{r}{n}$ 。

Hockey-stick 与 Vandermonde

$$\sum_{k=r}^n \binom{k}{r} = \binom{n+1}{r+1},$$

$$\sum_{k=0}^m \binom{n+k}{n} = \binom{n+m+1}{n+1},$$

$$\sum_{k=0}^m \binom{n-k}{m-k} = \binom{n+1}{m},$$

$$\sum_k \binom{a}{k} \binom{b}{n-k} = \binom{a+b}{n}.$$

多组形式为

$$\sum_{b_1+\dots+b_t=m} \prod_{i=1}^t \binom{a_i}{b_i} = \binom{a_1+\dots+a_t}{m}.$$

负二项式对应的卷积为

$$\sum_{b_1+\dots+b_t=m} \prod_{i=1}^t \binom{a_i+b_i}{b_i} = \binom{a_1+\dots+a_t+m+t-1}{m}.$$

它们分别来自 $(1+x)^{\sum a_i}$ 与 $(1-x)^{-\sum(a_i+1)}$ 的系数比较。

高频求和

$$\sum_{k=0}^n \binom{n}{k} = 2^n, \quad \sum_{k=0}^n (-1)^k \binom{n}{k} = [n=0],$$

$$\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n},$$

$$\sum_{k=0}^n k \binom{n}{k} = n2^{n-1},$$

$$\sum_{k=0}^n k(k-1) \binom{n}{k} = n(n-1)2^{n-2},$$

$$\sum_{k=0}^n k^2 \binom{n}{k} = n(n+1)2^{n-2}.$$

带 $k, k(k-1), \dots$ 的式子可对 $(1+x)^n$ 求导若干次, 再乘相应的 x 并令 $x=1$ 。

插板、上下界与不相邻选择

- $x_1 + \dots + x_m = n, x_i \geq 0$: 方案数 $\binom{n+m-1}{m-1}$ 。
- $x_i \geq l_i$: 先令 $y_i = x_i - l_i$, 答案为

$$\binom{n - \sum l_i + m - 1}{m-1}.$$

- 若还有上界 $x_i \leq r_i$, 令 $u_i = r_i - l_i + 1$, 容斥得到

$$\sum_{S \subseteq [m]} (-1)^{|S|} \binom{n - \sum l_i - \sum_{i \in S} u_i + m - 1}{m-1}.$$

- 从 $1, \dots, n$ 中选 k 个互不相邻的位置：答案 $\binom{n-k+1}{k}$ 。把第 i 个所选位置 p_i 映成 $p_i - (i - 1)$ 即变成普通选择。

容斥原理

对事件集合 $A_1, \dots, A_m$,

$$\left| \bigcup_{i=1}^m A_i \right| = \sum_{\emptyset \neq S \subseteq [m]} (-1)^{|S|-1} \left| \bigcap_{i \in S} A_i \right|.$$

若 T_k 表示所有 k 个事件交集大小的总和，则“恰好满足 r 个事件”的对象数为

$$E_r = \sum_{k=r}^m (-1)^{k-r} \binom{k}{r} T_k.$$

这里的“交集大小总和”不是“恰好满足 k 个事件”的对象数：一个实际满足 r 个事件的对象，会在 T_k 中被计入 $\binom{r}{k}$ 次，这正是需要反演的原因。

概率版本中，设 X 为成功事件个数，则

$$A_j = \sum_{|J|=j} \Pr \left(\bigcap_{i \in J} A_i \right) = \mathbb{E} \binom{X}{j}, \quad \Pr(X = r) = \sum_{j=r}^m (-1)^{j-r} \binom{j}{r} A_j.$$

这些等式不要求事件独立。 A_j 称为二项式矩，是第 j 阶下降阶乘矩除以 $j!$ 。推导、阈值版本以及如何先消去无限时间再反演，见 `34-probability-expectation.md` 的“联合成功概率、二项式矩与阈值等待时间”。

常用结果：

- 错排数

$$D_n = n! \sum_{k=0}^n \frac{(-1)^k}{k!}, \quad D_n = (n-1)(D_{n-1} + D_{n-2});$$

- 从 n 个元素到 k 个有标号盒子的满射数

$$\sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n = k! \left\{ \begin{matrix} n \\ k \end{matrix} \right\};$$

- Min-Max 容斥

$$\max(S) = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|-1} \min(T),$$

$$\min(S) = \sum_{\emptyset \neq T \subseteq S} (-1)^{|T|-1} \max(T).$$

Catalan、反射原理与 Ballot

Catalan 数的等价形式为

$$\begin{aligned}C_0 &= 1, & C_n &= \sum_{i=0}^{n-1} C_i C_{n-1-i}, \\C_n &= \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}, \\C_n &= \frac{4n-2}{n+1} C_{n-1}.\end{aligned}$$

它统计合法括号序列、固定入栈顺序的出栈序列、 n 个结点的二叉树、凸 $(n+2)$ 边形三角剖分，以及从 $(0,0)$ 到 (n,n) 不越过对角线的单调路径。

更一般地，从 $(0,0)$ 到 (p,q) ， $p \geq q$ ，始终满足横坐标不少于纵坐标的路径数为

$$\binom{p+q}{q} - \binom{p+q}{q-1} = \frac{p-q+1}{p+1} \binom{p+q}{q}.$$

第二项来自把第一段越过边界的非法前缀关于边界反射。

生成函数满足

$$C(x) = 1 + xC(x)^2, \quad C(x) = \frac{1 - \sqrt{1-4x}}{2x}.$$

Fibonacci 恒等式

取 $F_0 = 0, F_1 = 1$:

$$\sum_{i=0}^n F_i = F_{n+2} - 1, \quad \sum_{i=0}^n F_i^2 = F_n F_{n+1},$$

$$F_{n+1} F_{n-1} - F_n^2 = (-1)^n,$$

$$F_{n+k} = F_{k-1} F_n + F_k F_{n+1}, \quad \gcd(F_m, F_n) = F_{\gcd(m,n)}.$$

其 OGF 为

$$\sum_{n \geq 0} F_n x^n = \frac{x}{1-x-x^2}.$$

Fibonacci 是整除数列： $n \mid m \Rightarrow F_n \mid F_m$ ；反方向在含 $F_1 = F_2 = 1$ 等退化情形时不能直接使用。

Stirling 数与幂的换基

基本换基为

$$x^n = \sum_{k=0}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} x^{\underline{k}},$$

$$x^{\overline{n}} = \sum_{k=0}^n s(n, k) x^k, \quad x^{\overline{n}} = \sum_{k=0}^n \left[\begin{matrix} n \\ k \end{matrix} \right] x^k.$$

因此两种 Stirling 矩阵互逆:

$$\sum_{k=m}^n s(n, k) \left\{ \begin{matrix} k \\ m \end{matrix} \right\} = \sum_{k=m}^n \left\{ \begin{matrix} n \\ k \end{matrix} \right\} s(k, m) = \delta_{nm}.$$

第二类的显式公式为

$$\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^n.$$

它立刻给出满射数, 并能把普通幂换成下降幂。一个很实用的幂和公式是

$$\sum_{i=0}^N i^p = \sum_{k=0}^p \left\{ \begin{matrix} p \\ k \end{matrix} \right\} k! \binom{N+1}{k+1},$$

因为 $i^{\underline{k}} = k! \binom{i}{k}$, 再使用 Hockey-stick。

行和为

$$\sum_k \left[\begin{matrix} n \\ k \end{matrix} \right] = n!, \quad \sum_k \left\{ \begin{matrix} n \\ k \end{matrix} \right\} = B_n.$$

Bell 数还满足

$$B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k, \quad \sum_{n \geq 0} B_n \frac{x^n}{n!} = \exp(e^x - 1).$$

整行、固定列的快速算法见 `18-stirling.md`。

常见反演

二项式反演

$$g_n = \sum_{k=0}^n \binom{n}{k} f_k \iff f_n = \sum_{k=0}^n (-1)^{n-k} \binom{n}{k} g_k.$$

上三角形式为

$$g_k = \sum_{i=k}^n \binom{i}{k} f_i \iff f_k = \sum_{i=k}^n (-1)^{i-k} \binom{i}{k} g_i.$$

Möbius 反演

$$g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d)g(n/d).$$

判断互质时常插入

$$[\gcd(a, b) = 1] = \sum_{d|\gcd(a, b)} \mu(d),$$

从而

$$\#\{1 \leq a \leq A, 1 \leq b \leq B : \gcd(a, b) = 1\} = \sum_{d=1}^{\min(A, B)} \mu(d) \left\lfloor \frac{A}{d} \right\rfloor \left\lfloor \frac{B}{d} \right\rfloor.$$

子集 Zeta/Möbius

$$G(S) = \sum_{T \subseteq S} F(T) \iff F(S) = \sum_{T \subseteq S} (-1)^{|S|-|T|} G(T).$$

两边均可按位做蝶形，在 $O(d2^d)$ 内计算。超集版本把 $T \subseteq S$ 全部反向即可。

单位根过滤

若域中存在可逆的 m 次单位根 ω ，则

$$[n \equiv r \pmod{m}] = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{j(n-r)}.$$

所以可从 $A(x)$ 中抽取下标模 m 等于 r 的项：

$$\sum_{n \equiv r \pmod{m}} a_n x^n = \frac{1}{m} \sum_{j=0}^{m-1} \omega^{-jr} A(\omega^j x).$$

卷积与变换该选哪一种

目标	变换方式
$h_k = \sum_{i+j=k} f_i g_j$	普通卷积；NTT/FFT
$h_k = \sum_{i+j \equiv k \pmod{N}} f_i g_j$	长度 N 的循环卷积
$h_k = \sum_{i \oplus j = k} f_i g_j$	XOR FWT
$h(S) = \sum_{A \cup B = S} f(A)g(B)$	子集卷积或 OR FWT，取决于是否要求不交
$h_k = \sum_{\gcd(i, j) = k} f_i g_j$	倍数 Zeta，逐点乘，再做倍数 Möbius
$h_k = \sum_{\text{lcm}(i, j) = k} f_i g_j$	约数 Zeta，逐点乘，再做约数 Möbius

GCD/LCM 的方向最容易记反。原因是

$$d \mid \gcd(i, j) \iff d \mid i \wedge d \mid j,$$

所以 GCD 卷积应累计所有下标为 d 的倍数；而

$$\text{lcm}(i, j) \mid n \iff i \mid n \wedge j \mid n,$$

所以 LCM 卷积应累计 n 的所有约数。对应实现见 **19-sieve-convolutions.md**, XOR/OR/AND FWT 见 **10-快速沃尔什变换.md**。

常见对象的直接计数

排列、多重集与圆排列

- n 个不同元素的排列数： $n!$ ；从中选 k 个并排列： $n^{\underline{k}} = n!/(n-k)!$ 。
- 含有 $c_1, \dots, c_m$ 个相同元素的多重集排列数：

$$\frac{(c_1 + \dots + c_m)!}{c_1! \dots c_m!}.$$

- n 个不同元素只把旋转视为相同的圆排列数： $(n-1)!$ 。若翻转也视为相同，还需检查反射是否存在不动排列，不能无条件再除以 2。
- 恰有 k 个置换环： $\left[\begin{smallmatrix} n \\ k \end{smallmatrix} \right]$ ；恰有 k 个下降位置使用 Eulerian 数

$$\left\langle \begin{smallmatrix} n \\ k \end{smallmatrix} \right\rangle = (k+1) \left\langle \begin{smallmatrix} n-1 \\ k \end{smallmatrix} \right\rangle + (n-k) \left\langle \begin{smallmatrix} n-1 \\ k-1 \end{smallmatrix} \right\rangle.$$

标号树与 Prüfer 序列

Cayley 公式： n 个有标号顶点的无根树共有

$$n^{n-2}$$

棵；指定根后共有 n^{n-1} 棵。Prüfer 序列长度为 $n-2$ ，顶点 v 在序列中出现次数恰为 $\deg(v) - 1$ ，所以给定度数 $d_1, \dots, d_n$ 且 $\sum d_i = 2n - 2$ 时，树的数量为

$$\frac{(n-2)!}{\prod_{i=1}^n (d_i - 1)!}.$$

如果题目限制边集而不是只限制度数，应改用 Matrix-Tree 定理，不能套 Cayley。

盒子模型速查

设球数为 n 、盒子数为 k ：

球	盒子	是否允许空盒	方案数
不同	不同	允许	k^n
不同	不同	不允许	$k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$
相同	不同	允许	$\binom{n+k-1}{k-1}$
相同	不同	不允许	$\binom{n-1}{k-1}$
不同	相同	不允许	$\left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$

“相同球放进相同盒子且允许空盒”是至多 k 部分的整数分拆，没有单个组合数闭式。

生成函数操作字典

重复抽样中如何用 EGF 自动统计顺序、指数换元与标记变量的解释见 21-generating-functions.md; 概率生成函数与求矩见 34-probability-expectation.md。

若 $A(x) = \sum_{n \geq 0} a_n x^n$, 则:

操作	系数意义
$x^k A(x)$	下标整体右移 k
$A(x)/(1-x)$	前缀和 $\sum_{i=0}^n a_i$
$(1-x)A(x)$	一阶差分 $a_n - a_{n-1}$
$x A'(x)$	第 n 项乘 n
$A'(x)$	$(n+1)a_{n+1}$
$A(x)B(x)$	普通卷积 $\sum_{k=0}^n a_k b_{n-k}$
$(A(x) + A(-x))/2$	只保留偶数下标
$(A(x) - A(-x))/2$	只保留奇数下标

高频 OGF/EGF:

对象	生成函数
常数序列 1	$1/(1-x)$
$\binom{n+r-1}{r-1}$	$(1-x)^{-r}$
Fibonacci	$x/(1-x-x^2)$
Catalan	$(1-\sqrt{1-4x})/(2x)$
错排 EGF	$e^{-x}/(1-x)$
第二类 Stirling 固定 k 的 EGF	$(e^x - 1)^k/k!$
无符号第一类固定 k 的 EGF	$(-\ln(1-x))^k/k!$
Bell EGF	$\exp(e^x - 1)$
分拆数	$\prod_{j \geq 1} (1-x^j)^{-1}$
互异部分拆	$\prod_{j \geq 1} (1+x^j)$

标号组合类的常用构造规则:

- 有序序列 **SEQ(A)**: $1/(1-A(x))$;
- 无序标号集合 **SET(A)**: $\exp(A(x))$;
- 标号环 **CYC(A)**: $-\ln(1-A(x))$;
- k 个无序非空标号块: $(e^x - 1)^k/k!$;
- 任意个无序非空标号块: $\exp(e^x - 1)$ 。

使用这些规则前必须先确认对象是否有标号、子结构是否有序、空结构是否允许。

对数导数、Euler 变换与 Newton 恒等式

设 $F(x) = \sum_{n \geq 0} f_n x^n$ 且 $f_0 = 1$ 。若

$$\frac{x F'(x)}{F(x)} = \sum_{n \geq 1} c_n x^n,$$

把等式改写成 $x F' = F \sum c_n x^n$ 并比较 x^n 系数, 得到

$$nf_n = \sum_{k=1}^n c_k f_{n-k}.$$

这就是从乘积型生成函数推系数递推的常用入口。特别地，Euler 变换

$$F(x) = \prod_{d \geq 1} (1 - x^d)^{-a_d}$$

满足

$$c_n = \sum_{d|n} d a_d, \quad nf_n = \sum_{k=1}^n c_k f_{n-k}.$$

例如 $a_d = 1$ 时 F 是分拆生成函数， $c_n = \sigma(n)$ 。先按倍数循环求所有 c_n 需 $O(N \log N)$ ，再用递推求 f_n 需 $O(N^2)$ 。

Newton 恒等式是同一技巧的有限变量版本。令 $p_k = \sum_i x_i^k$ ， e_k 为第 k 个基本对称多项式， h_k 为第 k 个完全齐次对称多项式，则

$$ke_k = \sum_{i=1}^k (-1)^{i-1} e_{k-i} p_i, \quad kh_k = \sum_{i=1}^k h_{k-i} p_i,$$

其中 $e_0 = h_0 = 1$ 。第一式可由幂和恢复多项式 $\prod_i (t - x_i)$ 的系数；第二式常用于多重集选择。

这些递推都有“除以 n ”或“除以 k ”。在模意义下只有分母与模数互质时才能直接乘逆元。整数答案中“分子必然整除”不代表只知道分子模 M 后还能除；模数为合数或分母等于模数的倍数时，必须改用无除法递推、保存更高精度的整数，或在足够大的模数上保留额外信息。

没有 NTT 友好模数时

先判断是否真的需要通用卷积：

1. 分母次数为 k 的有理生成函数可直接比较系数，按递推做到 $O(Nk)$ ；只求很远的一项可用 Kitamasa，做到 $O(k^2 \log n)$ 。
2. 乘积因子或非零项很稀疏时，按背包或稀疏递推更新；分拆数的五边形递推就是典型例子，只用加减法。
3. N 不大时直接做 $O(N^2)$ 卷积，适用于任意正模数，也不要求存在逆元。
4. 只有确实需要大规模稠密卷积时，才考虑 FFT 拆系数或多个 NTT 素数加 CRT。多模 NTT 若要先还原整数卷积再对目标模数取模，辅助素数乘积必须大于真实系数的上界；不能只取两三个素数便默认正确。
5. 多项式求逆、对数、指数等除了卷积外还有常数项可逆、 $1..N$ 可逆等前提；“把 NTT 换成朴素乘法”不会自动消除这些代数条件。

所以赛场上遇到非 NTT 模数时，优先寻找低阶递推、稀疏乘积和专用恒等式，而不是立刻实现任意模卷积。

整数分拆与五边形数

分拆数 $p(n)$ 的 OGF 是

$$P(x) = \prod_{k \geq 1} \frac{1}{1 - x^k}.$$

Euler 五边形数定理给出

$$\prod_{k \geq 1} (1 - x^k) = \sum_{t \in \mathbb{Z}} (-1)^t x^{t(3t-1)/2}.$$

因此 $p(0) = 1$, 并有递推

$$p(n) = \sum_{t=1}^{\infty} (-1)^{t-1} \left(p\left(n - \frac{t(3t-1)}{2}\right) + p\left(n - \frac{t(3t+1)}{2}\right) \right),$$

负下标项视为 0。只求前 N 项时该递推复杂度为 $O(N\sqrt{N})$ 。

下面的代码只做加减法, 适用于任意正模数, 包括合数; `result[n]` 是 $p(n) \bmod \text{modulus}$ 。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

vector<int> partition_numbers(int maximum, int modulus) {
    assert(maximum >= 0 && modulus > 0);
    vector<int> result(maximum + 1);
    result[0] = 1 % modulus;
    for (int n = 1; n <= maximum; ++n) {
        long long value = 0;
        for (long long k = 1; ; ++k) {
            long long first = k * (3 * k - 1) / 2;
            if (first > n) break;
            long long second = k * (3 * k + 1) / 2;
            int sign = (k & 1) ? 1 : -1;

            value += sign * result[n - (int)first];
            if (value < 0) value += modulus;
            if (value >= modulus) value -= modulus;
            if (second <= n) {
                value += sign * result[n - (int)second];
                if (value < 0) value += modulus;
                if (value >= modulus) value -= modulus;
            }
        }
        result[n] = (int)value;
    }
    return result;
}
```

有限 q -二项式定理 (这里的 q 是形式变量, 不是模数) 也常用于分拆限制:

$$\prod_{i=0}^{n-1} (1 + yq^i) = \sum_{k=0}^n q^{k(k-1)/2} \begin{bmatrix} n \\ k \end{bmatrix}_q y^k,$$

并有

$$\begin{bmatrix} n \\ k \end{bmatrix}_q = \begin{bmatrix} n-1 \\ k \end{bmatrix}_q + q^{n-k} \begin{bmatrix} n-1 \\ k-1 \end{bmatrix}_q.$$

Gaussian 二项式 $\begin{bmatrix} n \\ k \end{bmatrix}_q$ 的 q^s 系数, 等于恰好装在 $k \times (n-k)$ 矩形内、大小为 s 的分拆数。题目同时限制部分个数和每部分大小时, 这个解释往往比硬推多项式更直接。

分拆中还常用三条双射/生成函数结论:

- 共轭 Ferrers 图说明“至多 k 个部分”和“每部分至多 k ”等势;
- 把每部分都减 1, 说明“恰好 k 个正部分的 n 的分拆数”等于“至多 k 个部分的 $n-k$ 的分拆数”;
- Euler 分拆定理说明“部分互不相同”的分拆数等于“所有部分均为奇数”的分拆数, 因为

$$\prod_{j \geq 1} (1 + x^j) = \prod_{j \geq 1} \frac{1}{1 - x^{2j-1}}.$$

Burnside 与 Pólya

群 G 作用在方案集合 X 上时, 本质不同方案数为

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|.$$

若一个位置置换 g 有 $c(g)$ 个循环, 用 q 种颜色任意染色, 则被 g 固定的染色数为 $q^{c(g)}$, 所以

$$\# \text{本质不同染色} = \frac{1}{|G|} \sum_{g \in G} q^{c(g)}.$$

长度为 n 、只视旋转相同的 q 色项链数为

$$\frac{1}{n} \sum_{r=0}^{n-1} q^{\text{gcd}(n,r)} = \frac{1}{n} \sum_{d|n} \varphi(d) q^{n/d}.$$

若还把翻转视为相同, 则要把二面体群中的反射也加入 Burnside 求和; n 的奇偶会导致反射循环结构不同, 不能直接把旋转答案除以 2。

Pólya 的循环指标为

$$Z_G(s_1, s_2, \dots) = \frac{1}{|G|} \sum_{g \in G} \prod_{j \geq 1} s_j^{c_j(g)},$$

其中 $c_j(g)$ 是 g 中长度为 j 的循环数。若颜色还带权, 把 s_j 替换为所有颜色权值的 j 次幂和, 即可同时记录各颜色使用次数。

推式子的常用检查方法

1. 双计数: 从两种顺序选择同一个带标记结构, 常得到 Vandermonde 或组合数乘积恒等式。
2. 差分: 证明 $F(n) = G(n)$ 时, 比较两边的一阶差分并核对初值; Hockey-stick 尤其适合这样处理。
3. 生成函数比较系数: 卷积、前缀和、递推与换基通常可直接转成乘法或除法。
4. 反射/双射: 格路边界、括号序列和 Catalan 问题优先寻找第一次越界点。
5. 容斥后交换求和: 先固定违反条件的集合, 再按每个对象被统计的次数合并。
6. 小值验符号: 第一类 Stirling、Möbius、五边形数和二项式反演都很容易差一个 (-1) ; 至少检查 $n = 0, 1, 2, 3$ 。

参考

- OI Wiki: 排列组合
 - OI Wiki: Catalan 数
 - OI Wiki: 分拆数与五边形数定理
 - OI Wiki: Pólya 计数
 - OI Wiki: 普通生成函数
-

高斯消元：秩、线性方程组与逆矩阵

浮点版本

使用 Gauss-Jordan 消元。`where[column]` 记录该未知量的主元行；消元结束后先检查形如 $0 = c \neq 0$ 的矛盾行，再由自由变量是否存在区分唯一解与无穷多解。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using ld = long double;

enum class LinearSystemStatus {
    no_solution, // 方程组无解
    unique,      // 方程组有唯一解
    infinite     // 方程组有无穷多组解
};

struct LinearSystemResult {
    LinearSystemStatus status;
    int rank;
    vector<ld> solution;
};

LinearSystemResult solve_linear_system(vector<vector<ld>> matrix,
                                       vector<ld> right_hand_side,
                                       ld epsilon = 1e-12L) {
    int row_count = (int)matrix.size();
    int column_count = row_count == 0 ? 0 : (int)matrix[0].size();
    assert((int)right_hand_side.size() == row_count);
    for (const auto& row : matrix) assert((int)row.size() == column_count);

    vector<int> where(column_count, -1);
    int pivot_row = 0;
    for (int column = 0; column < column_count && pivot_row < row_count;
        ++column) {
        int selected = pivot_row;
        for (int row = pivot_row + 1; row < row_count; ++row) {
            if (fabs1(matrix[row][column]) > fabs1(matrix[selected][column])) {
                selected = row;
            }
        }
        if (fabs1(matrix[selected][column]) <= epsilon) continue;
        swap(matrix[selected], matrix[pivot_row]);
```

```

swap(right_hand_side[selected], right_hand_side[pivot_row]);
where[column] = pivot_row;

ld divisor = matrix[pivot_row][column];
for (int next = column; next < column_count; ++next) {
    matrix[pivot_row][next] /= divisor;
}
right_hand_side[pivot_row] /= divisor;

for (int row = 0; row < row_count; ++row) {
    if (row == pivot_row) continue;
    ld factor = matrix[row][column];
    if (fabsl(factor) <= epsilon) continue;
    for (int next = column; next < column_count; ++next) {
        matrix[row][next] -= factor * matrix[pivot_row][next];
    }
    right_hand_side[row] -= factor * right_hand_side[pivot_row];
}
++pivot_row;
}

for (int row = 0; row < row_count; ++row) {
    bool all_zero = true;
    for (ld value : matrix[row]) {
        if (fabsl(value) > epsilon) all_zero = false;
    }
    if (all_zero && fabsl(right_hand_side[row]) > epsilon) {
        return {LinearSystemStatus::no_solution, pivot_row, {}};
    }
}

vector<ld> solution(column_count);
for (int column = 0; column < column_count; ++column) {
    if (where[column] != -1) {
        solution[column] = right_hand_side[where[column]];
    }
}
LinearSystemStatus status =
    pivot_row == column_count ? LinearSystemStatus::unique
                              : LinearSystemStatus::infinite;
return {status, pivot_row, solution};
}

int matrix_rank(vector<vector<ld>>> matrix, ld epsilon = 1e-12L) {
    vector<ld> zero(matrix.size());
    return solve_linear_system(move(matrix), move(zero), epsilon).rank;
}

optional<vector<vector<ld>>> inverse_matrix(
    vector<vector<ld>>> matrix, ld epsilon = 1e-12L) {
    int n = (int)matrix.size();
    for (const auto& row : matrix) assert((int)row.size() == n);
    vector<vector<ld>>> augmented(n, vector<ld>(2 * n));

```

```

for (int row = 0; row < n; ++row) {
    for (int column = 0; column < n; ++column) {
        augmented[row][column] = matrix[row][column];
    }
    augmented[row][n + row] = 1;
}

for (int column = 0; column < n; ++column) {
    int selected = column;
    for (int row = column + 1; row < n; ++row) {
        if (fabsl(augmented[row][column]) >
            fabsl(augmented[selected][column])) selected = row;
    }
    if (fabsl(augmented[selected][column]) <= epsilon) return nullopt;
    swap(augmented[selected], augmented[column]);
    ld divisor = augmented[column][column];
    for (ld& value : augmented[column]) value /= divisor;
    for (int row = 0; row < n; ++row) {
        if (row == column) continue;
        ld factor = augmented[row][column];
        for (int next = 0; next < 2 * n; ++next) {
            augmented[row][next] -= factor * augmented[column][next];
        }
    }
}

vector<vector<ld>> inverse(n, vector<ld>(n));
for (int row = 0; row < n; ++row) {
    copy(augmented[row].begin() + n, augmented[row].end(),
        inverse[row].begin());
}
return inverse;
}

```

复杂度 $O(rc \min(r, c))$ ，方阵时为 $O(n^3)$ 。固定绝对误差不适合数量级差异极大的矩阵；应先缩放数据或使用相对误差。整数/模意义问题不要转成浮点。

模质数版本

模意义下没有误差，但每个非零主元必须可逆，因此这里要求模数为质数。返回的无穷多解中，自由变量统一取 0。

```

struct ModularLinearSystemResult {
    LinearSystemStatus status;
    int rank;
    vector<int> solution;
};

int gaussian_power_mod(long long base, long long exponent, int prime) {
    long long result = 1;
    while (exponent > 0) {
        if (exponent & 1) result = result * base % prime;
        base = base * base % prime;
        exponent >>= 1;
    }
}

```

```

    }
    return (int)result;
}

ModularLinearSystemResult solve_linear_system_mod(
    vector<vector<int>> matrix, vector<int> right_hand_side, int prime) {
    int row_count = (int)matrix.size();
    int column_count = row_count == 0 ? 0 : (int)matrix[0].size();
    assert(prime >= 2 && (int)right_hand_side.size() == row_count);
    for (auto& row : matrix) {
        assert((int)row.size() == column_count);
        for (int& value : row) value = (value % prime + prime) % prime;
    }
    for (int& value : right_hand_side) value = (value % prime + prime) % prime;

    vector<int> where(column_count, -1);
    int pivot_row = 0;
    for (int column = 0; column < column_count && pivot_row < row_count;
        ++column) {
        int selected = pivot_row;
        while (selected < row_count && matrix[selected][column] == 0) ++selected;
        if (selected == row_count) continue;
        swap(matrix[selected], matrix[pivot_row]);
        swap(right_hand_side[selected], right_hand_side[pivot_row]);
        where[column] = pivot_row;

        int inverse = gaussian_power_mod(matrix[pivot_row][column], prime - 2,
            prime);
        for (int next = column; next < column_count; ++next) {
            matrix[pivot_row][next] = (int)(
                (long long)matrix[pivot_row][next] * inverse % prime);
        }
        right_hand_side[pivot_row] = (int)(
            (long long)right_hand_side[pivot_row] * inverse % prime);

        for (int row = 0; row < row_count; ++row) {
            if (row == pivot_row || matrix[row][column] == 0) continue;
            int factor = matrix[row][column];
            for (int next = column; next < column_count; ++next) {
                matrix[row][next] = (int)(
                    (matrix[row][next] -
                        (long long)factor * matrix[pivot_row][next]) % prime);
                if (matrix[row][next] < 0) matrix[row][next] += prime;
            }
            right_hand_side[row] = (int)(
                (right_hand_side[row] -
                    (long long)factor * right_hand_side[pivot_row]) % prime);
            if (right_hand_side[row] < 0) right_hand_side[row] += prime;
        }
        ++pivot_row;
    }

    for (int row = pivot_row; row < row_count; ++row) {

```



```

        bool all_zero = all_of(matrix[row].begin(), matrix[row].end(),
                                [](int value) { return value == 0; });
        if (all_zero && right_hand_side[row] != 0) {
            return {LinearSystemStatus::no_solution, pivot_row, {}};
        }
    }
    vector<int> solution(column_count);
    for (int column = 0; column < column_count; ++column) {
        if (where[column] != -1) solution[column] = right_hand_side[where[column]];
    }
    return {pivot_row == column_count ? LinearSystemStatus::unique
          : LinearSystemStatus::infinite,
            pivot_row, solution};
}

optional<vector<vector<int>>> inverse_matrix_mod(vector<vector<int>> matrix,
                                                int prime) {

    int n = (int)matrix.size();
    for (const auto& row : matrix) assert((int)row.size() == n);
    assert(prime >= 2);
    vector<vector<int>> augmented(n, vector<int>(2 * n));
    for (int row = 0; row < n; ++row) {
        for (int column = 0; column < n; ++column) {
            augmented[row][column] =
                (matrix[row][column] % prime + prime) % prime;
        }
        augmented[row][n + row] = 1;
    }

    for (int column = 0; column < n; ++column) {
        int selected = column;
        while (selected < n && augmented[selected][column] == 0) ++selected;
        if (selected == n) return nullopt;
        swap(augmented[selected], augmented[column]);

        int inverse_pivot =
            gaussian_power_mod(augmented[column][column], prime - 2, prime);
        for (int next = 0; next < 2 * n; ++next) {
            augmented[column][next] = (int)(
                (long long)augmented[column][next] * inverse_pivot % prime);
        }
        for (int row = 0; row < n; ++row) {
            if (row == column || augmented[row][column] == 0) continue;
            int factor = augmented[row][column];
            for (int next = 0; next < 2 * n; ++next) {
                augmented[row][next] = (int)(
                    augmented[row][next] -
                    (long long)factor * augmented[column][next]) % prime;
                if (augmented[row][next] < 0) augmented[row][next] += prime;
            }
        }
    }
}

```

```

vector<vector<int>> inverse(n, vector<int>(n));
for (int row = 0; row < n; ++row) {
    copy(augmented[row].begin() + n, augmented[row].end(),
        inverse[row].begin());
}
return inverse;
}

```

逆矩阵把单位矩阵直接拼在右侧，一次 Gauss-Jordan 消元即可完成，复杂度 $O(n^3)$ 。若模数不是质数，不能用费马小定理求逆；即使主元非零，也可能不是单位元，此模板不适用。

异或线性基

把无符号 64 位整数看成 $\mathbb{F}_2^{64}$ 向量。basis[bit] 保存最高位为 bit 的基向量；插入时从高位向低位消元。

```

#include <bits/stdc++.h>
using namespace std;

using u64 = unsigned long long;

class XorLinearBasis {
    array<u64, 64> basis{};
    int basis_rank = 0;

public:
    bool insert(u64 value) {
        for (int bit = 63; bit >= 0; --bit) {
            if ((value >> bit & 1ULL) == 0) continue;
            if (basis[bit] == 0) {
                basis[bit] = value;
                ++basis_rank;
                return true;
            }
            value ^= basis[bit];
        }
        return false;
    }

    bool contains(u64 value) const {
        for (int bit = 63; bit >= 0; --bit) {
            if ((value >> bit & 1ULL) == 0) continue;
            if (basis[bit] == 0) return false;
            value ^= basis[bit];
        }
        return true;
    }

    u64 maximum_xor(u64 seed = 0) const {
        u64 result = seed;
        for (int bit = 63; bit >= 0; --bit) {
            result = max(result, result ^ basis[bit]);
        }
    }
}

```

```

        return result;
    }

    void merge(const XorLinearBasis& other) {
        for (u64 value : other.basis) {
            if (value != 0) insert(value);
        }
    }

    int rank() const { return basis_rank; }
    u64 distinct_xor_count() const {
        return basis_rank == 64 ? 0 : (1ULL << basis_rank);
    }
};

```

插入、判定和最大异或都是 $O(64)$ 。当秩为 64 时不同异或值有 2^{64} 个，无法装入 u64，因此 `distinct_xor_count()` 用 0 表示溢出；若这个约定不合题意，应改用 u128 或单独返回秩。

哪些输入元素是必须基

这里的“基”指从给定的带编号元素中选出的极大线性无关子集；即使两个数值相同，它们也是两个不同元素。第 i 个元素属于每一组基，当且仅当

$$\text{rank}(S \setminus \{i\}) < \text{rank}(S),$$

等价地说，`value[i]` 不能被其余元素异或表示。这就是向量拟阵中的 `coloop`，也就是属于每一组基的元素。若只询问一个位置，建立“除它以外”的线性基，再调用 `contains(value[i])` 即可，复杂度为 $O(64n)$ 。

若要一次求出所有位置，可以先贪心得得到一组基 B 。未被选入 B 的元素显然不必须；把每个非基元素表示成 B 的异或组合，表示中出现的基元素都可以被它替换，也不必须。其余基元素才是必须基。下面同时维护基向量在当前基中的坐标，总复杂度为 $O(64n)$ 。

```

vector<bool> mandatory_basis_elements(const vector<u64>& values) {
    int n = (int)values.size();
    array<u64, 64> pivot_value{};
    array<u64, 64> pivot_coordinate{};
    vector<int> selected_index;
    vector<char> selected(n, false);

    for (int index = 0; index < n; ++index) {
        u64 value = values[index];
        u64 coordinate = 0;
        for (int bit = 63; bit >= 0; --bit) {
            if (((value >> bit) & 1ULL) == 0) continue;
            if (pivot_value[bit] != 0) {
                value ^= pivot_value[bit];
                coordinate ^= pivot_coordinate[bit];
                continue;
            }
            int id = (int)selected_index.size();
            pivot_value[bit] = value;
            pivot_coordinate[bit] = coordinate ^ (1ULL << id);
            selected_index.push_back(index);
            selected[index] = true;
        }
    }
}

```

```

        break;
    }
}

vector<char> replaceable(selected_index.size(), false);
for (int index = 0; index < n; ++index) {
    if (selected[index]) continue;
    u64 value = values[index];
    u64 coordinate = 0;
    for (int bit = 63; bit >= 0; --bit) {
        if (((value >> bit) & 1ULL) == 0) continue;
        assert(pivot_value[bit] != 0);
        value ^= pivot_value[bit];
        coordinate ^= pivot_coordinate[bit];
    }
    assert(value == 0);
    for (int id = 0; id < (int)selected_index.size(); ++id) {
        if ((coordinate >> id) & 1ULL) replaceable[id] = true;
    }
}

vector<bool> mandatory(n, false);
for (int id = 0; id < (int)selected_index.size(); ++id) {
    if (!replaceable[id]) mandatory[selected_index[id]] = true;
}
return mandatory;
}

```

零向量永远不是必须基；若同一个非零数出现至少两次，这两个位置也都不是必须基。不要把“被某一种消元顺序选进了基”误认为“必须出现在所有基中”。

线性同余方程组与 exCRT

目标是求任意方程组

$$a_i x \equiv b_i \pmod{m_i}, \quad m_i > 0.$$

每条方程不能直接当成 $x \equiv b_i \pmod{m_i}$ 。必须先令 $g = \gcd(a_i, m_i)$ ：若 $g \nmid b_i$ ，则整组无解；否则约去 g ，把它化为一个标准剩余类，再用 exCRT 逐项合并。

下面的接口返回最小非负解。所有需要长期保存的数据都是 `long long`，乘法、差值和合并过程使用 `__int128` 暂存。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

enum class CRTStatus {
    NoSolution, // 方程组无解

```

```

    Success,      // 最小非负解不超过调用者给定上界
    TooLarge     // 最小解超过上界, 或合并后的模数无法用 i64 保存
};

struct Congruence {
    i64 a;
    i64 b;
    i64 m;
};

struct CRTResult {
    CRTStatus status;
    i64 r;
    i64 mod;
};

struct ExgcdResult {
    i64 gcd;
    i64 x;
    i64 y;
};

// 要求  $a, b \geq 0$ ; 返回  $ax+by=gcd(a,b)$ 。
ExgcdResult exgcd(i64 a, i64 b) {
    assert(a >= 0 && b >= 0 && (a != 0 || b != 0));
    i64 old_remainder = a;
    i64 remainder = b;
    i128 old_x = 1, x = 0;
    i128 old_y = 0, y = 1;

    while (remainder != 0) {
        i64 quotient = old_remainder / remainder;
        i64 next_remainder = old_remainder % remainder;
        i128 next_x = old_x - (i128)quotient * x;
        i128 next_y = old_y - (i128)quotient * y;
        old_remainder = remainder;
        remainder = next_remainder;
        old_x = x;
        x = next_x;
        old_y = y;
        y = next_y;
    }

    assert(numeric_limits<i64>::min() <= old_x &&
           old_x <= numeric_limits<i64>::max());
    assert(numeric_limits<i64>::min() <= old_y &&
           old_y <= numeric_limits<i64>::max());
    return {old_remainder, (i64)old_x, (i64)old_y};
}

// C++ 的负数 % 正数仍可能为负, 统一规范到  $[0, mod)$ 。
i64 normalize_mod(i128 value, i64 mod) {
    assert(mod > 0);

```

```

    value %= mod;
    if (value < 0) value += mod;
    return (i64)value;
}

struct ResidueClass {
    i64 r;
    i64 mod;
};

// 把  $ax=b \pmod m$  化成  $x=r \pmod{\text{mod}}$ 。nullopt 表示无解。
optional<ResidueClass> reduce_linear_congruence(const Congruence& equation) {
    assert(equation.m > 0);
    i64 a = normalize_mod(equation.a, equation.m);
    i64 b = normalize_mod(equation.b, equation.m);
    i64 gcd = std::gcd(a, equation.m);
    if (b % gcd != 0) return nullopt;

    i64 reduced_modulus = equation.m / gcd;
    if (reduced_modulus == 1) return ResidueClass{0, 1};

    i64 reduced_a = a / gcd;
    i64 reduced_b = b / gcd;
    ExgcdResult inverse = exgcd(reduced_a, reduced_modulus);
    assert(inverse.gcd == 1);
    i64 remainder = normalize_mod(
        (i128)reduced_b * inverse.x, reduced_modulus);
    return ResidueClass{remainder, reduced_modulus};
}

// 合并  $x=A \pmod M$  与  $x=B \pmod N$ 。输入必须已经规范化。
CRTResult merge_residue_classes(const ResidueClass& current,
                                const ResidueClass& added) {
    assert(current.mod > 0 && added.mod > 0);
    assert(0 <= current.r && current.r < current.mod);
    assert(0 <= added.r && added.r < added.mod);

    i64 gcd = std::gcd(current.mod, added.mod);
    i128 difference = (i128)added.r - current.r;
    if (difference % gcd != 0) {
        return {CRTStatus::NoSolution, 0, 0};
    }

    // 令  $x=A+M*k$ , 需要解  $(M/g)k=(B-A)/g \pmod{N/g}$ 。
    i64 reduced_added_modulus = added.mod / gcd;
    i64 multiplier = 0;
    if (reduced_added_modulus != 1) {
        i64 reduced_current_modulus = current.mod / gcd;
        ExgcdResult inverse =
            exgcd(reduced_current_modulus, reduced_added_modulus);
        assert(inverse.gcd == 1);
        multiplier = normalize_mod(
            (difference / gcd) * inverse.x, reduced_added_modulus);
    }
}

```

```

    }

    i128 merged_modulus =
        (i128)(current.mod / gcd) * added.mod;
    i128 merged_remainder =
        (i128)current.r + (i128)current.mod * multiplier;
    assert(0 <= merged_remainder && merged_remainder < merged_modulus);

    if (merged_modulus > numeric_limits<i64>::max()) {
        return {CRTStatus::TooLarge, 0, 0};
    }
    return {CRTStatus::Success, (i64)merged_remainder,
            (i64)merged_modulus};
}

CRTResult solve_congruences(const vector<Congruence>& equations, i64 lim) {
    assert(lim >= 0);
    ResidueClass answer{0, 1};

    for (const Congruence& equation : equations) {
        optional<ResidueClass> reduced =
            reduce_linear_congruence(equation);
        if (!reduced.has_value()) {
            return {CRTStatus::NoSolution, 0, 0};
        }

        CRTResult merged = merge_residue_classes(answer, *reduced);
        if (merged.status != CRTStatus::Success) return merged;
        answer = {merged.r, merged.mod};
    }

    if (answer.r > lim) return {CRTStatus::TooLarge, answer.r, answer.mod};
    return {CRTStatus::Success, answer.r, answer.mod};
}

```

为什么这样化简

若 $g = \gcd(a, m)$ ，线性同余 $ax \equiv b \pmod{m}$ 有解当且仅当 $g \mid b$ 。有解时除以 g 得

$$\frac{a}{g}x \equiv \frac{b}{g} \pmod{m/g}.$$

此时 a/g 与 m/g 互质，可以用 `exgcd` 求逆元，最终得到

$$x \equiv \frac{b}{g} \left(\frac{a}{g} \right)^{-1} \pmod{m/g}.$$

a 、 b 先按原模数规范到非负数不会改变方程，也避免了负数除法和 `%` 的符号干扰。特别地，约化后模数为 1 表示该方程对 x 没有任何限制，不应再求逆元。

合并与边界

合并 $x \equiv A \pmod{M}$ 和 $x \equiv B \pmod{N}$ 时，令 $x = A + Mk$ ，得到

$$\frac{M}{g}k \equiv \frac{B-A}{g} \pmod{N/g}, \quad g = \gcd(M, N).$$

所以必须先检查 $g \mid (B-A)$ 。difference 使用 i128, 负数再交给 normalize_mod; 规范化后的 $k \in [0, N/g)$ 保证构造出的新余数天然满足 $0 \leq A' < \text{lcm}(M, N)$ 。

需要区分两种“过大”：

- 新模数仍能保存时，即使 $\text{mod} > \text{lim}$ 也必须继续；最终判断的是最小非负解 r ，例如 $x \equiv 1 \pmod{10^{18}}$ 在 $\text{lim}=1$ 时应当成功。
- 若合并后的 LCM 已超过 long long，按照本模板的接口约定立即返回 TooLarge，不再保存或继续合并真实的大整数。

只要模数仍在 long long 内，不能在中途因为 $r > \text{lim}$ 提前返回：后面还可能出现矛盾约束，最终状态应是 NoSolution。空方程组返回 $x \equiv 0 \pmod{1}$ 。总复杂度为 $O(k \log V)$ ，其中 V 是参与合并的模数规模。

杜教筛

杜教筛适合求积性函数的前缀和，前提是能快速计算它与另一个函数 Dirichlet 卷积后的前缀和。最常用的是

$$\varphi * 1 = \text{id}, \quad \mu * 1 = \varepsilon.$$

令 $S_f(n) = \sum_{i \leq n} f(i)$ ，把卷积按 $\lfloor n/i \rfloor$ 相同的整除块分组，可递归计算 S_φ, S_μ 。

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

class DujiaoSieve {
    int limit;
    vector<int> primes;
    vector<int> least_prime;
    vector<i64> phi;
    vector<int> mobius;
    vector<i128> prefix_phi;
    vector<i64> prefix_mobius;
    unordered_map<i64, i128> phi_cache;
    unordered_map<i64, i64> mobius_cache;

public:
    explicit DujiaoSieve(int sieve_limit)
        : limit(sieve_limit), least_prime(sieve_limit + 1),
          phi(sieve_limit + 1), mobius(sieve_limit + 1),
          prefix_phi(sieve_limit + 1), prefix_mobius(sieve_limit + 1) {
        assert(sieve_limit >= 1);
        phi[1] = 1;
        mobius[1] = 1;
        for (int value = 2; value <= limit; ++value) {
```



```

        if (least_prime[value] == 0) {
            least_prime[value] = value;
            primes.push_back(value);
            phi[value] = value - 1;
            mobius[value] = -1;
        }
        for (int prime : primes) {
            if (prime > limit / value) break;
            int product = value * prime;
            least_prime[product] = prime;
            if (value % prime == 0) {
                phi[product] = phi[value] * prime;
                mobius[product] = 0;
                break;
            }
            phi[product] = phi[value] * (prime - 1);
            mobius[product] = -mobius[value];
        }
    }
    for (int value = 1; value <= limit; ++value) {
        prefix_phi[value] = prefix_phi[value - 1] + phi[value];
        prefix_mobius[value] = prefix_mobius[value - 1] + mobius[value];
    }
}

i128 sum_phi(i64 n) {
    if (n <= limit) return prefix_phi[(int)n];
    if (auto iterator = phi_cache.find(n); iterator != phi_cache.end()) {
        return iterator->second;
    }
    i128 result = (i128)n * (n + 1) / 2;
    for (i64 left = 2, right; left <= n; left = right + 1) {
        i64 quotient = n / left;
        right = n / quotient;
        result -= (i128)(right - left + 1) * sum_phi(quotient);
    }
    phi_cache[n] = result;
    return result;
}

i64 sum_mobius(i64 n) {
    if (n <= limit) return prefix_mobius[(int)n];
    if (auto iterator = mobius_cache.find(n);
        iterator != mobius_cache.end()) return iterator->second;
    i128 result = 1;
    for (i64 left = 2, right; left <= n; left = right + 1) {
        i64 quotient = n / left;
        right = n / quotient;
        result -= (i128)(right - left + 1) * sum_mobius(quotient);
    }
    assert(result >= numeric_limits<i64>::min() &&
           result <= numeric_limits<i64>::max());
    mobius_cache[n] = (i64)result;
}

```

```

        return (i64)result;
    }
};

```

取预处理阈值约 $n^{2/3}$ 时，经典复杂度约为 $O(n^{2/3})$ ，实际也常按内存与询问次数调整。sum_phi 使用 i128，因为结果量级为 n^2 ；sum_mobius 返回 i64，递推中的“块长乘前缀和”仍用 i128 暂存。

Min_25 筛：Euler ϕ 前缀和实例

Min_25 筛分两层：先对所有不同的 $\lfloor n/i \rfloor$ 求素数处的前缀和，再按最小质因子递归枚举质数幂，从素数扩展到整个积性函数。

通用 Min_25 需要题目给出 $f(p)$ 的低次多项式表示以及 $f(p^e)$ 。下面选择 $f = \varphi$ 给出完整实例：

$$\varphi(p) = p - 1, \quad \varphi(p^e) = p^{e-1}(p - 1).$$

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

class Min25PhiSummatory {
    i64 maximum;
    vector<int> primes;
    vector<i64> quotient_values;
    unordered_map<i64, int> quotient_index;
    vector<i64> prime_count_sieve;
    vector<i128> prime_sum_sieve;
    vector<i128> prefix_prime_phi;

    i128 prime_phi_sum(i64 value) const {
        auto iterator = quotient_index.find(value);
        assert(iterator != quotient_index.end());
        int index = iterator->second;
        return prime_sum_sieve[index] - prime_count_sieve[index];
    }

    i128 enumerate(i64 value, int first_prime) const {
        if (value < 2) return 0;

        i128 result = prime_phi_sum(value) - prefix_prime_phi[first_prime];
        if (first_prime >= (int)primes.size() ||
            primes[first_prime] > value) return result;
        for (int index = first_prime; index < (int)primes.size(); ++index) {
            i64 prime = primes[index];
            if (prime > value / prime) break;
            i64 power = prime;
            i128 phi_power = prime - 1;
            while (power <= value / prime) {
                i64 next_power = power * prime;

```

```

        i128 next_phi_power = phi_power * prime;
        result += phi_power * enumerate(value / power, index + 1) +
            next_phi_power;
        power = next_power;
        phi_power = next_phi_power;
    }
}
return result;
}

public:
explicit Min25PhiSummatory(i64 upper_bound) : maximum(upper_bound) {
    assert(upper_bound >= 1);
    int square_root = (int)sqrtl((long double)upper_bound);
    while ((i64)(square_root + 1) <= upper_bound / (square_root + 1)) {
        ++square_root;
    }
    while ((i64)square_root > upper_bound / square_root) --square_root;

    vector<bool> composite(square_root + 1);
    for (int value = 2; value <= square_root; ++value) {
        if (!composite[value]) primes.push_back(value);
        for (int prime : primes) {
            if (prime > square_root / value) break;
            composite[value * prime] = true;
            if (value % prime == 0) break;
        }
    }

    prefix_prime_phi.assign(primes.size() + 1, 0);
    for (int index = 0; index < (int)primes.size(); ++index) {
        prefix_prime_phi[index + 1] =
            prefix_prime_phi[index] + primes[index] - 1;
    }

    for (i64 left = 1, right; left <= maximum; left = right + 1) {
        i64 value = maximum / left;
        right = maximum / value;
        quotient_index[value] = (int)quotient_values.size();
        quotient_values.push_back(value);
        prime_count_sieve.push_back(value - 1);
        prime_sum_sieve.push_back((i128)value * (value + 1) / 2 - 1);
    }

    i128 previous_prime_sum = 0;
    for (int prime_index = 0;
        prime_index < (int)primes.size(); ++prime_index) {
        i64 prime = primes[prime_index];
        i64 square = prime * prime;
        for (int index = 0;
            index < (int)quotient_values.size() &&
            quotient_values[index] >= square; ++index) {
            i64 value = quotient_values[index];

```

```

        int reduced = quotient_index.at(value / prime);
        prime_count_sieve[index] -=
            prime_count_sieve[reduced] - prime_index;
        prime_sum_sieve[index] -=
            (i128)prime *
            (prime_sum_sieve[reduced] - previous_prime_sum);
    }
    previous_prime_sum += prime;
}
}

i128 sum_phi() const {
    return 1 + enumerate(maximum, 0);
}
};

```

`prime_count_sieve` 与 `prime_sum_sieve` 必须分开筛，最后才能相减得到 $\sum_{p \leq x} (p-1)$ ；直接把 $p-1$ 当成完全积性函数做一张筛是错误的。

该实例的典型复杂度约为 $O(n^{3/4}/\log n)$ ，空间 $O(\sqrt{n})$ 。只求 $\sum \varphi$ 时杜教筛通常更短；Min_25 的优势是处理那些素数处容易求和、但没有方便 Dirichlet 卷积递推的积性函数。

怎样改成其他积性函数

没有必要为了“通用”再包一层难以调试的回调类。迁移这份代码只需要替换三处数学量：

1. `prime_phi_sum(x)`：改成 $\sum_{p \leq x} f(p)$ 。若 $f(p)$ 是 p 的低次多项式，就像当前代码同时筛 $\sum 1$ 和 $\sum p$ 一样，分别维护所需的 $\sum p^j$ 后线性组合。
2. `prefix_prime_phi[i]`：改成前 i 个素数的 $f(p)$ 前缀和，递归时用它排除小于 `first_prime` 的素数。
3. `phi_power` 和 `next_phi_power`：分别改成 $f(p^e)$ 、 $f(p^{e+1})$ 。递归式的结构不变。

常见替换如下：

$f(n)$	$f(p)$	$f(p^e)$	素数前缀和需要什么
$\mu(n)$	-1	$e = 1$ 时 -1，否则 0	$-\pi(x)$
$d(n)$	2	$e + 1$	$2\pi(x)$
$\sigma_k(n)$	$1 + p^k$	$1 + p^k + \dots + p^{ek}$	$\pi(x) + \sum_{p \leq x} p^k$
Jordan $J_k(n)$	$p^k - 1$	$p^{k(e-1)}(p^k - 1)$	$\sum_{p \leq x} p^k - \pi(x)$

最容易漏掉的是递归中的单独一项 `next_phi_power`：它表示只选择当前素数幂 p^{e+1} 、后面不再乘其他素数的数。只保留递归乘积项会漏掉所有纯素数幂。若答案取模，筛表、素数前缀和和 $f(p^e)$ 必须从头到尾使用同一个模数；若不取模，则先估计 $f(p^e)$ 和答案是否会超过 `i128`。

洲阁筛（Lucy DP）

洲阁筛直接计算所有质数不超过 n 的数量与和。它只保留 $\lfloor n/i \rfloor$ 的不同取值，把 Eratosthenes 筛中的状态压缩到 $O(\sqrt{n})$ 个。

初始时把 $2, 3, \dots, x$ 都视为候选：数量为 $x-1$ 、总和为 $x(x+1)/2-1$ 。处理质数 p 时，删除由 p 乘上“不小于 p 的剩余候选”产生的合数。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using i64 = long long;
using i128 = __int128_t;

class LucyPrimeSums {
    i64 maximum;
    vector<i64> values;
    unordered_map<i64, int> index_of;
    vector<i64> count_values;
    vector<i128> sum_values;

public:
    explicit LucyPrimeSums(i64 upper_bound) : maximum(upper_bound) {
        assert(upper_bound >= 1);
        for (i64 left = 1, right; left <= upper_bound; left = right + 1) {
            i64 value = upper_bound / left;
            right = upper_bound / value;
            index_of[value] = (int)values.size();
            values.push_back(value);
            count_values.push_back(value - 1);
            sum_values.push_back((i128)value * (value + 1) / 2 - 1);
        }

        for (i64 prime = 2; prime <= maximum / prime; ++prime) {
            if (prime_count_at(prime) == prime_count_at(prime - 1)) continue;
            i64 count_before = prime_count_at(prime - 1);
            i128 sum_before = prime_sum_at(prime - 1);
            i64 square = prime * prime;
            for (int index = 0;
                index < (int)values.size() && values[index] >= square;
                ++index) {
                i64 value = values[index];
                count_values[index] -=
                    prime_count_at(value / prime) - count_before;
                sum_values[index] -=
                    (i128)prime * (prime_sum_at(value / prime) - sum_before);
            }
        }

        i64 prime_count_at(i64 value) const {
            if (value < 2) return 0;
            auto iterator = index_of.find(value);
            assert(iterator != index_of.end());
            return count_values[iterator->second];
        }

        i128 prime_sum_at(i64 value) const {
            if (value < 2) return 0;
            auto iterator = index_of.find(value);

```

```

        assert(iterator != index_of.end());
        return sum_values[iterator->second];
    }

    164 prime_count() const { return prime_count_at(maximum); }
    1128 prime_sum() const { return prime_sum_at(maximum); }
};

```

`prime_count_at(x)` 和 `prime_sum_at(x)` 只允许查询构造过程中出现的 $x = \lfloor n/i \rfloor$ (以及 $x < 2$)；若只关心最终 n ，调用无参数接口即可。

复杂度通常写作约 $O(n^{3/4}/\log n)$ ，空间 $O(\sqrt{n})$ 。洲阁筛只得到素数上的和；Min_25 在它的结果上继续枚举质数幂，才得到一般积性函数的前缀和。

GF(2) 高斯消元

在 $GF(2)$ 上加减法都是异或，非零主元只能是 1，因此没有除法。每一行用 `bitset` 保存：下标 `0..variable_count-1` 是系数，下标 `variable_count` 是右端常数。

```

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

struct XorGaussResult {
    bool consistent;
    int rank;
    vector<int> solution;
};

template<size_t maximum_variables>
XorGaussResult xor_gaussian(
    vector<bitset<maximum_variables + 1>> matrix,
    int variable_count) {
    assert(0 <= variable_count);
    assert(variable_count <= (int)maximum_variables);
    int equation_count = (int)matrix.size();
    vector<int> pivot_row(variable_count, -1);
    int rank = 0;

    for (int column = 0; column < variable_count; ++column) {
        int pivot = rank;
        while (pivot < equation_count && !matrix[pivot][column]) ++pivot;
        if (pivot == equation_count) continue;
        swap(matrix[rank], matrix[pivot]);
        pivot_row[column] = rank;

        for (int row = 0; row < equation_count; ++row) {
            if (row != rank && matrix[row][column]) {
                matrix[row] ^= matrix[rank];
            }
        }
        ++rank;
    }
}

```

```

for (int row = 0; row < equation_count; ++row) {
    bool has_coefficient = false;
    for (int column = 0; column < variable_count; ++column) {
        has_coefficient |= matrix[row][column];
    }
    if (!has_coefficient && matrix[row][variable_count]) {
        return {false, rank, {}};
    }
}

// 令所有自由变量为 0, 得到一组特解。
vector<int> solution(variable_count);
for (int column = 0; column < variable_count; ++column) {
    if (pivot_row[column] != -1) {
        solution[column] = matrix[pivot_row[column]][variable_count];
    }
}
return {true, rank, move(solution)};
}

```

- 无解：出现 $0 = 1$ 的行。
- 唯一解： $\text{rank} == \text{variable_count}$ 。
- 多解：自由变量数为 $\text{variable_count} - \text{rank}$ ，解的数量为 $2^{\text{variable_count} - \text{rank}}$ 。
- 若只求一组解，代码把自由变量全设为 0；若要枚举解，分别指定自由变量，再由主元行回代即可。

复杂度为 $O(nm^2/w)$ 的位运算版本，其中 n 是方程数、 m 是变量数、 w 是机器字长。maximum_variables 是编译期上限，例如 `xor_gaussian<2000>(matrix, variable_count)`。

Sprague-Grundy 定理与 SG 打表

适用条件：游戏是有限的、无环的、双方可用操作完全相同，并采用“无法行动者输”的正常规则。把每个局面看成有向无环图，终止局面没有出边。

定义

$$SG(x) = \text{mex}\{SG(y) \mid x \rightarrow y\},$$

其中 **mex** 是集合中没有出现的最小非负整数，所以终止局面的 SG 值为 0。

Sprague-Grundy 定理：若干个互不影响的子游戏组成和游戏时，整体局面等价于 Nim，且

$$SG(x_1 + x_2 + \cdots + x_k) = SG(x_1) \oplus SG(x_2) \oplus \cdots \oplus SG(x_k).$$

因此当前局面必败当且仅当所有子游戏的 SG 值异或和为 0；否则至少存在一步把异或和变成 0。

打表思路

1. 明确“一个不可再分状态”的参数，以及它能转移到哪些更小状态。
2. 按拓扑序从小到大计算，或使用记忆化 DFS；收集所有后继 SG 值后取 **mex**。
3. 若一次操作把状态拆成多个独立部分，该后继的值不是某一个子状态的 SG，而是这些部分 SG 值的异或。
4. 每个状态最多有 d 个不同后继时，**mex** 不会超过 d ；可用长度 $d + 1$ 的时间戳数组标记，无需 **set**。

5. 打出前若干项后可以猜周期，但周期必须用转移只依赖有限窗口等性质证明；仅凭样例重复不能作为证明。

常见减法游戏中，状态是剩余石子数，直接枚举允许减去的数；区间切割游戏中，一次落子往往产生左右两个区间，后继值应写成 `sg[left] ^ sg[right]`。

以下情况不能直接套本定理：反常规则（最后行动者输）、双方操作集合不同的偏置游戏、存在无法保证终止的环。尤其反常 Nim 应单独处理“所有堆大小均不超过 1”的边界。

数论常用定理速查

本页只收录容易在推式子时临时忘记、但不值得单独写模板的结论。线性同余与 CRT 见 `27-crt.md`，原根见 `05-原根.md`，素性测试和分解见 Miller-Rabin / Pollard-Rho。

Bézout、Euler、Fermat 与 Wilson

- Bézout：整数 a, b 的所有线性组合 $ax + by$ 构成 $\gcd(a, b)$ 的倍数集合；所以 $ax \equiv c \pmod{m}$ 有解当且仅当 $\gcd(a, m) \mid c$ 。
- Euler： $\gcd(a, m) = 1$ 时 $a^{\varphi(m)} \equiv 1 \pmod{m}$ 。素数模下化为 Fermat 小定理 $a^{p-1} \equiv 1 \pmod{p}$ 。
- 指数真正的周期是乘法阶 $\text{ord}_m(a)$ ，它整除 $\varphi(m)$ ；直接把指数对 $\varphi(m)$ 取模之前必须先确认底数与模数互质。
- Wilson： $p > 1$ 为素数当且仅当 $(p-1)! \equiv -1 \pmod{p}$ 。它适合证明和小范围公式，不适合代替素性测试。

Carmichael 函数给出对所有可逆剩余都成立的更小公倍周期。互素分解时

$$\lambda\left(\prod p_i^{e_i}\right) = \text{lcm}_i \lambda(p_i^{e_i}),$$

奇素数幂以及 2, 4 上有 $\lambda(p^e) = \varphi(p^e)$ ； $e \geq 3$ 时 $\lambda(2^e) = 2^{e-2}$ 。

阶乘和组合数中的素因子

Legendre 公式：

$$v_p(n!) = \sum_{k \geq 1} \left\lfloor \frac{n}{p^k} \right\rfloor = \frac{n - s_p(n)}{p-1},$$

其中 $s_p(n)$ 是 n 的 p 进制数位和。因此

$$v_p\binom{n}{k} = v_p(n!) - v_p(k!) - v_p((n-k)!).$$

Kummer 定理给出等价的组合解释： $v_p\binom{n}{k}$ 等于用 p 进制计算 $k + (n-k)$ 时产生的进位次数。判断组合数是否被某个素数幂整除时，通常比真的计算组合数更方便。

LTE（指数提升）

记 $v_p(x)$ 为 x 中素因子 p 的指数。设 p 为奇素数、 $p \mid x-y$ 且 $p \nmid xy$ ，则

$$v_p(x^n - y^n) = v_p(x-y) + v_p(n).$$

若 $p \mid x + y$ 且 n 为奇数, 则

$$v_p(x^n + y^n) = v_p(x + y) + v_p(n).$$

对奇数 x, y , 二进制版本是

$$v_2(x^n - y^n) = \begin{cases} v_2(x - y), & n \text{ 为奇数,} \\ v_2(x - y) + v_2(x + y) + v_2(n) - 1, & n \text{ 为偶数.} \end{cases}$$

LTE 的整除前提不可省略: 不满足时应先因式分解或按乘法阶分析。

二次剩余与二次互反

对奇素数 p 且 $p \nmid a$, Euler 判别给出

$$a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p},$$

右侧 Legendre 符号为 1 或 -1 , 可先判定模平方根是否存在。不同奇素数 p, q 满足二次互反律

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{(p-1)(q-1)/4},$$

并有补充公式

$$\left(\frac{-1}{p}\right) = (-1)^{(p-1)/2}, \quad \left(\frac{2}{p}\right) = (-1)^{(p^2-1)/8}.$$

实际求平方根使用 Tonelli-Shanks; **23-polynomial-advanced.md** 中已有实现。合数模下即使 Jacobi 符号为 1 也不保证是二次剩余。

概率与期望建模

本章以竞赛中的离散随机过程为主。先在实数或有理数意义下证明公式与期望存在, 再考虑取模; 概率大小、收敛性和独立性都不能用模数值判断。

看到什么结构, 先想什么方法

题目结构	优先考虑
总收益、出现种类数、逆序对数	拆成指示变量, 交换求和, 用期望线性性
条件复杂, 但固定第一步或某个变量后简单	条件概率、全期望、首次转移分析
问第一次成功或首次到达的时间	几何分布、尾和公式、吸收状态方程
重复抽样, 限制只依赖每类出现次数	21-generating-functions.md 的 EGF 与指数换元
所有目标都出现才结束	分阶段收集、尾概率容斥、Min-Max 容斥
有自环或转移成环	自环移项; 线性方程或按 SCC 分块求解
要方差, 已有期望	二阶矩方程、协方差、全方差
独立随机变量之和、随机项数之和	概率生成函数的乘法、复合
多个坏事件或随机算法失败概率	Union bound; 独立和的 Chernoff 界

条件概率、全概率与独立性

对于 $\Pr(B) > 0$, $\Pr(A | B) = \Pr(A \cap B) / \Pr(B)$ 。若 B_i 是样本空间的互斥划分, 则

$$\Pr(A) = \sum_i \Pr(B_i) \Pr(A | B_i), \quad \Pr(B_i | A) = \frac{\Pr(B_i) \Pr(A | B_i)}{\sum_j \Pr(B_j) \Pr(A | B_j)}.$$

第二式要求 $\Pr(A) > 0$ 。零概率条件项不必定义, 可以从求和中略去。

互斥表示不能同时发生, 独立表示 $\Pr(A \cap B) = \Pr(A) \Pr(B)$ 。两个正概率的互斥事件不独立; 两两独立也不足以把任意多个事件的交概率写成乘积。每步独立抽样不代表固定步数下各类出现次数独立。

期望线性性: 把答案拆成贡献

对于可积随机变量, 有限和满足

$$\mathbb{E} \left[\sum_i a_i X_i \right] = \sum_i a_i \mathbb{E}[X_i].$$

这里不要求独立。设 I_A 为事件 A 的指示变量, 则 $\mathbb{E}[I_A] = \Pr(A)$ 。求数量的期望时, 优先给每个可能被计入的对象一个指示变量。

例如独立抽样 n 次, 每次选中类别 i 的概率为 p_i , 出现种类数 D 满足

$$\mathbb{E}[D] = \sum_i (1 - (1 - p_i)^n).$$

出现事件之间虽然相关, 但不妨碍把期望相加。类似地, 均匀随机排列的每一对位置构成逆序的概率为 $1/2$, 故逆序对期望为 $n(n-1)/4$ 。

乘法则不同: 独立时才可直接使用 $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$, 一般情况要算联合分布或条件期望。一般也没有 $\mathbb{E}[1/X] = 1/\mathbb{E}[X]$ 或 $\mathbb{E}[X/Y] = \mathbb{E}[X]/\mathbb{E}[Y]$ 。

全期望与全方差

先按 Y 分类, 再求每一类的条件期望:

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X | Y]] = \sum_y \Pr(Y = y) \mathbb{E}[X | Y = y].$$

对二阶矩有限的变量,

$$\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y]).$$

可以理解为: 总波动包含每一类内部的波动, 以及各类均值之间的波动。只平均条件方差会漏掉第二部分。

尾和公式: 等待时间转成“还没结束”

非负整数随机变量 T 满足逐点恒等式 $T = \sum_{n \geq 0} I_{T > n}$, 因此

$$\mathbb{E}[T] = \sum_{n \geq 0} \Pr(T > n) = \sum_{n \geq 1} \Pr(T \geq n),$$

$$\mathbb{E}[T^2] = \sum_{n \geq 0} (2n+1) \Pr(T > n).$$

第二式来自 $t^2 = \sum_{n=0}^{t-1} (2n+1)$ 。对非负变量交换无限和合法，结果也可能为 $+\infty$ ；要输出有限期望仍需证明收敛。非负连续变量有对应公式 $\mathbb{E}[T] = \int_0^\infty \Pr(T > t) dt$ 。

若 $r_n = \Pr(T = n+1)$ ，则应算 $\mathbb{E}[T] = \sum_{n \geq 0} (n+1)r_n$ ；仅在 $\sum_n r_n = 1$ 时可写成 $1 + \sum_n nr_n$ 。不要把首次完成概率和尾概率混用。

高频分布与等待阶段

分布与约定	期望	方差
Bernoulli(p): 取 0, 1	p	$p(1-p)$
Binomial(n, p): n 次独立试验的成功数	np	$np(1-p)$
Geometric(p): 含成功那次的试验数, 取 1, 2, ...	$1/p$	$(1-p)/p^2$
Negative binomial(r, p): 直到第 r 次成功的总试验数	r/p	$r(1-p)/p^2$
Poisson(λ): $\Pr(X = k) = e^{-\lambda} \lambda^k / k!$	λ	λ
Hypergeometric(N, K, n): 从 N 个中不放回取 n 个, 原有 K 个成功项	nK/N	$n(K/N)(1-K/N)(N-n)/(N-1)$

几何与负二项要求 $p > 0$ ，各次试验独立且成功率不变。超几何公式要求 $0 \leq K, n \leq N$ 且 $N > 1$ ； $N = 1$ 时合法抽样的成功数确定，方差为零。

几何分布满足 $\Pr(T > n) = (1-p)^n$ ，由尾和直接得到 $1/p$ ；也满足无记忆性 $\Pr(T > a+b \mid T > a) = \Pr(T > b)$ ，条件事件须有正概率。若改为统计成功前的失败数，期望变为 $(1-p)/p$ 。

如果过程能按“取得一次新进展”划为阶段，且第 i 阶段每步成功概率恒为 $q_i > 0$ ，该阶段期望耗时为 $1/q_i$ ，总期望为 $\sum_i 1/q_i$ 。相加本身不要求各阶段独立。若阶段成功率还依赖具体历史，则应保留状态并用条件期望，不能随意把随机成功率换成其平均值。

收集问题与 Min-Max 容斥

每次均匀抽取 m 种物品之一，已有 i 种时取得新种类的概率为 $(m-i)/m$ ，因此集齐全部的期望为

$$m \sum_{j=1}^m \frac{1}{j} = mH_m.$$

更一般地，每次独立抽样，必需类别的概率为 $p_1, \dots, p_m > 0$ ，允许其他无用类别。令 $p(A) = \sum_{i \in A} p_i$ ，容斥给出

$$\Pr(T > n) = \sum_{\emptyset \neq A \subseteq [m]} (-1)^{|A|+1} (1-p(A))^n,$$

$$\mathbb{E}[T] = \sum_{\emptyset \neq A \subseteq [m]} \frac{(-1)^{|A|+1}}{p(A)}.$$

解释：对于子集 A ，“其中一种首次出现”的等待时间是参数为 $p(A)$ 的几何分布；用 Min-Max 容斥把“全部出现”的最大等待时间转成这些最小等待时间。各类别的首次出现时间无需独立。容斥恒等式见 24-counting-formulas.md。

通过子集最低位递推预处理 $p(A)$ ，时间与空间均为 $O(2^m)$ ；不是大 m 的通用解。某个必需类别概率为零时不可能集齐。注意离散抽样中“任一类命中”的概率是互斥类别概率之和，不能换成独立事件的并概率。

联合成功概率、二项式矩与阈值等待时间

容斥的另一层：从“指定若干个成功”恢复“总共成功几个”

设 $I_1, \dots, I_C$ 是任意事件的指示变量， $X = \sum_i I_i$ 为成功个数。定义

$$A_j = \sum_{\substack{J \subseteq [C] \\ |J|=j}} \Pr(I_i = 1 \text{ 对所有 } i \in J), \quad A_0 = 1.$$

一个样本若实际成功了 r 个事件，它恰好出现在 $\binom{C}{j}$ 个大小为 j 的成功子集中。因此

$$A_j = \mathbb{E} \binom{X}{j} = \sum_{r=j}^C \binom{r}{j} \Pr(X = r).$$

A_j 统计“选出 j 个成功事件的方式数”的期望，并不是恰好成功 j 个的概率。它称为二项式矩； $j!A_j = \mathbb{E}[X(X-1)\cdots(X-j+1)]$ 才是通常不除以阶乘的阶乘矩。

写成生成函数，令 $G_X(u) = \mathbb{E}[u^X]$ ，则

$$\sum_{j=0}^C A_j v^j = \mathbb{E}[(1+v)^X] = G_X(1+v).$$

把 v 换成 $u-1$ 再取 $[u^r]$ ，直接得到二项式反演：

$$\Pr(X = r) = \sum_{j=r}^C (-1)^{j-r} \binom{j}{r} A_j.$$

这里没有用到独立性。对于 $1 \leq p \leq C$ ，把 $r = p, \dots, C$ 的概率相加，并使用交错二项式和，可直接得到阈值版本

$$\Pr(X \geq p) = \sum_{j=p}^C (-1)^{j-p} \binom{j-1}{p-1} A_j.$$

其中系数来自 $\sum_{r=p}^j (-1)^{j-r} \binom{j}{r} = (-1)^{j-p} \binom{j-1}{p-1}$ 。取 $p = 1$ 就退化为普通的并事件容斥。

两层容斥如何在重复抽样中配合

每步独立均匀抽取长度为 S 的列表中的一个位置。共有 C 个任务，每个任务至少需要一类课程，任务 i 的各类出现次数为正整数 $a_{i,r}$ 。假设所有这些课程类别对应的列表位置两两不交，故 $\sum_{i,r} a_{i,r} \leq S$ ；剩余位置可以是无用课程。

第一层对“缺失的课程类别”容斥。令

$$P_i(x) = \prod_r (1 - x^{a_{i,r}}), \quad \mathcal{L}_t(x^k) = (1 - k/S)^t.$$

则 $\mathcal{L}_t(P_i)$ 是任务 i 在 t 步后完成的概率。这里 x 是形式标记，指数是被禁用的位置总数；线性映射的完整解释见 `21-generating-functions.md`。

第二层对“完成的任务个数”反演。先计算

$$\prod_{i=1}^C (1 + u P_i(x)) = \sum_{j=0}^C f_j(x) u^j, \quad f_j(x) = \sum_{|J|=j} \prod_{i \in J} P_i(x).$$

任务的课程类别不交，保证 $\mathcal{L}_t(\prod_{i \in J} P_i)$ 编码这些任务的联合完成概率。令 X_t 为 t 步后完成任务数，便有

$$\mathcal{L}_t(f_j) = A_j(t) = \mathbb{E} \binom{X_t}{j}.$$

这里是先多项式层面相乘，再映射成联合概率，并未假设任务完成事件独立。一般 $\mathcal{L}_t(P_i P_h) \neq \mathcal{L}_t(P_i) \mathcal{L}_t(P_h)$ 。

f_j 是初等对称多项式，也可当作“恰好选 j 个任务”的 01 背包：初始 $f_0 = 1$ ，其他为零；加入任务 P_i 时，倒序更新 $f_j \leftarrow f_j + P_i f_{j-1}$ 。较大规模用分治乘积和二元卷积，见 `21-generating-functions.md` 的 Kronecker 代换；倒序背包只说明代数关系，不保证达到分治复杂度。

先减去极限，才能对时间求和

所需课程概率均为正，所以所有任务最终都完成， $A_j(t) \rightarrow \binom{C}{j}$ 。因此 $j \geq 1$ 时直接计算 $\sum_t A_j(t)$ 会发散。应先定义趋于零的差值

$$D_j(t) = \binom{C}{j} - A_j(t), \quad h_j = \sum_{t \geq 0} D_j(t).$$

每个 P_i 常数项为 1，所以 $f_j(0) = \binom{C}{j}$ 。若 $f_j(x) = \sum_{k=0}^S c_{j,k} x^k$ ，便有

$$D_j(t) = - \sum_{k=1}^S c_{j,k} (1 - k/S)^t,$$

$$h_j = - \sum_{k=1}^S c_{j,k} \frac{S}{k} = S \int_0^1 \frac{\binom{C}{j} - f_j(x)}{x} dx.$$

这同时解释了实现中的三个细节：只枚举 $k \geq 1$ 是因为已经减去常数项；乘 S/k 是几何级数求和；整体负号来自“极限减当前值”。 $h_0 = 0$ 。

直接还原第 p 个任务完成的期望

完成状态随时间单调, 令 $T_p = \min\{t : X_t \geq p\}$ 。对 $1 \leq p \leq C$, 尾和公式给出 $\mathbb{E}[T_p] = \sum_{t \geq 0} \Pr(X_t < p)$ 。在阈值反演式中把 X 取为恒等于 C 的变量, 得到

$$1 = \sum_{j=p}^C (-1)^{j-p} \binom{j-1}{p-1} \binom{C}{j}.$$

因此可以先相减、再对时间求和, 直接得到

$$\Pr(X_t < p) = \sum_{j=p}^C (-1)^{j-p} \binom{j-1}{p-1} D_j(t),$$

$$\boxed{\mathbb{E}[T_p] = \sum_{j=p}^C (-1)^{j-p} \binom{j-1}{p-1} h_j.}$$

无需逐个时间恢复完整概率分布。给定全部 h_j , 用双重循环可在 $O(C^2)$ 时间内算出全部阈值的期望。

例如 $C = 2$ 时, $A_1 = \Pr(X_t = 1) + 2\Pr(X_t = 2)$, $A_2 = \Pr(X_t = 2)$, 所以 $\mathbb{E}[T_1] = h_1 - h_2$, $\mathbb{E}[T_2] = h_2$ 。若两个任务分别只需一种课程, 概率为 $a/S, b/S$, 则

$$h_1 = \frac{S}{a} + \frac{S}{b}, \quad h_2 = \frac{S}{a} + \frac{S}{b} - \frac{S}{a+b},$$

故 $\mathbb{E}[T_1] = S/(a+b)$, 正好是首次抽中任一种所需课程的几何等待时间。

$p > C$ 时不能达到阈值; $p = 0$ 时等待时间为零。若有必需类别概率为零的任务, 应先去掉这些永远不能完成的任务, 再确定 C 。模质数 M 计算仍须检查 S 与实际使用的 k 可逆, $S < M$ 是足够条件。

首次转移分析: 自环、环与二阶矩

状态必须包含决定后续转移分布的全部信息。对有限状态过程, 设 E_i 为从状态 i 到目标的期望总代价, 目标状态取 $E_i = 0$; 从 i 转移到 j 的概率为 P_{ij} , 支付确定的边代价 c_{ij} , 则

$$E_i = \sum_j P_{ij}(c_{ij} + E_j).$$

每步代价为 1 时, 非目标状态满足 $E_i = 1 + \sum_j P_{ij}E_j$ 。存在自环可以移项:

$$E_i = \frac{1 + \sum_{j \neq i} P_{ij}E_j}{1 - P_{ii}}.$$

这是“失败后回到原状态”的通用公式, 不能在有自环时直接递归计算。DAG 可逆拓扑 DP; 一般图先按正概率边分 SCC, 再从后继 SCC 向前求解。一个大小为 b 的 SCC 可用 $O(b^3)$ 高斯消元, 见 `25-gaussian-elimination.md`。

对全部可达非目标状态构造子转移矩阵 R , 令 $b_i = \sum_j P_{ij}c_{ij}$, 则 $(I - R)E = b$ 。先判断是否会吸收: 有限状态下, 若从起点能以正概率到达不含目标的封闭 SCC, 则存在永不结束的概率, 单位步长下期望无限; 若所有可达状态都能到达目标, 则最终吸收概率为 1, 且期望有限。

命中概率也可列 $h_i = \sum_j P_{ij} h_j$, 目标边界为 1, 无法到达目标的状态为 0。先固定这些边界再解方程, 避免把封闭非目标类留在齐次系统中造成不唯一。

要方差时, 设 V_i 是剩余步数的二阶矩, 目标取 $V_i = 0$, 则

$$V_i = 1 + 2 \sum_j P_{ij} E_j + \sum_j P_{ij} V_j, \quad \text{Var}(T_i) = V_i - E_i^2.$$

先解 E , 再用同一个系数矩阵解 V 。有确定边代价时, 右侧改为 $\sum_j P_{ij}(c_{ij}^2 + 2c_{ij}E_j + V_j)$ 。

协方差与二阶矩

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2, \quad \text{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y],$$

$$\text{Var}\left(\sum_i X_i\right) = \sum_i \text{Var}(X_i) + 2 \sum_{i < j} \text{Cov}(X_i, X_j).$$

两两独立足以使上式协方差项为零; 不独立时不能只加方差。对指示变量 I_i , 有 $I_i^2 = I_i$, 并且 $\mathbb{E}[I_i I_j] = \Pr(A_i \cap A_j)$, 所以计数变量的方差通常只需单事件概率和两事件交概率。

概率生成函数与随机和

非负整数变量 X 的概率生成函数 (PGF) 是

$$G_X(s) = \mathbb{E}[s^X] = \sum_{k \geq 0} \Pr(X = k) s^k.$$

这是概率序列的 OGF, 系数本身就是概率, 不含 $1/k!$ 。相关矩有限时,

$$G_X(1) = 1, \quad G'_X(1) = \mathbb{E}[X], \quad G''_X(1) = \mathbb{E}[X(X-1)],$$

$$\text{Var}(X) = G''_X(1) + G'_X(1) - G'_X(1)^2.$$

独立的 X, Y 满足 $G_{X+Y} = G_X G_Y$ 。若 N 是非负整数变量, X_i 独立同分布且整个序列与 N 独立, 令 $Z = \sum_{i=1}^N X_i$, 则

$$G_Z(s) = G_N(G_X(s)), \quad \mathbb{E}[Z] = \mathbb{E}[N]\mathbb{E}[X],$$

$$\text{Var}(Z) = \mathbb{E}[N] \text{Var}(X) + \text{Var}(N) \mathbb{E}[X]^2.$$

复合来自先固定 $N = n$ 得到 $G_X(s)^n$, 再对 N 求平均。矩公式要求对应矩有限。对于依赖已观察结果的停止时刻, 不能直接使用上述 PGF 复合与方差式。

一个常用扩展是 Wald 等式: 若 X_i 独立同分布且可积, N 是相对于它们自然历史的停止时刻, 且 $\mathbb{E}[N] < \infty$, 则 $\mathbb{E}[\sum_{i=1}^N X_i] = \mathbb{E}[N]\mathbb{E}[X_1]$ 。这里 N 不必与序列独立, 但“是否继续第 i 次”只能依赖此前的信息; 只知道 N 有限或事后挑选终点不够。

泊松化与 EGF 的联系

固定抽样次数 n 时, 各类别次数服从多项分布, 通常相关。若先独立取 $N \sim \text{Poisson}(t)$, 再进行 N 次独立类别抽样, 概率为 p_i 、总和为 1, 则联合概率为

$$\Pr(N_i = k_i \text{ 对所有 } i) = e^{-t} \prod_i \frac{(tp_i)^{k_i}}{k_i!} = \prod_i \left(e^{-tp_i} \frac{(tp_i)^{k_i}}{k_i!} \right).$$

因此 N_i 变为相互独立的 $\text{Poisson}(tp_i)$ 变量。例如全部必需类别都出现的概率变为 $\prod_i (1 - e^{-tp_i})$ 。这是精确恒等式, 并非大样本近似。

如果固定 n 步的事件概率为 a_n , EGF 为 $A(z) = \sum_n a_n z^n / n!$, 泊松化后概率就是 $e^{-t} A(t)$ 。这解释了 EGF 中为什么自然出现指数, 但固定 n 的问题仍要提取系数或另做严格转换, 不能直接把 $t = n$ 当作精确答案。

也可把每次抽样之间插入独立的 $\text{Exp}(1)$ 等待时间。若停止抽样次数 T 只由类别序列决定且期望有限, 连续完成时间 τ 满足 $\mathbb{E}[\tau | T] = T$, 故 $\mathbb{E}[\tau] = \mathbb{E}[T]$; 两者分布和方差一般不同。这个构造与泊松过程的分裂性质相联系, 参见 Harchol-Balter 教材第 12 章。

概率界与概率法

Markov、Chebyshev 和 Union bound 见 `../other/01-basic.md`, 都不要求变量或事件相互独立; Markov 要求非负, Chebyshev 要求方差有限。

若 $X = \sum_i X_i$, 其中 X_i 为相互独立的 Bernoulli 变量, $\mu = \mathbb{E}[X] > 0$, 常用 Chernoff 界为

$$\Pr(X \geq (1 + \delta)\mu) \leq \left(\frac{e^\delta}{(1 + \delta)^{1 + \delta}} \right)^\mu \leq \exp\left(-\frac{\mu\delta^2}{2 + \delta}\right), \quad \delta > 0,$$

$$\Pr(X \leq (1 - \delta)\mu) \leq e^{-\mu\delta^2/2}, \quad 0 < \delta < 1.$$

推导入口是对 e^{tX} 使用 Markov, 再用独立性拆开指数矩; 所以不能仅凭知道期望就套用这组界。参见 MIT Chernoff 讲义。

概率法的两个常见入口: 有限样本空间中, 总有一个方案的收益不小于平均收益; 若坏事件数 B 满足 $\mathbb{E}[B] < 1$, 由 $\Pr(B \geq 1) \leq \mathbb{E}[B] < 1$ 知存在没有坏事件的方案。若能高效计算条件期望, 可逐次固定一个选择, 使条件期望不下降, 从存在性证明构造出至少达到初始期望的方案。

模意义下的概率与期望

有理数 a/b 映射为 $ab^{-1} \bmod M$, 需要约分后的分母与 M 互质。每一步直接做模除法, 还要求当时的分母可逆; 中间不可逆不等价于最终约分后的答案必然没有模意义。

常见问题包括: 实数中成功概率大于零但模 M 后为零; 实数吸收过程的 $I - R$ 可逆, 但模 M 后奇异; 实数期望本来无限却仍机械解出一个模数值。应先证明过程会终止且期望有限, 再检查具体代数分母。

模意义下无限几何级数并不解析收敛。使用 $1/(1 - r)$ 等公式, 是先在 $|r| < 1$ 的实数模型中求和、化为有理数恒等式, 再将可逆分母映射到模数域。